

AWS : conteneurs, serverless, données et observabilité

Théorie, fonctionnement interne et mise en pratique dans la console et avec AWS CLI v2.

ECS

Conteneurs managés

EKS

Kubernetes managé

Lambda

Fonctions serverless

RDS

Bases relationnelles

DynamoDB

NoSQL serverless

Monitoring

CloudWatch, X-Ray,
CloudTrail

Support de formation, version 1.1, octobre 2026. Fil rouge : la boutique en ligne UtopiosShop. Région des exemples : eu-west-3 (Paris).

Étape 0 : préparer le poste

Compte, droits IAM, AWS CLI v2, variables d'environnement, réseau du lab, rappels IAM et VPC.

1. Amazon ECS

Clusters, task definitions, services, Fargate, déploiements, auto scaling.

2. Amazon EKS

Control plane managé, nœuds, accès, Pod Identity, réseau, add-ons, mises à niveau.

3. AWS Lambda

Cycle de vie, modes d'invocation, concurrence, versions et alias, tarification.

4. Amazon RDS

Moteurs, Multi-AZ, réplicas, sauvegardes, sécurité, Aurora, RDS Proxy.

5. Amazon DynamoDB

Clés, partitions, capacité, index, Query et Scan, Streams, TTL, modélisation.

6. Monitoring

CloudWatch (métriques, alarmes, logs), X-Ray, Container Insights, CloudTrail, EventBridge.

Synthèse

Aide au choix, nettoyage complet du lab, questions de révision, ressources.

Dans chaque module

Concepts, schémas de fonctionnement numérotés, détails, puis un exemple dans la console et le même en CLI.

Navigation : flèches gauche et droite ou barre d'espace, **M** pour le sommaire, **F** pour le plein écran. Le bouton « Copier » de chaque bloc copie les commandes.

Introduction

Étape 0 : ce qu'il faut avant de commencer

Prérequis	Détail	Vérification
Compte AWS	Compte personnel ou bac à sable de formation, avec facturation active. La plupart des ressources du lab sont payantes à l'heure.	Connexion à <code>console.aws.amazon.com</code>
Identité IAM	Utilisateur IAM Identity Center (SSO) ou utilisateur IAM avec clés d'accès. Droits d'administration sur le compte du lab (<code>AdministratorAccess</code>).	<code>aws sts get-caller-identity</code>
AWS CLI v2	Version 2 obligatoire (les commandes <code>aws logs tail</code> , <code>aws configure sso</code> n'existent pas en v1).	<code>aws --version</code> affiche <code>aws-cli/2.x</code>
Docker ou Podman	Uniquement pour construire et pousser une image vers ECR (module ECS).	<code>docker version</code>
kubectl et eksctl	Uniquement pour le module EKS. Installation détaillée dans le module.	<code>kubectl version --client</code> , <code>eksctl version</code>
jq, zip, psql	jq pour lire le JSON, zip pour emballer Lambda, psql pour tester RDS PostgreSQL.	<code>jq --version</code> , <code>psql --version</code>
Terminal	bash ou zsh (macOS, Linux, WSL2 sous Windows). Toutes les commandes du support sont écrites pour ces shells.	<code>echo \$SHELL</code>

Coûts. EKS (control plane + nœuds), RDS et les load balancers sont facturés tant qu'ils existent. Chaque module se termine par un bloc « Nettoyage ». Le module Monitoring réutilise `shop-hello` et `shop-pg` (et, en option, `shop-cluster` et `shop-eks`) : lancez les nettoyages après lui si vous l'enchaînez.

Étape 0 : installer et configurer AWS CLI v2

\$ Installation selon le système

```
# macOS
curl -sO https://awscli.amazonaws.com/AWSCLIV2.pkg
sudo installer -pkg AWSCLIV2.pkg -target /

# Linux x86_64 (ou WSL2)
curl -s https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip \
  -o awscliv2.zip
unzip -q awscliv2.zip && sudo ./aws/install

# Windows (PowerShell en administrateur)
# msixexec.exe /i https://awscli.amazonaws.com/AWSCLIV2.msi

aws --version
```

\$ Authentification : deux possibilités

```
# A. IAM Identity Center (recommandé)
aws configure sso
aws sso login --profile formation
export AWS_PROFILE=formation

# B. Clés d'accès d'un utilisateur IAM
aws configure
#   AWS Access Key ID      : (clé fournie)
#   AWS Secret Access Key : (secret fourni)
#   Default region name   : eu-west-3
#   Default output format : json

# Contrôle : compte, utilisateur, ARN
aws sts get-caller-identity
```

Les identifiants sont stockés dans `~/.aws/config` et `~/.aws/credentials`.
Ne les versionnez jamais dans Git.

Étape 0 : variables et réseau utilisés dans tout le support

Les exemples réutilisent ces variables. Gardez le même terminal, ou relancez ce bloc à chaque nouvelle session.

\$ À exécuter une fois par terminal

```
export AWS_REGION=eu-west-3
export AWS_DEFAULT_REGION=$AWS_REGION
export AWS_PAGER="" # désactive le pager : la sortie s'affiche directement
export AWS_ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)

# Réseau : on utilise le VPC par défaut de la région (sous-réseaux publics dans chaque AZ)
export VPC_ID=$(aws ec2 describe-vpcs --filters Name=isDefault,Values=true \
  --query 'Vpcs[0].VpcId' --output text)
export SUBNET_IDS=$(aws ec2 describe-subnets --filters Name=vpc-id,Values=$VPC_ID \
  --query 'Subnets[].SubnetId' --output text | tr '\t' ',')

echo "Compte=$AWS_ACCOUNT_ID VPC=$VPC_ID Subnets=$SUBNET_IDS"
```

VPC_ID vaut « None » ? Le VPC par défaut a été supprimé. Recréez-le avec `aws ec2 create-default-vpc`, puis relancez le bloc.

En production, on n'utilise pas le VPC par défaut : sous-réseaux privés pour les tâches, les nœuds et les bases, sous-réseaux publics pour les load balancers uniquement.

Rappels indispensables : région, VPC, IAM

Régions et zones de disponibilité

- Une **région** (eu-west-3) regroupe au moins 3 **AZ** isolées physiquement.
- La haute disponibilité se construit en répartissant sur plusieurs AZ : tâches ECS, nœuds EKS, RDS Multi-AZ.
- Lambda et DynamoDB sont multi-AZ par conception.

VPC, sous-réseaux, security groups

- Un **sous-réseau** appartient à une seule AZ. Public s'il a une route vers une Internet Gateway.
- Un **security group** est un pare-feu à états attaché à une interface réseau : on autorise des flux entrants, tout le reste est refusé.
- Un SG peut référencer un autre SG comme source.

Rôles IAM

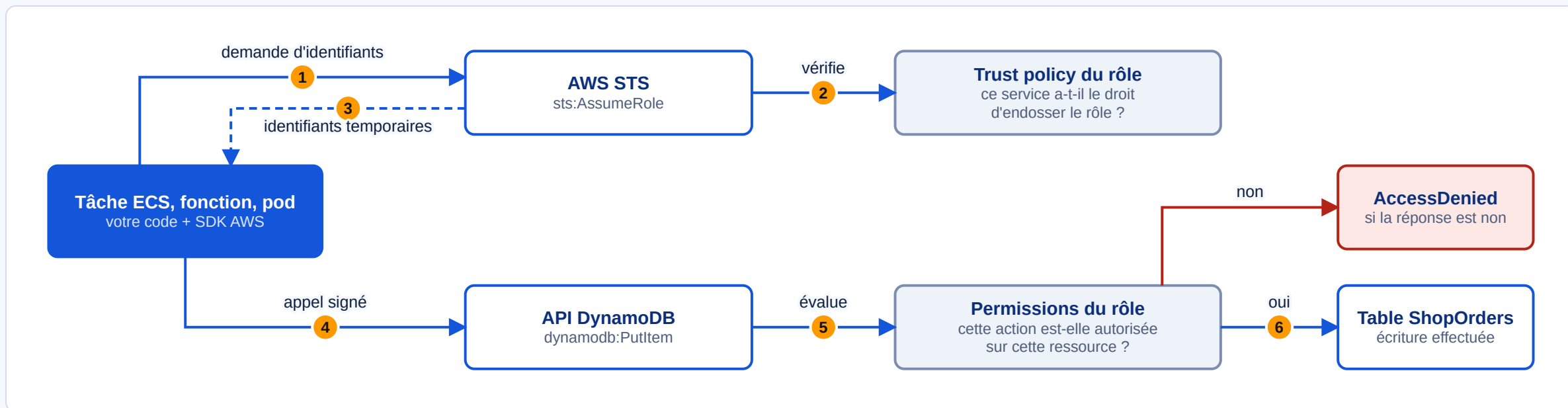
- Un rôle a une **trust policy** (qui peut l'endosser : un service, un compte, un pod) et des **permissions** (ce qu'il peut faire).
- Les services n'utilisent jamais de clés statiques : ils endossent un rôle et reçoivent des identifiants temporaires via STS.

`{}` Trust policy : le service ECS peut endosser ce rôle (même principe pour `lambda.amazonaws.com`, `pods.eks.amazonaws.com...`)

```
{ "Version": "2012-10-17",  
  "Statement": [ { "Effect": "Allow",  
                    "Principal": { "Service": "ecs-tasks.amazonaws.com" },  
                    "Action": "sts:AssumeRole" } ] }
```

Responsabilité partagée. AWS sécurise l'infrastructure et les services managés. Vous restez responsable de la configuration : droits IAM, règles réseau, chiffrement, sauvegardes, code.

Schéma : comment un service obtient ses droits



1. Au démarrage, le SDK AWS du code demande des identifiants pour le rôle attaché à la tâche, à la fonction ou au pod.

2. STS lit la trust policy du rôle : le service demandeur doit y figurer comme principal.

3. STS renvoie des identifiants temporaires (clé, secret, jeton), valables quelques heures et renouvelés automatiquement.

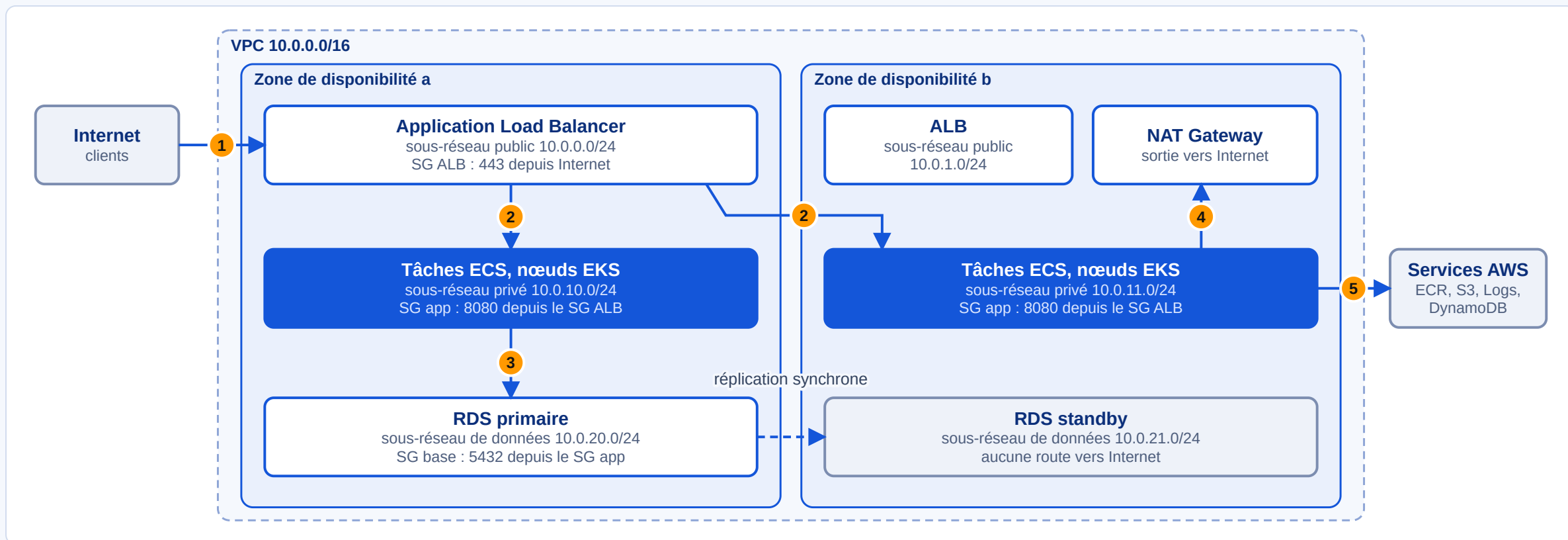
4. Chaque appel d'API est signé avec ces identifiants.

5. Le service appelé demande à IAM si les permissions du rôle autorisent cette action sur cette ressource.

6. Oui : l'action est exécutée. Non, ou rien de prévu : `AccessDenied`. Tout ce qui n'est pas autorisé est refusé.

Ce mécanisme est le même partout : **task role** pour ECS, **rôle d'exécution** pour Lambda, **Pod Identity** pour EKS. Aucune clé d'accès n'est stockée dans le code, l'image ou les variables d'environnement.

Schéma : le réseau type en production



1. Les clients n'atteignent que le load balancer, seul composant placé en sous-réseau public.
2. L'ALB répartit vers les tâches ou les pods des deux zones ; leur security group n'accepte que l'ALB.
3. L'application joint la base ; le security group de la base n'accepte que celui de l'application.
4. Pour sortir vers Internet (API externes, mises à jour), le trafic passe par la NAT Gateway.
5. Vers les services AWS : par la NAT Gateway, ou sans Internet par des VPC endpoints.

Dans le lab, tout est simplifié : VPC par défaut, sous-réseaux publics, IP publiques. Ce schéma est la cible en production.

Où se situent les six services

Plus on va vers la droite, plus AWS gère de couches à votre place, et moins vous contrôlez l'environnement d'exécution.



Données

- **RDS** : SQL, schéma, jointures, transactions. On dimensionne une instance (ou Aurora Serverless).
- **DynamoDB** : clé-valeur et documents, latence en millisecondes à toute échelle, modélisation guidée par les requêtes.

Observabilité, transverse à tout

- **CloudWatch** : métriques, logs, alarmes, tableaux de bord.
- **X-Ray** et Application Signals : traces distribuées.
- **CloudTrail** : journal d'audit des appels d'API. **EventBridge** : réaction aux événements.

Fil rouge UtopiosShop. Le front web tourne sur ECS puis EKS, une fonction Lambda traite les commandes, le catalogue est dans RDS PostgreSQL, les commandes dans DynamoDB, et le tout est supervisé avec CloudWatch.

Module 1

Amazon ECS

Elastic Container Service : l'orchestrateur de conteneurs natif d'AWS.

- Concepts : cluster, task definition, tâche, service
- Modes de calcul : Fargate, EC2, Managed Instances
- Rôles IAM, réseau et exposition
- Déploiements et auto scaling
- Exemple console puis CLI complet
- Bonnes pratiques et nettoyage

ECS planifie et maintient en vie des conteneurs sur une capacité de calcul. Vous décrivez **quoi** exécuter (image, CPU, mémoire, ports) et **combien** d'exemplaires ; ECS place les conteneurs, les remplace s'ils tombent et les branche au load balancer.

- **Control plane gratuit** : on ne paie que la capacité (Fargate à la seconde, ou instances EC2).
- **Intégration native** : IAM par tâche, ALB/NLB, CloudWatch Logs, Secrets Manager, ECR, Auto Scaling.
- **API propre à AWS** : pas de Kubernetes, pas de manifestes YAML, pas de cluster à mettre à niveau.
- Images au format OCI/Docker, Linux ou Windows, x86_64 ou ARM64 (Graviton).

Quand choisir ECS

- Équipe centrée sur AWS, sans besoin de l'écosystème Kubernetes.
- API web, workers, traitements batch en conteneurs.
- Volonté de limiter l'exploitation au strict minimum (avec Fargate).

Quand regarder ailleurs

- Portabilité multi-cloud ou besoin d'opérateurs, CRD, Helm : EKS.
- Traitements courts déclenchés par événements : Lambda.

Vocabulaire ECS	Équivalent Kubernetes	Remarque
Task definition	Spécification de pod (template)	Versionnée par révisions immuables.
Tâche	Pod	Un ou plusieurs conteneurs qui partagent le réseau.
Service	Deployment + Service	Maintient le nombre de tâches et les relie au load balancer.
Capacity provider	Node pool, autoscaler de nœuds	Fargate, Spot ou groupe d'instances EC2.

Les concepts clés d'ECS

Cluster

Regroupement logique de services et de tâches, dans une région. Il porte la capacité : Fargate, instances EC2, ou les deux via des *capacity providers*.

Task definition

Le plan d'une application, en JSON : 1 à 10 conteneurs, images, CPU et mémoire, ports, variables, secrets, volumes, logs, rôles IAM. Chaque modification crée une **révision** immuable (`shop-web:3`).

Tâche (task)

Une instance en cours d'exécution d'une task definition. C'est l'équivalent d'un pod Kubernetes : les conteneurs d'une tâche partagent réseau et cycle de vie.

Service

Maintient N tâches en permanence, les remplace si elles échouent, les enregistre dans un target group, orchestre les déploiements. Pour un traitement ponctuel, on lance une tâche seule (`run-task`).

Capacity provider

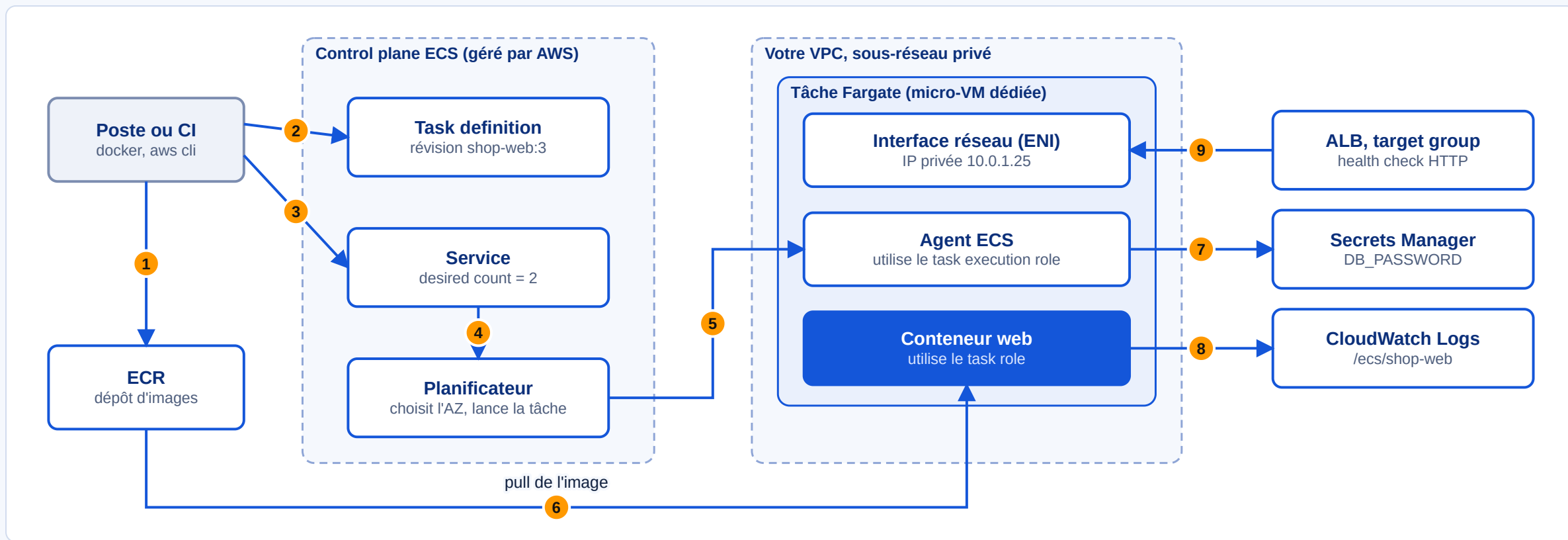
Stratégie de capacité : `FARGATE`, `FARGATE_SPOT`, ou un Auto Scaling Group EC2. On pondère avec *base* et *weight* (exemple : 2 tâches à la demande, le reste en Spot).

Agent de conteneurs

Processus qui dialogue avec le control plane ECS : démarre les conteneurs, remonte leur état. Vous le gérez sur EC2 ; il est invisible sur Fargate.

Chaîne à retenir : **image** (ECR) → **task definition** (révision) → **service** (N tâches) → **cluster** (capacité).

Schéma : du push de l'image à la tâche en service

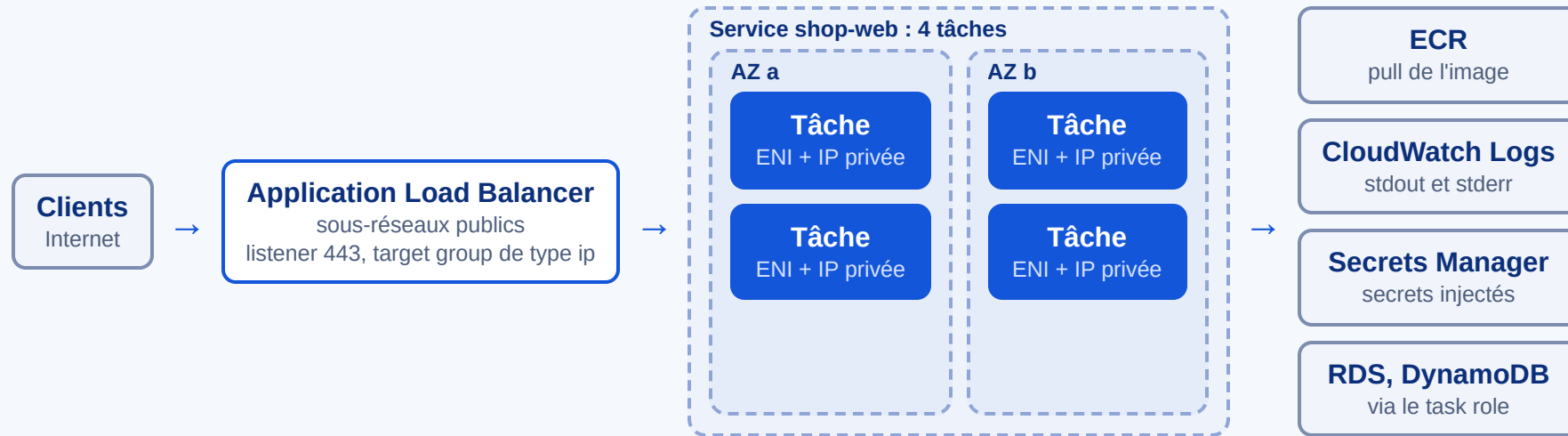


1. L'image est construite puis poussée dans ECR.
2. La task definition est enregistrée : une nouvelle révision référence l'image.
3. Le service est créé ou mis à jour avec cette révision.

4. Le service constate l'écart entre tâches voulues et tâches actives.
5. Le planificateur choisit une AZ et lance la tâche sur Fargate.
6. L'agent tire l'image depuis ECR avec le task execution role.

7. Il lit les secrets référencés et les injecte dans le conteneur.
8. Le conteneur démarre ; stdout et stderr partent vers CloudWatch Logs.
9. L'IP de la tâche est enregistrée dans le target group ; après un health check réussi, l'ALB lui envoie du trafic.

Architecture type d'un service ECS sur Fargate



Répartition

Le service étale les tâches sur les AZ des sous-réseaux fournis. La perte d'une AZ laisse la moitié de la capacité en service, et ECS relance les tâches manquantes ailleurs.

Santé

Deux contrôles : le *health check* du target group (HTTP) et, en option, le `healthCheck` du conteneur. Une tâche jugée malsaine est arrêtée et remplacée.

Sortie vers Internet

En sous-réseau privé, il faut une NAT Gateway ou des VPC endpoints (ECR, S3, CloudWatch Logs, Secrets Manager) pour tirer l'image et écrire les logs.

Amazon ECS

Modes de calcul : où tournent les tâches

Mode	Principe	Points forts	Limites
Fargate	Chaque tâche tourne dans une micro-VM dédiée gérée par AWS. Facturation au vCPU et au Go par seconde.	Aucun serveur à patcher, isolation forte, démarrage simple, Fargate Spot jusqu'à 70 % moins cher.	Pas de GPU, pas de mode privilégié, pas de DaemonSet ; coût unitaire plus élevé à forte charge constante.
EC2	Vos instances (Auto Scaling Group) rejoignent le cluster via l'agent ECS. Plusieurs tâches par instance.	Contrôle total (type d'instance, GPU, AMI), meilleur coût à charge stable, instances réservées et Spot.	À vous l'AMI, les correctifs, le dimensionnement et le <i>bin packing</i> .
Managed Instances	ECS provisionne et gère lui-même des instances EC2 dans votre compte (depuis fin 2025).	Accès aux capacités EC2 (GPU, familles spécifiques) sans gérer le parc ; correctifs automatiques.	Frais de gestion en plus du prix EC2 ; instances remplacées régulièrement.
ECS Anywhere	Serveurs hors AWS (on-premises) enregistrés comme capacité externe.	Même control plane pour le cloud et le datacenter.	Pas de load balancer AWS ni de mode awsvpc.

Tailles Fargate valides (vCPU → mémoire)

0,25 → 0,5 à 2 Go ; 0,5 → 1 à 4 Go ; 1 → 2 à 8 Go ; 2 → 4 à 16 Go ; 4 → 8 à 30 Go ; 8 → 16 à 60 Go ; 16 → 32 à 120 Go. Une combinaison invalide fait échouer l'enregistrement de la task definition.

Règle pratique. Commencer en Fargate. Passer en EC2 ou Managed Instances quand un besoin précis l'impose (GPU, coût à grande échelle, accès à l'hôte).

`taskdef.json` (Fargate)

```
{
  "family": "shop-web",
  "requiresCompatibilities": ["FARGATE"],
  "networkMode": "awsvpc",
  "cpu": "256", "memory": "512",
  "runtimePlatform": { "cpuArchitecture": "X86_64",
    "operatingSystemFamily": "LINUX" },
  "executionRoleArn": "arn:aws:iam::111122223333:role/ecsTaskExecutionRole",
  "taskRoleArn": "arn:aws:iam::111122223333:role/shop-web-task",
  "containerDefinitions": [ {
    "name": "web",
    "image": "public.ecr.aws/nginx/nginx:stable",
    "essential": true,
    "portMappings": [ { "containerPort": 80, "protocol": "tcp" } ],
    "environment": [ { "name": "STAGE", "value": "dev" } ],
    "secrets": [ { "name": "DB_PASSWORD", "valueFrom":
      "arn:aws:secretsmanager:eu-west-3:111122223333:secret:shop/db" } ],
    "logConfiguration": { "logDriver": "awslogs", "options": {
      "awslogs-group": "/ecs/shop-web", "awslogs-region": "eu-west-3",
      "awslogs-stream-prefix": "web" } }
  } ]
}
```

- **family** : nom logique ; chaque enregistrement crée `family:révision`.
- **cpu / memory** au niveau tâche : obligatoires en Fargate (256 = 0,25 vCPU).
- **awsvpc** : chaque tâche reçoit sa propre interface réseau et son security group.
- **essential** : si ce conteneur s'arrête, toute la tâche s'arrête.
- **secrets** : la valeur est lue dans Secrets Manager ou SSM au démarrage, jamais écrite dans la définition.
- **awslogs** : stdout et stderr partent dans CloudWatch Logs.

Tag d'image. Évitez `latest` : utilisez un tag immuable ou un digest, sinon un rollback ne garantit pas de revenir à l'image précédente.

Task execution role

Endossé par **l'agent ECS** pour préparer la tâche, avant que votre code ne démarre.

- Tirer l'image depuis ECR.
- Créer les flux et écrire dans CloudWatch Logs.
- Lire les secrets référencés dans `secrets`.

Politique gérée : `AmazonECSTaskExecutionRolePolicy`, à compléter pour les secrets.

Task role

Endossé par **votre application** pendant l'exécution. Le SDK AWS récupère automatiquement des identifiants temporaires.

- Lire un bucket S3, écrire dans DynamoDB, publier sur SQS.
- Un rôle par service, au moindre privilège.
- Nécessaire aussi pour ECS Exec (droits `ssmmessages`).

Symptôme	Rôle en cause	Explication
CannotPullContainerError	Execution role (ou réseau)	Pas de droit ECR, ou pas de route vers ECR depuis le sous-réseau.
ResourceInitializationError (secrets)	Execution role	Droit <code>secretsmanager:GetSecretValue</code> ou <code>kms:Decrypt</code> manquant.
AccessDenied dans les logs de l'application	Task role	Le code appelle une API AWS non autorisée.

Les deux rôles ont la même trust policy : le principal `ecs-tasks.amazonaws.com`. Un troisième rôle, dit d'infrastructure, sert à ECS pour gérer des ressources en votre nom (volumes EBS, Managed Instances).

Modes réseau

- **awsvpc** (seul mode Fargate) : une ENI et une IP du VPC par tâche, security group par tâche.
- **bridge, host** (EC2 uniquement) : réseau Docker de l'hôte, ports dynamiques.

Exposition externe

- **ALB** (HTTP/HTTPS, routage par hôte ou chemin) ou **NLB** (TCP/UDP).
- Target group de type **ip** avec awsvpc : ECS enregistre et retire les IP des tâches.
- `healthCheckGracePeriodSeconds` laisse le temps à l'application de démarrer.

Communication entre services

- **Service Connect** : nom court (`http://catalog:8080`), proxy géré par ECS, métriques et reprises automatiques. C'est l'option recommandée.
- **Service discovery** (Cloud Map) : enregistrements DNS, sans proxy.
- **ALB interne** : quand il faut un point d'entrée stable et partagé.

Security groups en chaîne

- SG de l'ALB : 443 depuis Internet.
- SG des tâches : port applicatif depuis le **SG de l'ALB** uniquement.
- SG de la base : 5432 depuis le **SG des tâches** uniquement.

assignPublicIp. En sous-réseau public sans NAT, une tâche Fargate doit avoir `assignPublicIp=ENABLED` pour tirer son image. C'est acceptable en lab, pas en production.

Rolling update (par défaut)

- `minimumHealthyPercent` (100) : capacité minimale conservée pendant le déploiement.
- `maximumPercent` (200) : plafond de tâches pendant la bascule.
- Avec 100/200 et 4 tâches : ECS lance 4 nouvelles tâches, attend qu'elles soient saines, puis arrête les 4 anciennes.

Circuit breaker

Si les nouvelles tâches échouent à répétition, le déploiement passe en `FAILED` et, avec `rollback=true`, ECS revient seul à la dernière révision stable.

Blue/green, linéaire, canary

Stratégies natives du service ECS : deux flottes en parallèle, bascule du trafic au niveau du load balancer, période d'observation (*bake time*), retour arrière immédiat.

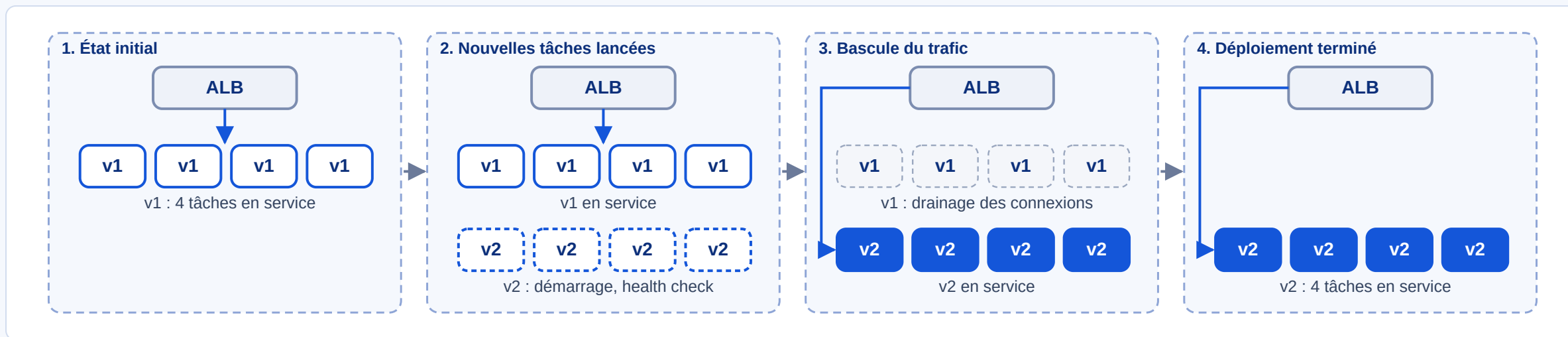
Service Auto Scaling

- **Target tracking** : « maintenir le CPU moyen à 60 % ». ECS ajoute ou retire des tâches. C'est le mode à privilégier.
- **Step scaling** : paliers déclenchés par des alarmes CloudWatch.
- **Scheduled** : capacité planifiée (ouverture de boutique, soldes).
- Métriques usuelles : CPU, mémoire, requêtes par cible de l'ALB, longueur d'une file SQS.

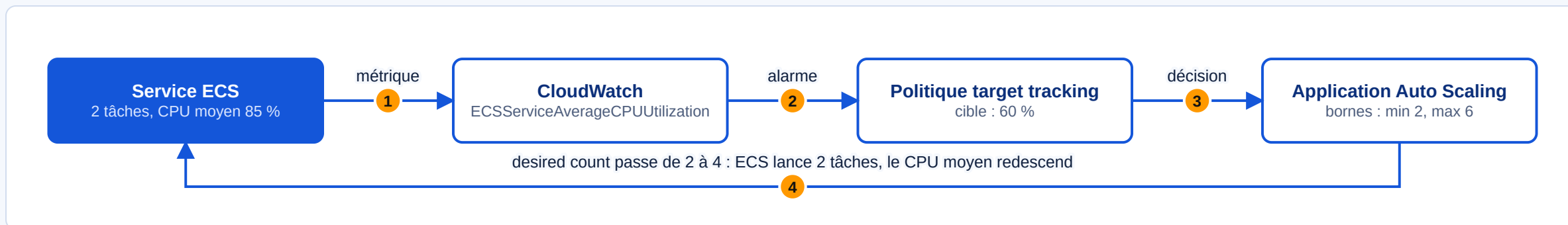
Deux niveaux en mode EC2. Le service ajuste le nombre de tâches ; le capacity provider ajuste le nombre d'instances (*managed scaling*). En Fargate, seul le premier niveau existe.

À activer systématiquement. Circuit breaker avec rollback, health check du load balancer, et au moins 2 tâches réparties sur 2 AZ.

Schéma : rolling update et boucle d'auto scaling



Rolling update (`minimumHealthyPercent=100` , `maximumPercent=200`) : jamais moins de 4 tâches en service. Si les tâches v2 échouent à l'étape 2, le circuit breaker arrête tout et les v1 restent.



Auto scaling. La boucle tourne en continu : quand le CPU repasse durablement sous la cible, le nombre de tâches redescend, dans les bornes fixées.

Console : créer un cluster et un service

aws

Elastic Container Service > Clusters > Create cluster

Cluster name

shop-cluster

Infrastructure

☒ AWS Fargate (serverless)

Monitoring

Container Insights with enhanced observability ▼

Create

aws

Task definitions > Create new task definition

Task definition family

shop-web

Launch type

☒ AWS Fargate

CPU / Memory

.25 vCPU / .5 GB ▼

Task execution role

ecsTaskExecutionRole ▼

Container name / Image URI

web / public.ecr.aws/nginx/nginx:stable

Container port

80 / TCP / HTTP

Log collection

☒ Use log collection (CloudWatch)

Create

aws

Clusters > shop-cluster > Services > Create

Deployment configuration

Task definition family

shop-web (LATEST) ▼

Service name

shop-web

Desired tasks

2

Deployment failure detection

☒ Use the Amazon ECS deployment circuit breaker, rollback on failures

Networking

VPC / Subnets

VPC par défaut / tous les sous-réseaux ▼

Security group

☒ Create new : HTTP 80 depuis Anywhere

Public IP

☒ Turned on

Load balancing (optionnel)

Load balancer type

Application Load Balancer ▼

Create

Ensuite : onglet **Tasks** pour l'état des tâches, **Logs** pour la sortie des conteneurs, **Deployments** pour suivre la bascule, **Health and metrics** pour CPU et mémoire. Les libellés de la console évoluent : fiez-vous à la structure, pas au mot exact.

\$ Prérequis : variables de l'étape 0 chargées dans le terminal

1. Rôle d'exécution des tâches (une seule fois par compte)

```
cat > ecs-trust.json <<'EOF'
{"Version":"2012-10-17","Statement":[{"Effect":"Allow",
  "Principal":{"Service":"ecs-tasks.amazonaws.com"},"Action":["sts:AssumeRole"]}]}
EOF
aws iam create-role --role-name ecsTaskExecutionRole \
  --assume-role-policy-document file://ecs-trust.json
aws iam attach-role-policy --role-name ecsTaskExecutionRole \
  --policy-arn arn:aws:iam::aws:policy/service-role/AmazonECSTaskExecutionRolePolicy
```

2. Cluster (Fargate disponible d'office) et groupe de logs

```
aws ecs create-cluster --cluster-name shop-cluster \
  --settings name=containerInsights,value=enhanced
aws logs create-log-group --log-group-name /ecs/shop-web
```

3. Task definition : le heredoc sans guillemets remplace les variables

```
cat > taskdef.json <<EOF
{"family":"shop-web","requiresCompatibilities":["FARGATE"],"networkMode":"awsvpc",
  "cpu":"256","memory":"512",
  "executionRoleArn":"arn:aws:iam::${AWS_ACCOUNT_ID}:role/ecsTaskExecutionRole",
  "containerDefinitions":[{"name":"web","image":"public.ecr.aws/nginx/nginx:stable",
    "essential":true,"portMappings":[{"containerPort":80,"protocol":"tcp"}],
    "logConfiguration":{"logDriver":"awslogs","options":{"awslogs-group":"/ecs/shop-web",
      "awslogs-region":"${AWS_REGION}","awslogs-stream-prefix":"web"}}}]}
EOF
aws ecs register-task-definition --cli-input-json file://taskdef.json \
  --query 'taskDefinition.taskDefinitionArn'
```

\$ Créer le service et récupérer l'adresse d'une tâche

```
# 4. Security group : HTTP entrant (lab uniquement)
export WEB_SG=$(aws ec2 create-security-group --group-name shop-web-sg \
--description "HTTP vers shop-web" --vpc-id $VPC_ID --query GroupId --output text)
aws ec2 authorize-security-group-ingress --group-id $WEB_SG \
--protocol tcp --port 80 --cidr 0.0.0.0/0

# 5. Service : 2 tâches Fargate, circuit breaker avec rollback
aws ecs create-service --cluster shop-cluster --service-name shop-web \
--task-definition shop-web --desired-count 2 --launch-type FARGATE \
--network-configuration \
"awsvpcConfiguration={subnets=[$SUBNET_IDS],securityGroups=[$WEB_SG],assignPublicIp=ENABLED}" \
--deployment-configuration "deploymentCircuitBreaker={enable=true,rollback=true}"

# 6. Attendre la stabilisation (toutes les tâches RUNNING et saines)
aws ecs wait services-stable --cluster shop-cluster --services shop-web

# 7. Retrouver l'IP publique d'une tâche : tâche -> interface réseau -> IP
TASK_ARN=$(aws ecs list-tasks --cluster shop-cluster --service-name shop-web \
--query 'taskArns[0]' --output text)
ENI_ID=$(aws ecs describe-tasks --cluster shop-cluster --tasks $TASK_ARN \
--query "tasks[0].attachments[0].details[?name=='networkInterfaceId'].value" --output text)
TASK_IP=$(aws ec2 describe-network-interfaces --network-interface-ids $ENI_ID \
--query 'NetworkInterfaces[0].Association.PublicIp' --output text)
curl -sI http://$TASK_IP | head -n 1          # attendu : HTTP/1.1 200 OK
```

Sans load balancer, l'IP change à chaque remplacement de tâche. En production, on ajoute `--load-balancers` `targetGroupArn=...,containerName=web,containerPort=80` et l'on passe par le nom DNS de l'ALB.

\$ Observer, déployer, mettre à l'échelle

```
# État du service et de ses déploiements
aws ecs describe-services --cluster shop-cluster \
  --services shop-web --query \
  'services[0].{voulu:desiredCount,actif:runningCount,
  deploiements:deployments[].[status,rolloutState]}'

# Derniers événements (placement, santé, erreurs)
aws ecs describe-services --cluster shop-cluster \
  --services shop-web \
  --query 'services[0].events[:5].message'

# Logs des conteneurs en continu
aws logs tail /ecs/shop-web --follow --since 10m

# Déployer une nouvelle révision de la task definition
aws ecs register-task-definition \
  --cli-input-json file://taskdef.json
aws ecs update-service --cluster shop-cluster \
  --service shop-web --task-definition shop-web

# Redéployer la même révision (nouveau pull d'image)
aws ecs update-service --cluster shop-cluster \
  --service shop-web --force-new-deployment

# Passer à 4 tâches
aws ecs update-service --cluster shop-cluster \
  --service shop-web --desired-count 4
```

\$ Auto scaling : CPU moyen maintenu à 60 %

```
aws application-autoscaling register-scalable-target \
  --service-namespace ecs \
  --resource-id service/shop-cluster/shop-web \
  --scalable-dimension ecs:service:DesiredCount \
  --min-capacity 2 --max-capacity 6

aws application-autoscaling put-scaling-policy \
  --service-namespace ecs \
  --resource-id service/shop-cluster/shop-web \
  --scalable-dimension ecs:service:DesiredCount \
  --policy-name shop-web-cpu60 \
  --policy-type TargetTrackingScaling \
  --target-tracking-scaling-policy-configuration \
  '{"TargetValue":60.0,"PredefinedMetricSpecification":
  {"PredefinedMetricType":"ECSServiceAverageCPUUtilization"},
  "ScaleOutCooldown":60,"ScaleInCooldown":120}'
```

ECS Exec (shell dans un conteneur) : il faut un task role avec les droits `ssmmessages`, le service créé avec `--enable-execute-command` et le plugin Session Manager installé sur le poste. Commande : `aws ecs execute-command --cluster shop-cluster --task $TASK_ARN --container web --interactive --command "/bin/sh"`.

ECR : publier sa propre image

\$ Prérequis : Docker démarré, un Dockerfile dans le dossier courant

```
# 1. Créer le dépôt (scan des vulnérabilités à chaque push, tags immuables)
aws ecr create-repository --repository-name shop-web \
  --image-scanning-configuration scanOnPush=true \
  --image-tag-mutability IMMUTABLE

export ECR=$AWS_ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com

# 2. Authentifier Docker auprès d'ECR (jeton valable 12 heures)
aws ecr get-login-password | \
  docker login --username AWS --password-stdin $ECR

# 3. Construire pour l'architecture de la task definition, puis pousser
#   (sur un Mac Apple Silicon, --platform est indispensable pour X86_64)
docker build --platform linux/amd64 -t $ECR/shop-web:1.0.0 .
docker push $ECR/shop-web:1.0.0

# 4. Vérifier l'image et le résultat du scan
aws ecr describe-images --repository-name shop-web --output table \
  --query 'imageDetails[][imageTags[0],imageSizeInBytes,imagePushedAt]'
aws ecr describe-image-scan-findings --repository-name shop-web \
  --image-id imageTag=1.0.0 --query 'imageScanFindings.findingSeverityCounts'
```

Ce qu'apporte ECR

- Registre privé, chiffré, contrôlé par IAM.
- Scan de vulnérabilités (basique, ou avancé avec Inspector).
- **Lifecycle policies** : purge automatique des anciennes images.
- Réplication entre régions et entre comptes.
- **Pull through cache** : miroir de Docker Hub, Quay, GHCR.

Erreur classique. « exec format error » au démarrage de la tâche : l'image est en ARM64 (Mac M-series) alors que la tâche attend du X86_64. Construire avec `--platform` ou déclarer `cpuArchitecture: ARM64`.

Dans la task definition, remplacer l'image par `$ECR/shop-web:1.0.0`.

Bonnes pratiques

- Une task definition par service, un conteneur principal, des *sidecars* seulement s'ils partagent le cycle de vie.
- Un task role par service, au moindre privilège. Secrets dans Secrets Manager, jamais en variable en clair.
- Tags d'image immuables, circuit breaker avec rollback, health checks réalistes.
- Tâches en sous-réseaux privés, exposées par un ALB.
- Fixer la rétention des groupes de logs (par défaut : illimitée, donc facturée sans fin).

Pièges fréquents

- Tâche qui redémarre en boucle : lire `stoppedReason` avec `describe-tasks`.
- Mémoire dépassée : le conteneur est tué (`OutOfMemoryError`, code 137).
- Health check trop strict : les tâches sont tuées avant d'avoir fini de démarrer.

\$ Nettoyage du module ECS (après le module Monitoring si vous le suivez)

```
aws application-autoscaling deregister-scalable-target \
  --service-namespace ecs \
  --resource-id service/shop-cluster/shop-web \
  --scalable-dimension ecs:service:DesiredCount

aws ecs delete-service --cluster shop-cluster \
  --service shop-web --force
aws ecs wait services-inactive --cluster shop-cluster \
  --services shop-web
aws ecs delete-cluster --cluster shop-cluster

# Désenregistrer toutes les révisions de shop-web
for TD in $(aws ecs list-task-definitions \
  --family-prefix shop-web \
  --query 'taskDefinitionArns[]' --output text); do
  aws ecs deregister-task-definition \
    --task-definition $TD --query 'taskDefinition.status'
done

aws logs delete-log-group --log-group-name /ecs/shop-web
# Attendre la libération des interfaces réseau des tâches
sleep 60
aws ec2 delete-security-group --group-id $WEB_SG
# Seulement si le dépôt ECR a été créé
aws ecr delete-repository --repository-name shop-web --force
```

Module 2

Amazon EKS

Elastic Kubernetes Service : un Kubernetes conforme, dont AWS exploite le control plane.

- Control plane managé et data plane
- Options de calcul : node groups, Fargate, Karpenter, Auto Mode
- Accès au cluster et IAM pour les pods
- Réseau, stockage, add-ons
- Versions et mises à niveau
- Exemple console puis CLI, comparaison avec ECS

EKS fournit un cluster Kubernetes certifié conforme : mêmes API, mêmes manifestes, mêmes outils (`kubectl` , Helm, Argo CD) que n'importe quel Kubernetes. AWS exploite l'API server et etcd ; vous gérez les charges de travail et, selon le mode choisi, les nœuds.

- **Control plane** réparti sur 3 AZ, mis à l'échelle et réparé automatiquement, SLA de 99,95 %.
- **Intégrations AWS** : IAM pour l'accès et pour les pods, VPC pour le réseau des pods, ELB, EBS, EFS, CloudWatch.
- **Tarif** : 0,10 \$ par heure et par cluster en support standard, 0,60 \$ en support étendu, plus la capacité de calcul.
- Déclinaisons : EKS dans le cloud, EKS Anywhere et EKS Hybrid Nodes pour l'on-premises.

Ce qu'AWS gère

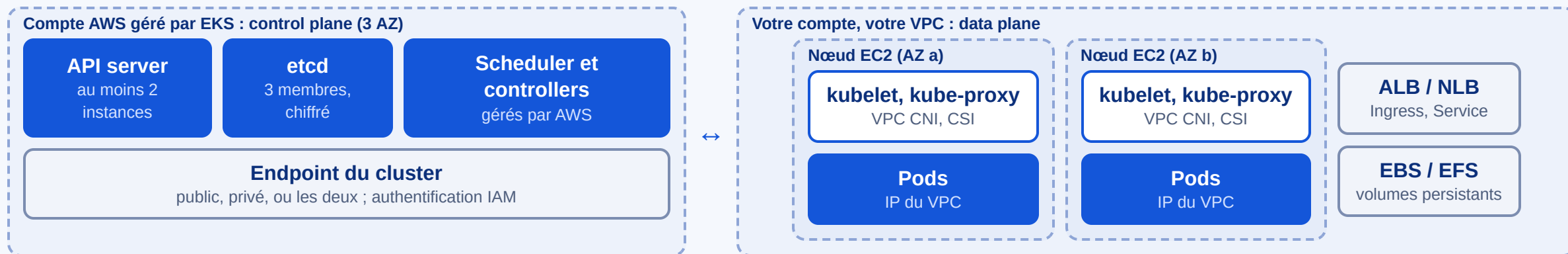
- API server, etcd, scheduler, controller manager.
- Sauvegarde d'etcd, correctifs du control plane.
- Les nœuds aussi, en Auto Mode et sur Fargate.

Ce qui reste à votre charge

- Mises à niveau de version (à déclencher).
- Nœuds, AMI et add-ons hors Auto Mode.
- RBAC, network policies, sécurité des pods.
- Observabilité, sauvegardes applicatives.

Qui gère quoi	Control plane	Nœuds (OS, correctifs, scaling)	Réseau, stockage, load balancing
EKS standard	AWS	Vous (aidé par les managed node groups)	Vous : add-ons et contrôleurs à installer
EKS Auto Mode	AWS	AWS	AWS : intégrés au cluster
EKS sur Fargate	AWS	AWS : pas de nœud visible	Vous, avec des restrictions

Architecture : control plane et data plane



Liaison entre les deux plans

EKS place des interfaces réseau dans vos sous-réseaux pour que l'API server joigne les kubelets (logs, exec, webhooks). Les nœuds joignent l'API par l'endpoint.

Endpoint public ou privé

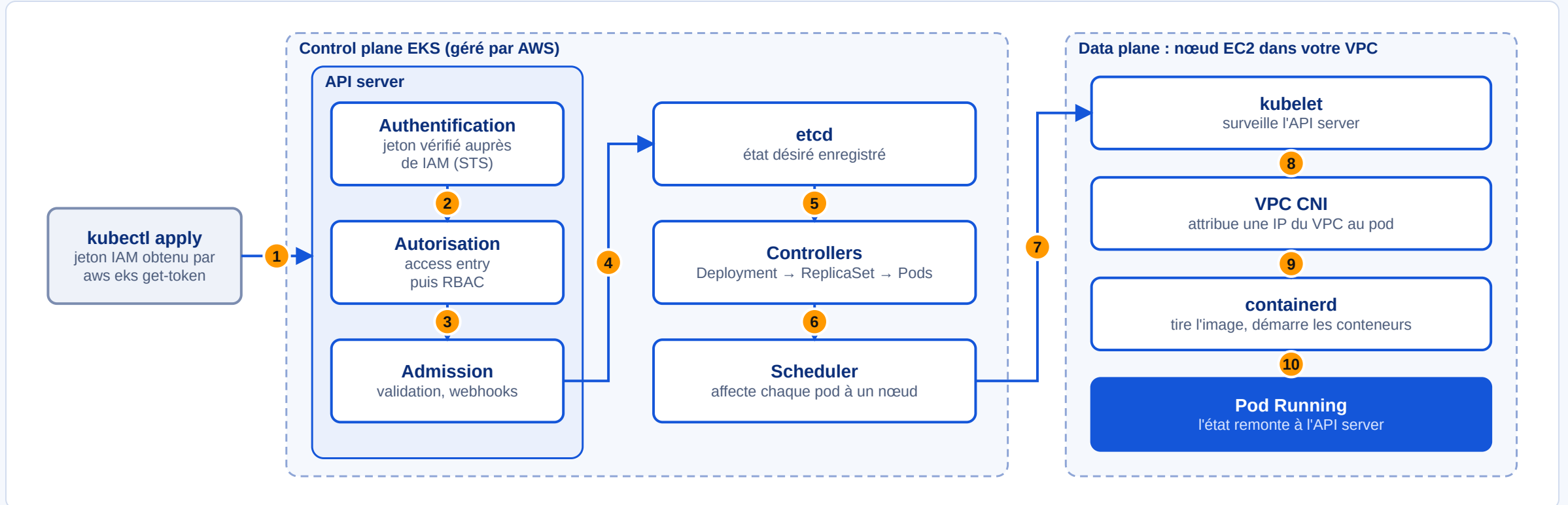
Public par défaut (filtrable par CIDR). En production : endpoint privé, accès via VPN, Direct Connect ou un bastion, éventuellement public restreint.

Sous-réseaux

Au moins 2 AZ. Prévoir large : chaque pod consomme une adresse IP du VPC. Les sous-réseaux sont tagués pour que les load balancers s'y placent.

Journaux du control plane. Cinq types activables vers CloudWatch Logs : `api`, `audit`, `authenticator`, `controllerManager`, `scheduler`. `audit` et `authenticator` sont les plus utiles pour savoir qui a fait quoi dans le cluster.

Schéma : le trajet d'un kubectl apply



1. `kubectl` envoie la requête à l'endpoint du cluster avec un jeton IAM.
2. Authentification : l'API server fait valider le jeton et obtient l'identité IAM.
3. Autorisation : access entry et RBAC décident si l'action est permise.

4. Après l'admission, l'objet Deployment est enregistré dans etcd.
5. Les controllers créent le ReplicaSet puis les Pods, encore sans nœud.
6. Le scheduler affecte chaque pod à un nœud (ressources, AZ, contraintes).

7. Le kubelet du nœud voit qu'un pod lui est attribué.
8. Le VPC CNI lui donne une adresse IP du VPC.
9. containerd tire l'image et démarre les conteneurs.
10. Le pod passe à Running ; le kubelet remonte son état.

Amazon EKS

Options de calcul pour les nœuds

Option	Principe	Idéal pour	À savoir
Managed node groups	EKS crée et met à jour un Auto Scaling Group d'instances EC2 (AMI optimisée EKS, drain avant remplacement).	Le cas général, charge prévisible.	Vous déclenchez les mises à jour d'AMI ; scaling par Cluster Autoscaler ou Karpenter.
Karpenter	Contrôleur qui provisionne la bonne instance au bon moment d'après les pods en attente, puis consolide.	Charges variables, mix Spot et à la demande, optimisation de coût.	À installer et configurer (NodePool, EC2NodeClass).
EKS Auto Mode	AWS gère nœuds, scaling (Karpenter intégré), réseau, stockage et load balancing.	Démarrer vite, réduire l'exploitation.	Frais de gestion par instance ; nœuds immuables, remplacés au plus tard tous les 21 jours.
Fargate	Un pod = une micro-VM dédiée, sélectionné par un <i>Fargate profile</i> (namespace, labels).	Isolation forte, petites charges, jobs.	Pas de DaemonSet, de GPU ni de pod privilégié ; démarrage plus lent.
Nœuds autogérés	Vos propres instances et AMI, que vous rattachez au cluster.	Besoins très spécifiques (OS, noyau).	Tout est à votre charge.
Hybrid Nodes	Serveurs on-premises rattachés à un control plane EKS dans le cloud.	Contraintes de localisation des données ou de latence.	Réseau privé requis vers AWS.

AMI. Les nœuds utilisent Amazon Linux 2023 ou Bottlerocket (OS minimal pensé pour les conteneurs). Amazon Linux 2 n'est plus fourni pour les versions récentes de Kubernetes.

Amazon EKS

Qui peut accéder au cluster : access entries

L'authentification passe toujours par **IAM** (jeton signé produit par `aws eks get-token`, appelé par `kubectl`). L'autorisation se fait ensuite dans Kubernetes.

Access entries (méthode actuelle)

- Une entrée associe un principal IAM (rôle ou utilisateur) au cluster, via l'API EKS.
- On lui attache une **access policy** gérée, à l'échelle du cluster ou de namespaces, ou des groupes Kubernetes pour un RBAC sur mesure.
- Gérable en CLI, Terraform, CloudFormation, sans toucher au cluster.

ConfigMap aws-auth (ancienne méthode)

Correspondance IAM → groupes écrite dans un ConfigMap de `kube-system`. Fragile : une erreur de YAML peut verrouiller le cluster. Dépréciée.

Access policy	Droits
AmazonEKSClusterAdminPolicy	Administrateur du cluster (équivalent cluster-admin).
AmazonEKSAAdminPolicy	Administrateur d'un namespace, RBAC compris.
AmazonEKSEditPolicy	Lecture et écriture des ressources applicatives.
AmazonEKSVIEWPolicy	Lecture seule, sans les Secrets.

authenticationMode	Effet
API	Access entries uniquement (recommandé).
API_AND_CONFIG_MAP	Les deux, pour migrer.
CONFIG_MAP	Historique. Le passage vers API est sans retour.

Créateur du cluster. Le principal IAM qui crée le cluster en devient administrateur. Ajoutez tout de suite une entrée pour un rôle d'administration partagé.

Amazon EKS

IAM pour les pods : IRSA et Pod Identity

Un pod qui appelle S3 ou DynamoDB ne doit pas hériter des droits du nœud. On associe un rôle IAM à un **ServiceAccount**.

	IRSA	EKS Pod Identity
Mécanisme	Fournisseur OIDC du cluster + annotation sur le ServiceAccount	Agent en DaemonSet + association créée par l'API EKS
Trust policy	Propre à chaque cluster (URL OIDC)	Générique : <code>pods.eks.amazonaws.com</code>
Réutilisation du rôle	Modifier la trust policy par cluster	Même rôle pour tous les clusters
Mise en place	Fournisseur OIDC IAM à créer	Add-on <code>eks-pod-identity-agent</code>
Fargate	Pris en charge	Non pris en charge

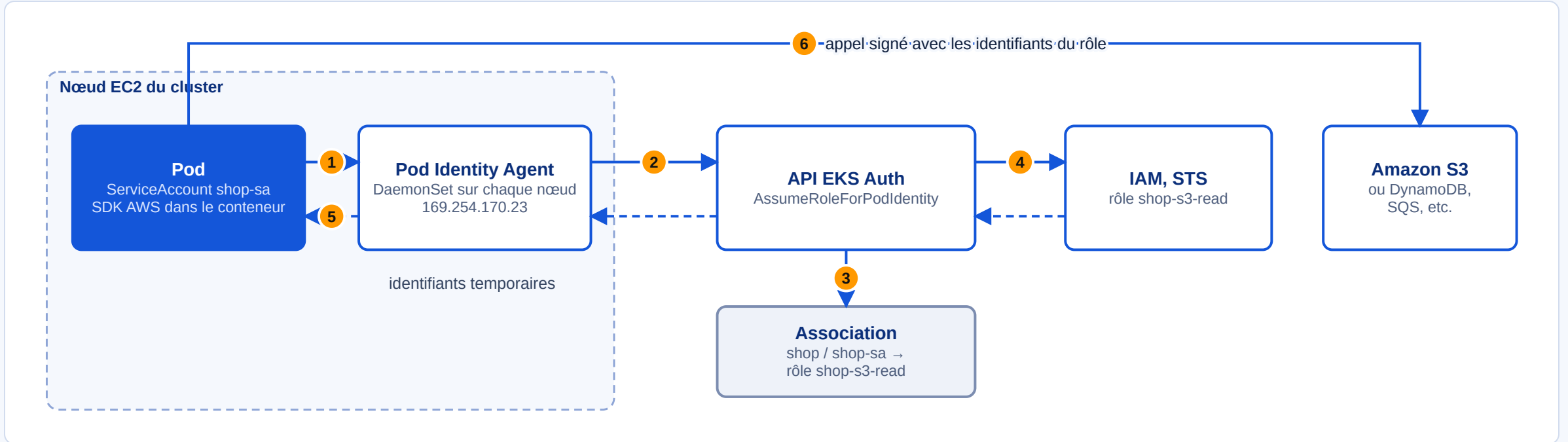
Recommandation. Pod Identity pour les nouveaux clusters sur EC2 ; IRSA pour Fargate et l'existant.

```
{  
  Trust policy d'un rôle pour Pod Identity  
  
  {  
    "Version": "2012-10-17",  
    "Statement": [ {  
      "Effect": "Allow",  
      "Principal": { "Service": "pods.eks.amazonaws.com" },  
      "Action": [ "sts:AssumeRole", "sts:TagSession" ]  
    } ] }  
}
```

```
yaml IRSA : annotation sur le ServiceAccount  
  
apiVersion: v1  
kind: ServiceAccount  
metadata:  
  name: shop-sa  
  namespace: shop  
  annotations:  
    eks.amazonaws.com/role-arn: arn:aws:iam::111122223333:role/shop-s3-read
```

Dans les deux cas, le SDK AWS du conteneur trouve seul ses identifiants temporaires : aucun changement de code.

Schéma : Pod Identity pas à pas



1. Le SDK AWS du conteneur demande des identifiants à l'agent local, en présentant le jeton du ServiceAccount.

2. L'agent transmet la demande à l'API EKS Auth.

3. EKS vérifie qu'une association existe pour ce namespace et ce ServiceAccount.

4. EKS endosse le rôle IAM associé auprès de STS.

5. Des identifiants temporaires reviennent jusqu'au pod ; le SDK les renouvelle seul avant expiration.

6. Le pod appelle S3 avec les seuls droits du rôle, pas ceux du nœud.

Sans association, le pod retombe sur le rôle du nœud : c'est ce qu'on veut éviter. Avec IRSA, le principe est le même, mais le pod échange lui-même son jeton auprès de STS grâce au fournisseur OIDC du cluster.

Amazon VPC CNI

- Chaque pod reçoit une **vraie adresse IP du VPC**, routable sans surcouche : performances natives, security groups et flow logs utilisables.
- Les IP viennent des interfaces réseau du nœud : le nombre de pods par nœud dépend du type d'instance.
- **Prefix delegation** : des blocs /28 par interface, bien plus de pods par nœud.
- Network policies Kubernetes prises en charge par le CNI.
- Manque d'adresses : ajouter un CIDR secondaire au VPC (*custom networking*) ou passer en IPv6.

Dimensionnement. Un sous-réseau /24 n'offre que 251 adresses pour les nœuds et les pods réunis. Prévoir des /20 ou plus pour les sous-réseaux de pods.

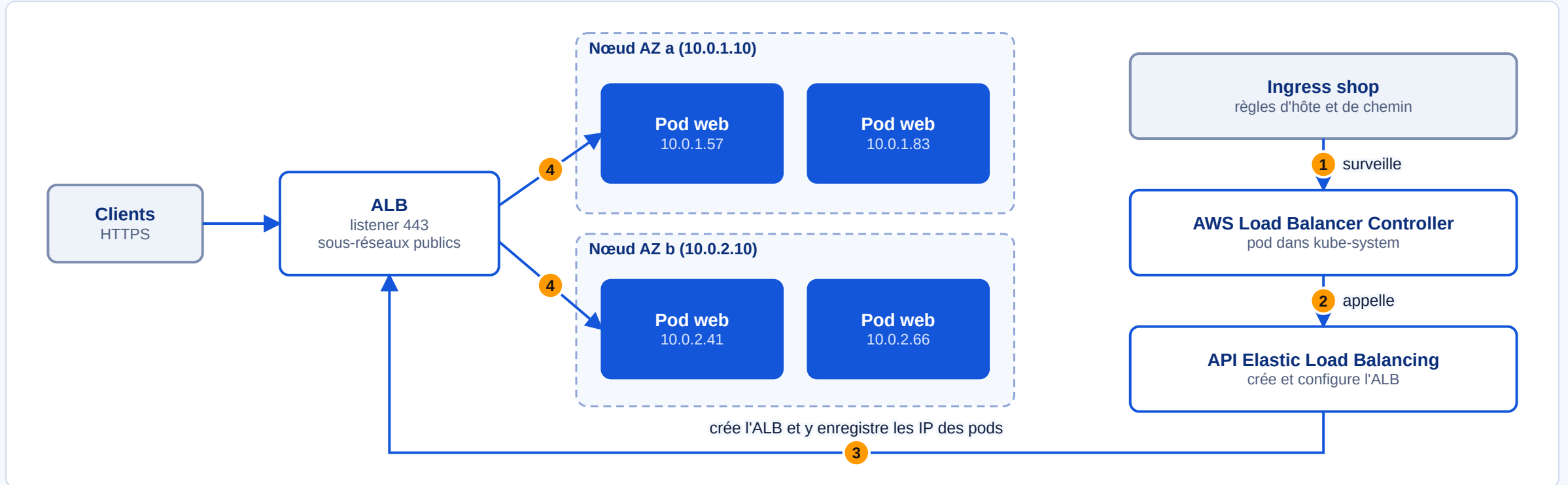
AWS Load Balancer Controller

- Objet **Ingress** (ou Gateway API) → un **ALB** (HTTP/HTTPS, routage par hôte et chemin, certificats ACM, WAF).
- **Service** de type LoadBalancer → un **NLB** (TCP/UDP, IP fixes).
- Cibles en mode **ip** : le load balancer envoie directement aux pods.
- S'installe par Helm avec un rôle IAM dédié ; intégré d'office en Auto Mode.

DNS et service mesh

- CoreDNS pour la résolution interne ; ExternalDNS pour créer les enregistrements Route 53.
- Communication entre clusters et VPC : VPC Lattice, ou un mesh (Istio, Linkerd).

Schéma : du client au pod par un Ingress



1. Le contrôleur surveille les objets Ingress et Service du cluster.
2. À la création de l'Ingress, il appelle l'API Elastic Load Balancing.

3. L'ALB est créé ; les adresses IP des pods sont inscrites comme cibles, et mises à jour à chaque changement.

4. Le trafic va de l'ALB directement aux pods, sans passer par un NodePort.

Ce trajet direct est possible parce que le VPC CNI donne à chaque pod une adresse IP du VPC. Quand un pod disparaît, le contrôleur retire sa cible : d'où l'importance des sondes readiness et d'un délai d'arrêt propre.

Pilote CSI	Type	Usage
EBS CSI	Bloc, ReadWriteOnce, lié à une AZ	Bases de données, StatefulSets. StorageClass gp3.
EFS CSI	Fichier NFS, ReadWriteMany, multi-AZ	Contenu partagé entre pods et entre nœuds.
Mountpoint for S3 CSI	Objet monté en fichier	Lecture de jeux de données volumineux.
FSx CSI	Lustre, ONTAP, OpenZFS	Calcul intensif, besoins NAS avancés.

yaml StorageClass gp3 chiffrée, utilisée par défaut

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gp3
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: ebs.csi.aws.com
volumeBindingMode: WaitForFirstConsumer # volume créé dans l'AZ du pod
parameters:
  type: gp3
  encrypted: "true"
```

Add-ons EKS

Composants opérationnels installés et mis à jour par l'API EKS, avec des versions validées pour chaque version de Kubernetes.

- **Réseau** : `vpc-cni`, `coredns`, `kube-proxy`.
- **Identité** : `eks-pod-identity-agent`.
- **Stockage** : `aws-ebs-csi-driver`, `aws-efs-csi-driver`.
- **Observabilité** : `amazon-cloudwatch-observability`, `adot`, `metrics-server`.

Les add-ons ne sont **pas** mis à jour automatiquement lors d'une montée de version du cluster : c'est une étape à part entière de la procédure.

EBS et AZ. Un volume EBS vit dans une seule AZ : le pod ne peut être replanifié que dans celle-ci. Prévoir des nœuds dans chaque AZ.

Cycle de vie d'une version sur EKS

- Environ trois versions mineures par an, disponibles sur EKS quelques semaines après la sortie officielle.
- **Support standard : 14 mois**, puis **support étendu : 12 mois** (facturé 0,60 \$ par heure).
- Au bout de 26 mois, le control plane est mis à niveau d'office.
- Exemple : la 1.36 est arrivée sur EKS en juin 2026, avec un support standard jusqu'en août 2027.

Règles

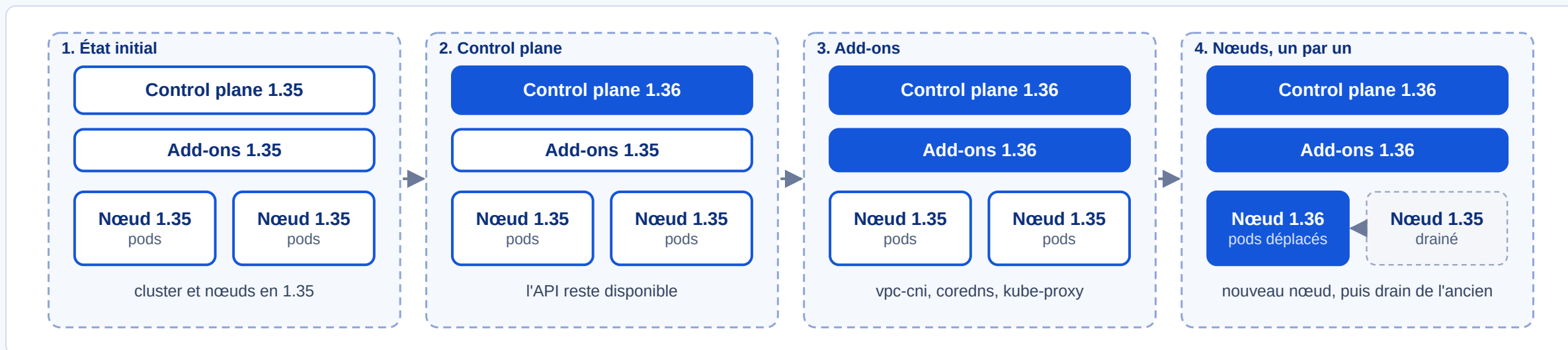
- Une seule version mineure à la fois (1.35 → 1.36), **sans retour arrière**.
- Les nœuds ne doivent jamais être plus récents que le control plane ; ils peuvent avoir jusqu'à 3 versions de retard.

Procédure de mise à niveau

1. Lire les notes de version et les **Upgrade insights** du cluster (API dépréciées encore utilisées, compatibilité des add-ons).
2. Corriger les manifestes et les charts concernés.
3. Mettre à niveau le **control plane** (opération sans coupure de l'API, quelques minutes à quelques dizaines de minutes).
4. Mettre à jour les **add-ons**.
5. Mettre à niveau les **nœuds** : remplacement progressif, pods drainés dans le respect des PodDisruptionBudgets.
6. Valider, puis passer à l'environnement suivant.

Conditions d'un upgrade sans coupure. Au moins 2 réplicas par Deployment, des PodDisruptionBudgets, des sondes readiness correctes.

Schéma : une mise à niveau, étape par étape



Control plane

EKS démarre de nouveaux API servers dans la version cible, vérifie leur santé, puis retire les anciens. L'API reste joignable ; les pods ne sont pas touchés. En cas d'échec, EKS revient seul à l'état précédent.

Add-ons

Chaque add-on est mis à jour vers une version compatible avec la nouvelle version de Kubernetes. Leurs pods sont remplacés progressivement, comme n'importe quel Deployment ou DaemonSet.

Nœuds

Le node group lance un nœud dans la nouvelle version, attend qu'il soit prêt, interdit la planification sur un ancien (*cordon*), en évacue les pods (*drain*) dans le respect des PodDisruptionBudgets, puis le supprime. Et ainsi de suite.

Pendant l'étape 4, chaque pod est arrêté puis recréé sur un autre nœud. Avec un seul réplica, le service est coupé le temps du redémarrage : d'où la règle d'au moins deux réplicas et d'un PodDisruptionBudget.

Amazon EKS

Console : créer et explorer un cluster

aws Elastic Kubernetes Service > Clusters > Create cluster

Configuration options

Mode

☒ Quick configuration (with EKS Auto Mode)

☐ Custom configuration

Cluster configuration

Name

shop-eks

Kubernetes version

1.36 ▼

Cluster IAM role

AmazonEKSAutoClusterRole ▼

Node IAM role

AmazonEKSAutoNodeRole ▼

Networking

VPC

VPC par défaut ▼

Subnets

3 sous-réseaux (une AZ chacun) ▼

Create

Onglet du cluster	Ce qu'on y fait
Overview	Version, état, endpoint, Upgrade insights .
Resources	Parcourir pods, deployments, services, nodes sans kubectl.
Compute	Node groups, Fargate profiles, node pools Auto Mode.
Networking	VPC, sous-réseaux, accès public ou privé à l'endpoint.
Add-ons	Installer et mettre à jour les add-ons.
Access	Access entries, Pod Identity associations.
Observability	Journaux du control plane, tableau de bord, Container Insights.

Custom configuration donne la main sur le mode d'authentification, le chiffrement des Secrets par KMS, l'endpoint, les journaux du control plane et les add-ons. La création prend 10 à 15 minutes.

\$ Prérequis : variables de l'étape 0, droits d'administration sur le compte

```
# 1. Installer kubectl et eksctl
#   macOS (Homebrew)
brew install kubernetes-cli eksctl
#   Linux x86_64
curl -sLO "https://dl.k8s.io/release/$(curl -sL https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -m 0755 kubectl /usr/local/bin/kubectl
curl -sLO "https://github.com/eksctl-io/eksctl/releases/latest/download/eksctl_Linux_amd64.tar.gz"
tar -xzf eksctl_Linux_amd64.tar.gz -C /tmp && sudo install -m 0755 /tmp/eksctl /usr/local/bin/eksctl

# 2. Version par défaut proposée par EKS, et versions disponibles
export K8S_VERSION=$(aws eks describe-cluster-versions --default-only \
  --query 'clusterVersions[0].clusterVersion' --output text)
aws eks describe-cluster-versions \
  --query 'clusterVersions[].[clusterVersion,versionStatus,endOfStandardSupportDate]' --output table

# 3. Créer le cluster : VPC dédié, control plane, node group managé (15 à 20 minutes)
eksctl create cluster --name shop-eks --region $AWS_REGION --version $K8S_VERSION \
  --managed --nodegroup-name ng-main --node-type t3.medium \
  --nodes 2 --nodes-min 2 --nodes-max 4

# 4. Configurer kubectl (eksctl le fait déjà ; commande à connaître pour un cluster existant)
aws eks update-kubeconfig --name shop-eks --region $AWS_REGION
kubectl get nodes -o wide
kubectl get pods -n kube-system
```

eksctl pilote CloudFormation : il crée un VPC, les rôles IAM, le cluster et le node group. Avec la seule CLI AWS, il faut enchaîner `aws eks create-cluster` (rôle et sous-réseaux fournis) puis `aws eks create-nodegroup` .

\$ Déploiement, service, mise à l'échelle

```
kubectl create namespace shop

kubectl -n shop create deployment web \
  --image=public.ecr.aws/nginx/nginx:stable \
  --replicas=2 --port=80

# Service LoadBalancer : EKS crée un load balancer
kubectl -n shop expose deployment web \
  --port=80 --type=LoadBalancer

# Attendre le nom DNS (1 à 3 minutes)
kubectl -n shop get service web --watch
LB=$(kubectl -n shop get service web -o \
  jsonpath='{.status.loadBalancer.ingress[0].hostname}')
curl -sI http://$LB | head -n 1

# Mise à l'échelle et mise à jour progressive
kubectl -n shop scale deployment web --replicas=4
kubectl -n shop set image deployment/web \
  nginx=public.ecr.aws/nginx/nginx:mainline
kubectl -n shop rollout status deployment/web
kubectl -n shop rollout undo deployment/web

# Répartition des pods sur les nœuds et les AZ
kubectl -n shop get pods -o wide
kubectl get nodes -L topology.kubernetes.io/zone
```

\$ Inspecter le cluster avec la CLI AWS

```
aws eks describe-cluster --name shop-eks --query \
  'cluster.{version:version,etat:status,
  endpoint:endpoint,auth:accessConfig.authenticationMode}'

aws eks list-nodegroups --cluster-name shop-eks
aws eks describe-nodegroup --cluster-name shop-eks \
  --nodegroup-name ng-main \
  --query 'nodegroup.{scaling:scalingConfig,
  types:instanceTypes,ami:amiType,etat:status}'

# Agrandir le node group
aws eks update-nodegroup-config --cluster-name shop-eks \
  --nodegroup-name ng-main \
  --scaling-config minSize=2,maxSize=5,desiredSize=3
```

Sans AWS Load Balancer Controller, le contrôleur intégré crée un **Classic Load Balancer**, suffisant pour un lab. En production, on installe le contrôleur et l'on déclare un Ingress (ALB) ou un Service annoté (NLB).

Le **DNS du load balancer** met parfois quelques minutes à se propager : en cas d'erreur de résolution, patienter puis relancer `curl`.

\$ Donner un accès en lecture au namespace shop

```
# Rôle IAM endossable depuis ce compte
cat > devs-trust.json <<EOF
{"Version":"2012-10-17","Statement":[{"Effect":"Allow",
  "Principal":{"AWS":"arn:aws:iam::${AWS_ACCOUNT_ID}:root"},
  "Action":["sts:AssumeRole"]}]}
EOF
aws iam create-role --role-name ShopDevs \
  --assume-role-policy-document file://devs-trust.json
DEV_ROLE=arn:aws:iam::${AWS_ACCOUNT_ID}:role/ShopDevs

aws eks create-access-entry --cluster-name shop-eks \
  --principal-arn $DEV_ROLE
aws eks associate-access-policy --cluster-name shop-eks \
  --principal-arn $DEV_ROLE --policy-arn \
  arn:aws:eks::aws:cluster-access-policy/AmazonEKSVIEWPolicy \
  --access-scope type=namespace,namespaces=shop
aws eks list-access-entries --cluster-name shop-eks
```

\$ Pod Identity : un pod lit S3 sans clé

```
aws eks create-addon --cluster-name shop-eks \
  --addon-name eks-pod-identity-agent
cat > pod-trust.json <<'EOF'
{"Version":"2012-10-17","Statement":[{"Effect":"Allow",
  "Principal":{"Service":"pods.eks.amazonaws.com"},
  "Action":["sts:AssumeRole","sts:TagSession"]}]}
EOF
aws iam create-role --role-name shop-s3-read \
  --assume-role-policy-document file://pod-trust.json
aws iam attach-role-policy --role-name shop-s3-read \
  --policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess
kubectl -n shop create serviceaccount shop-sa
aws eks create-pod-identity-association \
  --cluster-name shop-eks --namespace shop \
  --service-account shop-sa --role-arn \
  arn:aws:iam::${AWS_ACCOUNT_ID}:role/shop-s3-read
# Test : le pod liste les buckets avec le rôle associé
kubectl -n shop run awscli --rm -it --restart=Never \
  --image=public.ecr.aws/aws-cli/aws-cli \
  --overrides='{ "spec": { "serviceAccountName": "shop-sa" } }' \
  -- s3 ls
```

\$ Add-ons : inventaire, versions, mise à jour

```
# Add-ons installés sur le cluster
aws eks list-addons --cluster-name shop-eks
aws eks describe-addon --cluster-name shop-eks \
  --addon-name vpc-cni \
  --query 'addon.{version:addonVersion,etat:status}'

# Catalogue des add-ons pour la version du cluster
CUR=$(aws eks describe-cluster --name shop-eks \
  --query cluster.version --output text)
aws eks describe-addon-versions \
  --kubernetes-version $CUR --query 'addons[].addonName'

# Dernière version compatible d'un add-on
LATEST=$(aws eks describe-addon-versions \
  --addon-name vpc-cni --kubernetes-version $CUR \
  --query 'addons[0].addonVersions[0].addonVersion' \
  --output text)

# Mise à jour en conservant les réglages personnalisés
aws eks update-addon --cluster-name shop-eks \
  --addon-name vpc-cni --addon-version $LATEST \
  --resolve-conflicts PRESERVE
aws eks wait addon-active --cluster-name shop-eks \
  --addon-name vpc-cni
```

\$ Mise à niveau : control plane, puis nœuds

```
# 1. Points d'attention détectés par EKS
aws eks list-insights --cluster-name shop-eks \
  --query 'insights[].[name,insightStatus.status]' \
  --output table

# 2. Control plane : version mineure suivante
NEXT=1.$(( ${CUR#1.} + 1 ))
aws eks update-cluster-version --name shop-eks \
  --kubernetes-version $NEXT
aws eks wait cluster-active --name shop-eks

# 3. Add-ons : bloc de gauche, avec CUR=$NEXT

# 4. Nœuds : remplacement progressif, pods drainés
aws eks update-nodegroup-version \
  --cluster-name shop-eks --nodegroup-name ng-main
aws eks wait nodegroup-active \
  --cluster-name shop-eks --nodegroup-name ng-main
kubectl get nodes
```

L'étape 2 est refusée si le cluster est déjà sur la version la plus récente proposée par EKS, ce qui est le cas d'un cluster créé avec la dernière version. Elle est **irréversible**.

Pendant la mise à niveau des nœuds, suivre `kubectl get nodes --watch` : les nouveaux nœuds apparaissent, les anciens passent en `SchedulingDisabled` puis disparaissent.

Amazon EKS

ECS ou EKS : comment choisir

Critère	ECS	EKS
Modèle	API AWS : task definition, service.	API Kubernetes : Pod, Deployment, Service, Ingress, CRD.
Courbe d'apprentissage	Courte : peu de concepts.	Longue : Kubernetes plus les spécificités AWS.
Exploitation	Pas de version de cluster, pas d'add-on à maintenir.	Montées de version régulières, add-ons, contrôleurs (allégé par Auto Mode).
Coût fixe	Aucun pour le control plane.	0,10 \$ par heure et par cluster (environ 73 \$ par mois).
Écosystème	Outils AWS (Copilot, CDK, CloudFormation, Terraform).	Tout l'écosystème CNCF : Helm, Argo CD, opérateurs, Istio, Prometheus.
Portabilité	Liée à AWS (les images restent portables).	Manifestes réutilisables sur tout Kubernetes, on-premises compris.
IAM par charge	Task role, natif.	Pod Identity ou IRSA.
Extensibilité	Limitée à ce que propose ECS.	Très forte : CRD, opérateurs, admission webhooks.

ECS si l'équipe est petite, 100 % AWS, et veut livrer des conteneurs sans exploiter une plateforme.

EKS si l'organisation standardise sur Kubernetes, vise le multi-cloud ou l'hybride, ou dépend d'outils de l'écosystème.

Bonnes pratiques

- Access entries et mode `API`, rôles IAM plutôt qu'utilisateurs, accès limité par namespace.
- Pod Identity ou IRSA pour chaque charge ; ne rien accorder au rôle des nœuds au-delà du nécessaire.
- Endpoint privé, nœuds en sous-réseaux privés, Secrets chiffrés par KMS.
- Requests et limits sur tous les pods, PodDisruptionBudgets, répartition multi-AZ (`topologySpreadConstraints`).
- Journaux du control plane activés (`audit` , `authenticator`).
- Rester en support standard : planifier une montée de version par trimestre.
- Infrastructure décrite en code (Terraform, eksctl, CDK) et déploiements en GitOps.

Ordre de suppression. Supprimer d'abord les Services LoadBalancer et les Ingress : sinon les load balancers restent, bloquent la suppression du VPC et continuent d'être facturés.

\$ Nettoyage du module EKS (après le module Monitoring si vous le suivez)

```
# 1. Ressources Kubernetes qui créent des ressources AWS
kubectl delete namespace shop
```

```
# 2. Association Pod Identity et accès
ASSOC=$(aws eks list-pod-identity-associations \
  --cluster-name shop-eks \
  --query 'associations[0].associationId' --output text)
aws eks delete-pod-identity-association \
  --cluster-name shop-eks --association-id $ASSOC
aws eks delete-access-entry --cluster-name shop-eks \
  --principal-arn arn:aws:iam::$AWS_ACCOUNT_ID:role/ShopDevs
```

```
# 3. Cluster, node group et VPC créés par eksctl
eksctl delete cluster --name shop-eks \
  --region $AWS_REGION --wait
```

```
# 4. Rôles IAM du module
aws iam detach-role-policy --role-name shop-s3-read \
  --policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess
aws iam delete-role --role-name shop-s3-read
aws iam delete-role --role-name ShopDevs
```

```
# 5. Contrôle : plus aucun cluster
aws eks list-clusters
```

Module 3

AWS Lambda

Exécuter du code en réponse à des événements, sans serveur à gérer, facturé à la milliseconde.

- Modèle d'exécution et cycle de vie
- Invocation synchrone, asynchrone, par interrogation
- Configuration, limites, permissions
- Concurrence, versions, alias, layers
- Tarification
- Exemple console puis CLI complet

Vous fournissez une **fonction** (du code et un point d'entrée, le *handler*).
Lambda crée des environnements d'exécution à la demande, y route chaque événement, et les supprime quand ils ne servent plus. Sans trafic, rien n'est facturé.

- **Mise à l'échelle automatique** : un environnement traite une invocation à la fois ; 100 requêtes simultanées = 100 environnements.
- **Runtimes** : Node.js, Python, Java, .NET, Ruby, runtime personnalisé (`provided.al2023`) pour Go et Rust, ou image de conteneur.
- **Architectures** : x86_64 ou arm64 (Graviton, environ 20 % moins cher).
- **Haute disponibilité** intégrée : exécution répartie sur plusieurs AZ.

Cas d'usage typiques

- API HTTP (API Gateway, Function URL).
- Traitement de fichiers déposés dans S3.
- Consommation de files SQS et de flux.
- Tâches planifiées, automatisation d'exploitation.

Moins adapté

- Traitements de plus de 15 minutes.
- Charge élevée et constante (un conteneur coûte moins cher).
- Latence très stricte sans tolérance au démarrage à froid.
- État local, connexions longues.

Vocabulaire	Définition
Handler	La fonction du code que Lambda appelle : <code>fichier.fonction</code> , par exemple <code>lambda_function.lambda_handler</code> .
Event, context	Les deux arguments du handler : l'événement en JSON, et les informations d'exécution (identifiant de requête, temps restant).
Déclencheur (trigger)	La source qui invoque la fonction : API Gateway, S3, SQS, EventBridge.

Cycle de vie et démarrage à froid



Démarrage à froid

Premier appel, montée en charge ou nouveau déploiement : Lambda doit créer un environnement. Compter de 100 ms à plus d'une seconde selon le runtime, la taille du paquet et le code d'initialisation.

Démarrage à chaud

L'environnement gelé est réutilisé : les variables globales, les connexions ouvertes et le contenu de `/tmp` sont conservés. D'où la règle : initialiser les clients **hors** du handler.

Réduire le démarrage à froid

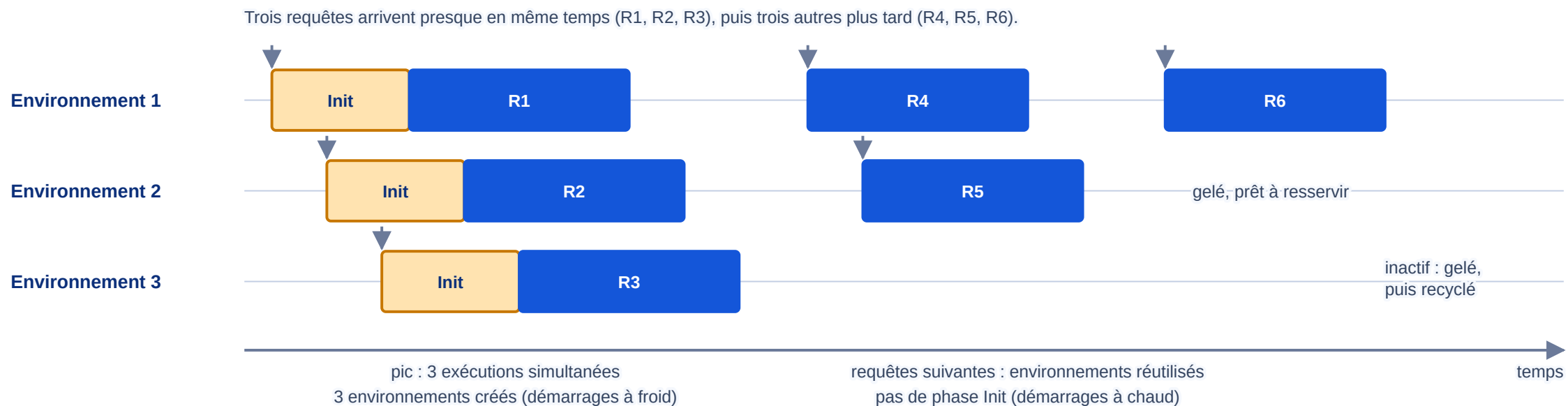
Paquet léger, moins de dépendances, plus de mémoire (donc plus de CPU), arm64, **SnapStart** (Java, Python, .NET), ou **provisioned concurrency** pour des environnements préchauffés.

txt Ligne REPORT écrite dans les logs à chaque invocation : Init Duration n'apparaît que lors d'un démarrage à froid

```
REPORT RequestId: 3f1c9a52-7b1e-4c0a-9d2e-5a8f6e1b2c34 Duration: 12.41 ms Billed Duration: 13 ms
Memory Size: 256 MB Max Memory Used: 41 MB Init Duration: 118.52 ms
```

Ne rien supposer sur la réutilisation. Un environnement peut être recyclé à tout moment : pas d'état métier en mémoire, pas de traitement en arrière-plan après le retour du handler.

Schéma : environnements à froid et à chaud



Un environnement, une requête à la fois

R2 et R3 arrivent pendant que l'environnement 1 traite R1 : Lambda en crée deux autres. La concurrence est le nombre d'environnements occupés au même instant.

Réutilisation

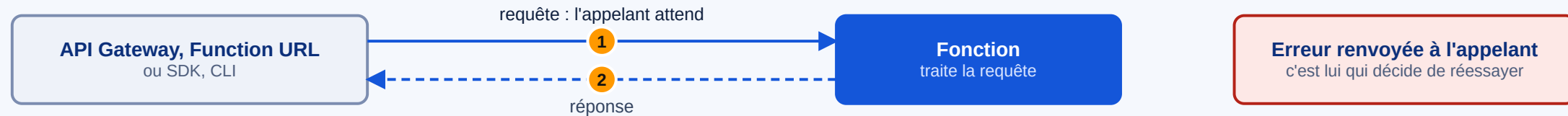
R4, R5 et R6 trouvent un environnement libre déjà initialisé : pas de phase Init, la réponse part plus vite. Les variables globales et les connexions sont toujours là.

Recyclage

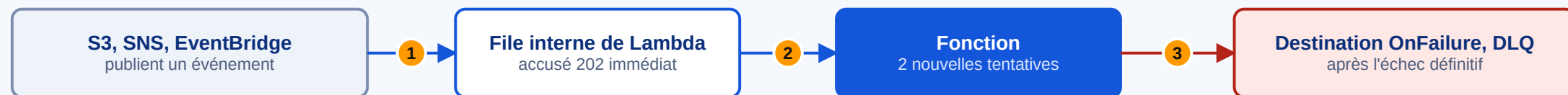
Un environnement inactif est gelé, puis supprimé au bout d'un moment. Le prochain pic recréera des environnements : le démarrage à froid revient.

Schéma : les trois modes d'invocation

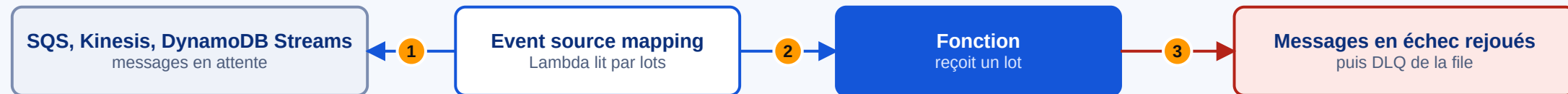
Synchrone



Asynchrone



Interrogation (event source mapping)



Synchrone. 1 : la requête arrive, l'appelant reste en attente. 2 : il reçoit la réponse ou l'erreur.

Asynchrone. 1 : l'événement est mis en file, la source est libérée. 2 : Lambda invoque la fonction et réessaie seul. 3 : après l'échec définitif, l'événement part vers la destination d'échec.

Interrogation. 1 : Lambda lit lui-même la source. 2 : il invoque la fonction avec un lot. 3 : les messages en échec sont rejoués.

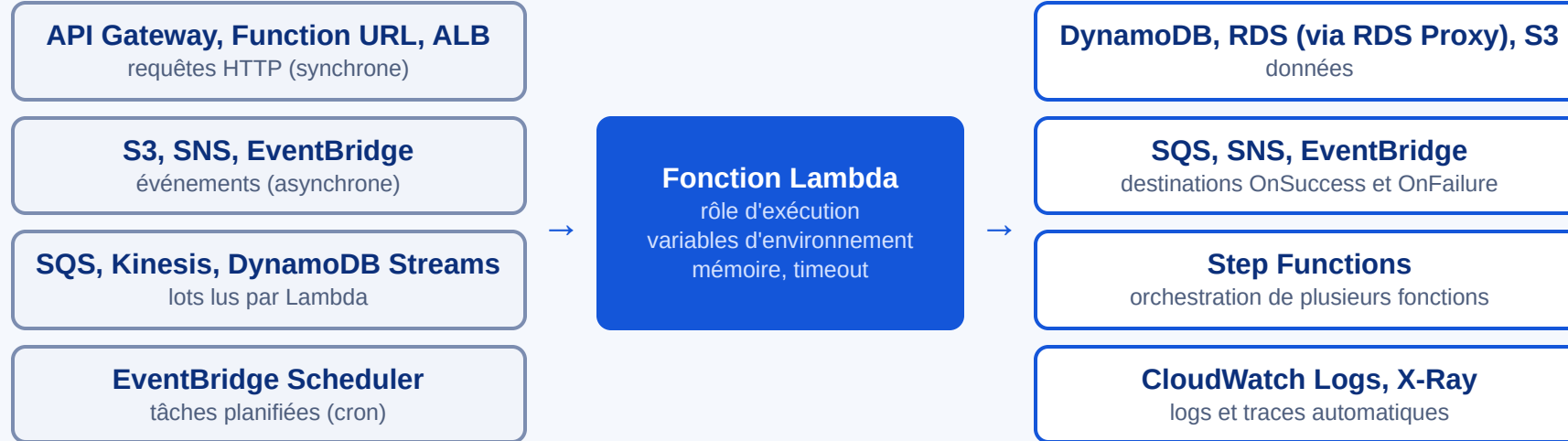
Trois modes d'invocation

	Synchrone	Asynchrone	Interrogation (event source mapping)
Principe	L'appelant attend la réponse.	Lambda met l'événement en file interne et répond aussitôt (202).	Lambda lit lui-même la source par lots et invoque la fonction.
Sources	API Gateway, ALB, Function URL, SDK, CLI.	S3, SNS, EventBridge, CloudWatch Logs.	SQS, Kinesis, DynamoDB Streams, Kafka (MSK), Amazon MQ.
En cas d'erreur	L'erreur revient à l'appelant, qui décide de réessayer.	2 nouvelles tentatives par défaut, événement conservé jusqu'à 6 heures.	SQS : le message redevient visible. Flux : le lot est rejoué jusqu'au succès ou à l'expiration.
Échecs définitifs	À gérer côté client.	Destination <code>OnFailure</code> ou dead-letter queue.	DLQ de la file SQS ; destination d'échec pour les flux.
Charge utile max	6 Mo (200 Mo en réponse diffusée en continu).	1 Mo.	Selon la source et la taille du lot.

Idempotence. En asynchrone et par interrogation, un même événement peut être livré plusieurs fois. Le traitement doit donner le même résultat s'il est rejoué (clé d'idempotence, écriture conditionnelle).

Lots partiels. Avec SQS et les flux, renvoyer `batchItemFailures` (option `ReportBatchItemFailures`) pour ne rejouer que les messages en échec, pas tout le lot.

Sources d'événements et destinations



`{}` Événement reçu : dépôt d'un objet dans S3 (extrait)

```
{ "Records": [ {  
  "eventSource": "aws:s3",  
  "eventName": "ObjectCreated:Put",  
  "s3": { "bucket": { "name": "shop-uploads" },  
    "object": { "key": "factures/2026/10/A1001.pdf",  
      "size": 48213 } } } ] }
```

Chaque source a son format

Le handler reçoit un document JSON dont la structure dépend de la source : requête HTTP pour API Gateway, liste `Records` pour S3, SQS, DynamoDB Streams. La console propose des modèles d'événements de test.

Lambda dans un VPC : nécessaire pour joindre RDS ou ElastiCache en privé. La fonction perd alors l'accès à Internet, sauf NAT Gateway ou VPC endpoints.

Paramètre	Valeurs
Mémoire	128 Mo à 10 240 Mo. Le CPU est proportionnel (jusqu'à 6 vCPU).
Timeout	3 s par défaut, 900 s (15 min) au maximum.
Stockage éphémère /tmp	512 Mo à 10 240 Mo.
Paquet zip	50 Mo compressé en envoi direct, 250 Mo décompressé, layers compris.
Image de conteneur	Jusqu'à 10 Go, stockée dans ECR.
Variables d'environnement	4 Ko au total, chiffrées au repos.
Layers	5 par fonction.
Concurrence par région	1 000 exécutions simultanées par défaut (moins sur un compte neuf), quota ajustable.
Vitesse de montée en charge	Jusqu'à 1 000 environnements supplémentaires toutes les 10 secondes, par fonction.

Mémoire, CPU et coût

Augmenter la mémoire augmente le CPU : la durée baisse souvent assez pour que le coût reste stable, voire diminue. L'outil **AWS Lambda Power Tuning** mesure le meilleur réglage.

Timeout

Le régler juste au-dessus de la durée attendue. Derrière API Gateway, la limite d'intégration est de 29 secondes par défaut ; avec SQS, le *visibility timeout* de la file doit valoir au moins 6 fois le timeout de la fonction.

Évolutions récentes

Durable functions : flux en plusieurs étapes avec reprise sur point de contrôle et attentes longues. **Lambda Managed Instances** : fonctions exécutées sur des instances EC2 gérées par Lambda, pour les charges soutenues.

Permissions : deux sens à ne pas confondre

Rôle d'exécution : ce que la fonction peut faire

Rôle IAM endossé par Lambda (trust policy : `lambda.amazonaws.com`). Les identifiants temporaires sont injectés dans l'environnement.

- Minimum : `AWSLambdaBasicExecutionRole` (écrire dans CloudWatch Logs).
- Source SQS, Kinesis ou DynamoDB Streams : droits de lecture sur la source, car c'est Lambda qui interroge.
- VPC : `AWSLambdaVPCAccessExecutionRole` pour créer les interfaces réseau.

Politique basée sur la ressource : qui peut invoquer

Attachée à la fonction. Autorise un service ou un autre compte à l'appeler.

- S3, SNS, EventBridge, API Gateway **poussent** des événements : il leur faut une autorisation `lambda:InvokeFunction`.
- Ajoutée par `aws lambda add-permission`, ou par la console quand on crée un déclencheur.
- Toujours restreindre avec `--source-arn` (et `--source-account` pour S3).

\$ Exemple : autoriser le bucket shop-uploads à invoquer la fonction

```
aws lambda add-permission --function-name shop-hello --statement-id s3-shop-uploads \
  --action lambda:InvokeFunction --principal s3.amazonaws.com \
  --source-arn arn:aws:s3:::shop-uploads --source-account $AWS_ACCOUNT_ID
aws lambda get-policy --function-name shop-hello --query Policy --output text | jq .
```

Diagnostic. La fonction n'est jamais déclenchée : regarder la politique basée sur la ressource. La fonction démarre mais échoue sur `AccessDenied` : regarder le rôle d'exécution.

Concurrence et montée en charge

Calcul de la concurrence

Concurrence = requêtes par seconde × durée moyenne (en secondes).

200 requêtes par seconde à 250 ms : $200 \times 0,25 = 50$ environnements simultanés. À 2 secondes par requête : 400.

Throttling

Au-delà de la concurrence disponible, Lambda refuse l'invocation (erreur 429, `TooManyRequestsException`). En synchrone, l'appelant reçoit l'erreur ; en asynchrone, Lambda réessaie pendant 6 heures ; avec SQS, les messages restent dans la file.

Métriques à surveiller : `ConcurrentExecutions` , `Throttles` , et `ClaimedAccountConcurrency` au niveau du compte.

Réglage	Rôle
Reserved concurrency	Réserve une part du quota pour une fonction et la plafonne. Protège une base en aval, ou garantit de la capacité à une fonction critique. Gratuit. Une valeur de 0 coupe la fonction.
Provisioned concurrency	Garde N environnements initialisés en permanence : plus de démarrage à froid pour ces N. Payant à l'heure, à poser sur un alias ou une version, pilotable par Auto Scaling.
SnapStart	Lambda prend un instantané de l'environnement initialisé à la publication d'une version, puis le restaure à la demande. Java, Python, .NET.
Maximum concurrency (SQS)	Plafonne le nombre d'invocations simultanées déclenchées par une file, sans throttling.

Base relationnelle en aval. 500 environnements = 500 connexions ouvertes. Plafonner la concurrence et passer par **RDS Proxy**.

Versions

`$LATEST` est modifiable. `publish-version` fige le code et la configuration dans une version numérotée, **immuable** (1, 2, 3...), avec son propre ARN.

Alias

Pointeur nommé (`prod` , `staging`) vers une version. Les déclencheurs visent l'alias : on déploie en déplaçant le pointeur, on revient en arrière de la même façon.

Routage pondéré : l'alias peut envoyer par exemple 10 % du trafic vers la nouvelle version (canary). CodeDeploy automatise la progression et le rollback sur alarme.

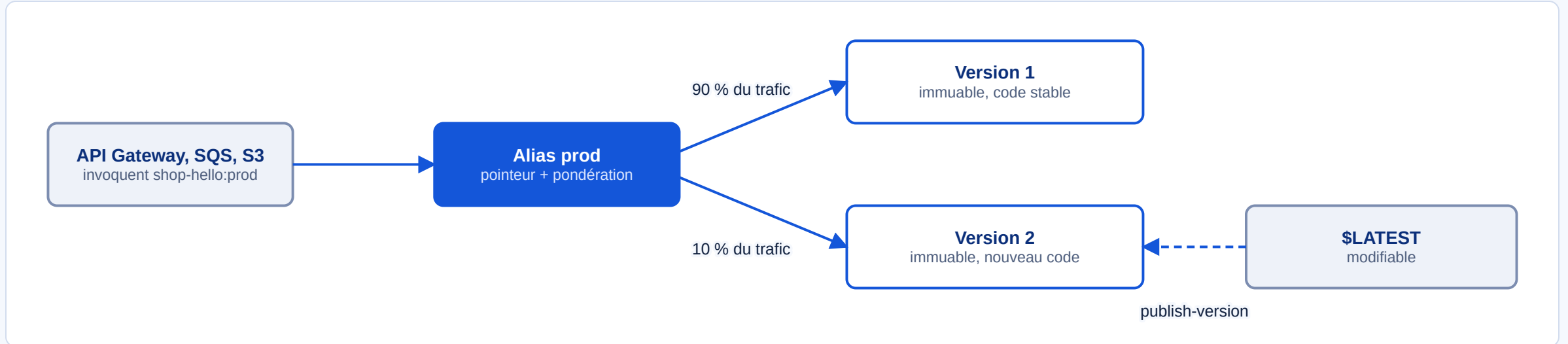
Layers

Archive partagée (bibliothèques, runtime personnalisé, extensions) montée dans `/opt` . Allège le paquet de la fonction et mutualise les dépendances entre fonctions.

	Archive zip	Image de conteneur
Taille max	250 Mo décompressé	10 Go
Outillage	zip, SAM, CDK	Dockerfile, ECR, CI existante
Dépendances	Dans le zip ou en layers	Dans l'image
Démarrage à froid	Le plus court	Un peu plus long la première fois
À choisir pour	Fonctions simples	Dépendances lourdes (ML, binaires)

Infrastructure as code. En projet réel, on ne déploie pas à la main : AWS SAM, CDK, Terraform ou Serverless Framework décrivent fonction, rôle, déclencheurs et alias dans le dépôt.

Schéma : déployer avec un alias



1. Le nouveau code est envoyé dans `$LATEST`, puis figé par `publish-version` : c'est la version 2.
2. L'alias `prod` garde la version 1 comme cible principale et envoie 10 % des invocations à la version 2.
3. On observe les erreurs et la durée de la version 2.
4. Tout va bien : l'alias pointe entièrement sur la version 2. Sinon : on retire la pondération, retour immédiat à la version 1.

Les déclencheurs visent l'ARN de l'alias (`shop-hello:prod`) : ils ne changent jamais, seul le pointeur bouge. Le code des versions étant immuable, le retour arrière est instantané.

CodeDeploy automatise cette progression (canary, linéaire) et annule tout seul si une alarme CloudWatch se déclenche.

Composante	Tarif indicatif
Requêtes	0,20 \$ par million.
Durée, x86_64	0,0000166667 \$ par Go-seconde.
Durée, arm64	0,0000133334 \$ par Go-seconde.
Offre gratuite mensuelle	1 million de requêtes et 400 000 Go-secondes.
Granularité	Facturation à la milliseconde.

Go-seconde = mémoire configurée (en Go) × durée d'exécution (en secondes). Les tarifs varient légèrement selon la région : vérifier la page de tarification avant un chiffrage.

S'ajoutent : provisioned concurrency, stockage éphémère au-delà de 512 Mo, transfert de données, CloudWatch Logs, API Gateway, NAT Gateway.

UtopiosShop : API des commandes

3 millions d'appels par mois, 200 ms en moyenne, 512 Mo, x86_64.

Requêtes	3 M × 0,20 \$ / M	0,60 \$
Calcul	3 000 000 × 0,2 s × 0,5 Go = 300 000 Go-s	
Durée	300 000 × 0,0000166667 \$	5,00 \$
Total avant offre gratuite		5,60 \$
Après offre gratuite	2 M de requêtes facturées, durée couverte	0,40 \$

Lecture

À faible ou moyen trafic, Lambda est très économique. Le point de bascule vers un conteneur arrive quand la fonction tourne en continu : une tâche Fargate à 0,25 vCPU coûte une dizaine de dollars par mois, quel que soit le nombre de requêtes.

Lambda > Functions > Create function

Option

☐ Author from scratch

Function name

shop-hello

Runtime

Python 3.13 ▼

Architecture

☐ arm64

Change default execution role

Execution role

☐ Create a new role with basic Lambda permissions

Additional configurations

Function URL

☒ Enable (Auth type : NONE)

Create function

Onglet	Ce qu'on y fait
Code	Éditeur intégré, bouton Deploy , bouton Test avec des événements enregistrés (modèles S3, SQS, API Gateway).
Test	Créer et rejouer des événements de test.
Monitor	Graphiques (invocations, durée, erreurs, throttles), lien vers CloudWatch Logs et les traces.
Configuration	Mémoire et timeout, Triggers , Permissions (rôle et politique de ressource), variables d'environnement, VPC, concurrence, Function URL.
Aliases, Versions	Publier une version, créer un alias, régler le routage pondéré.

Lecture du résultat d'un test. La console affiche la réponse, les logs de l'exécution et la ligne `REPORT` : durée, durée facturée, mémoire maximale utilisée, et `Init Duration` s'il s'agissait d'un démarrage à froid.

\$ Prérequis : variables de l'étape 0, zip installé

1. Rôle d'exécution

```
cat > lambda-trust.json <<'EOF'
{"Version":"2012-10-17","Statement":[{"Effect":"Allow",
  "Principal":{"Service":"lambda.amazonaws.com"},
  "Action":["sts:AssumeRole"]}]}
EOF
aws iam create-role --role-name shop-lambda-role \
  --assume-role-policy-document file://lambda-trust.json
aws iam attach-role-policy --role-name shop-lambda-role \
  --policy-arn \
  arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole
sleep 10 # propagation IAM avant de créer la fonction
```

2. Code : fichier lambda_function.py (bloc de droite)
zip function.zip lambda_function.py

3. Création

```
aws lambda create-function --function-name shop-hello \
  --runtime python3.13 --architectures arm64 \
  --handler lambda_function.lambda_handler \
  --role arn:aws:iam::$AWS_ACCOUNT_ID:role/shop-lambda-role \
  --zip-file fileb://function.zip \
  --timeout 10 --memory-size 256 \
  --environment "Variables={STAGE=dev}"
aws lambda wait function-active-v2 \
  --function-name shop-hello
```

py lambda_function.py

```
import json, os

# Exécuté une seule fois par environnement (phase Init)
STAGE = os.environ.get("STAGE", "dev")

def lambda_handler(event, context):
    params = event.get("queryStringParameters") or {}
    name = params.get("name", "monde")
    # Log structuré : exploitable dans Logs Insights
    print(json.dumps({"request_id": context.aws_request_id,
                      "name": name, "stage": STAGE}))

    return {
        "statusCode": 200,
        "headers": {"Content-Type": "application/json"},
        "body": json.dumps({"message": f"Bonjour {name}",
                             "stage": STAGE}),
    }
```

\$ 4. Invocation synchrone

```
aws lambda invoke --function-name shop-hello \
  --cli-binary-format raw-in-base64-out \
  --payload '{"queryStringParameters":{"name":"Lille"}}' \
  out.json && cat out.json
```

\$ Function URL publique : lab uniquement (sinon AWS_IAM)

```
aws lambda create-function-url-config \
  --function-name shop-hello --auth-type NONE

# Deux autorisations sont nécessaires pour un accès public
aws lambda add-permission --function-name shop-hello \
  --statement-id url-public --principal "*" \
  --action lambda:InvokeFunctionUrl \
  --function-url-auth-type NONE
aws lambda add-permission --function-name shop-hello \
  --statement-id url-public-invoke --principal "*" \
  --action lambda:InvokeFunction \
  --invoked-via-function-url

URL=$(aws lambda get-function-url-config \
  --function-name shop-hello \
  --query FunctionUrl --output text)
curl -s "${URL}?name=UtopiosShop"

# Modifier la configuration (variables, mémoire)
aws lambda update-function-configuration \
  --function-name shop-hello --memory-size 512 \
  --environment "Variables={STAGE=prod}"
aws lambda wait function-updated-v2 \
  --function-name shop-hello
```

\$ Publier, créer l'alias prod, déployer en canary

```
# Version 1 = état actuel, figé
V1=$(aws lambda publish-version \
  --function-name shop-hello \
  --query Version --output text)
aws lambda create-alias --function-name shop-hello \
  --name prod --function-version $V1

# Nouveau code (après modification du fichier)
zip function.zip lambda_function.py
aws lambda update-function-code \
  --function-name shop-hello \
  --zip-file fileb://function.zip
aws lambda wait function-updated-v2 \
  --function-name shop-hello
V2=$(aws lambda publish-version \
  --function-name shop-hello \
  --query Version --output text)

# Canary : 10 % du trafic de prod vers la version 2
aws lambda update-alias --function-name shop-hello \
  --name prod --function-version $V1 \
  --routing-config "AdditionalVersionWeights={$V2=0.1}"

# Bascule complète, puis retour arrière possible vers V1
aws lambda update-alias --function-name shop-hello \
  --name prod --function-version $V2 \
  --routing-config "AdditionalVersionWeights={}"
```

\$ Brancher une file SQS sur la fonction

```
# File de commandes (visibility timeout = 6 x timeout Lambda)
QUEUE_URL=$(aws sqs create-queue --queue-name shop-orders \
  --attributes VisibilityTimeout=60 \
  --query QueueUrl --output text)
QUEUE_ARN=$(aws sqs get-queue-attributes \
  --queue-url $QUEUE_URL --attribute-names QueueArn \
  --query Attributes.QueueArn --output text)

# Le rôle doit pouvoir lire la file : Lambda l'interroge
POLICIES=arn:aws:iam::aws:policy/service-role
aws iam attach-role-policy --role-name shop-lambda-role \
  --policy-arn $POLICIES/AWSLambdaSQSQueueExecutionRole

aws lambda create-event-source-mapping \
  --function-name shop-hello \
  --event-source-arn $QUEUE_ARN --batch-size 10 \
  --function-response-types ReportBatchItemFailures \
  --scaling-config MaximumConcurrency=5

# Envoyer un message, puis observer l'exécution
aws sqs send-message --queue-url $QUEUE_URL \
  --message-body '{"orderId":"A1001","total":129.9}'
aws logs tail /aws/lambda/shop-hello --since 5m
```

\$ Concurrence

```
# Plafonner la fonction à 20 exécutions simultanées
aws lambda put-function-concurrency \
  --function-name shop-hello \
  --reserved-concurrent-executions 20

# 5 environnements préchauffés sur l'alias prod (payant)
aws lambda put-provisioned-concurrency-config \
  --function-name shop-hello --qualifier prod \
  --provisioned-concurrent-executions 5

# Quota et consommation du compte dans la région
aws lambda get-account-settings \
  --query '{limite:AccountLimit,usage:AccountUsage}'
```

\$ Logs et traces

```
aws logs tail /aws/lambda/shop-hello --follow --format short
aws lambda update-function-configuration \
  --function-name shop-hello --tracing-config Mode=Active
```

Compte neuf. Le quota de concurrence y est réduit (souvent 10) : les deux réglages de concurrence sont alors refusés tant que le quota n'a pas été relevé (Service Quotas).

Bonnes pratiques

- Clients SDK et connexions créés **hors du handler**, pour profiter des démarrages à chaud.
- Fonctions petites, à responsabilité unique, **idempotentes**.
- Un rôle par fonction, au moindre privilège ; secrets dans Secrets Manager ou Parameter Store.
- Logs structurés en JSON, traçage X-Ray actif, alarmes sur **Errors**, **Throttles** et la durée p95.
- Timeout et mémoire réglés d'après les mesures ; arm64 par défaut.
- Destination d'échec ou DLQ sur tout traitement asynchrone.
- Déclencheurs branchés sur un alias, jamais sur **\$LATEST**, en production.

Pièges fréquents

- Boucle infinie : la fonction écrit dans le bucket qui la déclenche.
- Dépendances natives compilées pour la mauvaise architecture.
- Fonction en VPC sans NAT ni endpoint : les appels aux API AWS expirent.

\$ Nettoyage du module Lambda (après le module Monitoring)

```
# Déclencheur SQS, puis file
UUID=$(aws lambda list-event-source-mappings \
  --function-name shop-hello \
  --query 'EventSourceMappings[0].UUID' --output text)
aws lambda delete-event-source-mapping --uuid $UUID
aws sqs delete-queue --queue-url $QUEUE_URL

# Concurrence provisionnée (facturée à l'heure)
aws lambda delete-provisioned-concurrency-config \
  --function-name shop-hello --qualifier prod

# Fonction : supprime aussi versions, alias, Function URL
aws lambda delete-function --function-name shop-hello

# Groupe de logs
aws logs delete-log-group \
  --log-group-name /aws/lambda/shop-hello

# Rôle : détacher les politiques avant de supprimer
for P in AWSLambdaBasicExecutionRole \
  AWSLambdaSQSQueueExecutionRole; do
  aws iam detach-role-policy --role-name shop-lambda-role \
    --policy-arn arn:aws:iam::aws:policy/service-role/$P
done
aws iam delete-role --role-name shop-lambda-role
```

Module 4

Amazon RDS

Relational Database Service : des bases relationnelles exploitées par AWS.

- Moteurs et partage des responsabilités
- Composants d'une instance
- Haute disponibilité : Multi-AZ et réplicas de lecture
- Sauvegardes, restauration, sécurité
- Aurora, RDS Proxy, déploiements blue/green
- Exemple console puis CLI complet

RDS installe, sauvegarde, corrige et surveille le moteur de base de données à votre place. Vous gardez le SQL, le schéma et les réglages ; vous perdez l'accès au système d'exploitation.

Moteur	Particularités
PostgreSQL, MySQL, MariaDB	Open source, les plus utilisés. Multi-AZ, réplicas, blue/green.
Oracle, SQL Server, Db2	Licence incluse ou apportée (BYOL). Fonctions via <i>option groups</i> .
Aurora (MySQL, PostgreSQL)	Moteur AWS compatible, stockage distribué, serverless possible.
RDS Custom	Accès à l'OS pour Oracle et SQL Server (applications qui l'exigent).

AWS s'occupe de

- Provisionnement, stockage, réplication.
- Sauvegardes automatiques et restauration.
- Correctifs de l'OS et du moteur (fenêtre de maintenance).
- Détection de panne et bascule.

Vous gardez

- Schéma, index, requêtes, plans d'exécution.
- Dimensionnement et paramètres du moteur.
- Utilisateurs SQL, droits, règles réseau.
- Stratégie de sauvegarde et tests de restauration.

Ce qui est facturé. Heures d'instance (doublées en Multi-AZ), stockage provisionné au Go par mois, IOPS au-delà du forfait, stockage de sauvegarde au-delà de la taille de la base, transfert de données.

Réduire la facture. Instances Graviton, arrêt des bases de développement hors horaires (7 jours au maximum, redémarrage automatique ensuite), instances réservées pour la production stable.

Les composants d'une instance RDS

Classe d'instance

CPU et mémoire. `db.t4g` : à crédits, pour le dev et les petites charges. `db.m` : usage général. `db.r` : optimisée mémoire. Suffixe **g** = Graviton. Exemple : `db.m7g.large`.

Stockage

gp3 : SSD général, 3 000 IOPS de base, IOPS et débit réglables indépendamment de la taille. **io2** : IOPS provisionnées pour les charges intensives. **Autoscaling** du stockage jusqu'à un plafond que vous fixez. On ne peut pas réduire la taille.

Endpoint

Nom DNS stable de l'instance (`shop-pg.abc123.eu-west-3.rds.amazonaws.com`). En cas de bascule Multi-AZ, le DNS pointe vers la nouvelle primaire : l'application ne change rien.

DB subnet group

Liste de sous-réseaux, dans au moins 2 AZ, où RDS peut placer l'instance et son standby. En production : des sous-réseaux privés.

Security group

Filtre le port du moteur (5432 PostgreSQL, 3306 MySQL et MariaDB, 1433 SQL Server, 1521 Oracle). Source idéale : le security group de l'application.

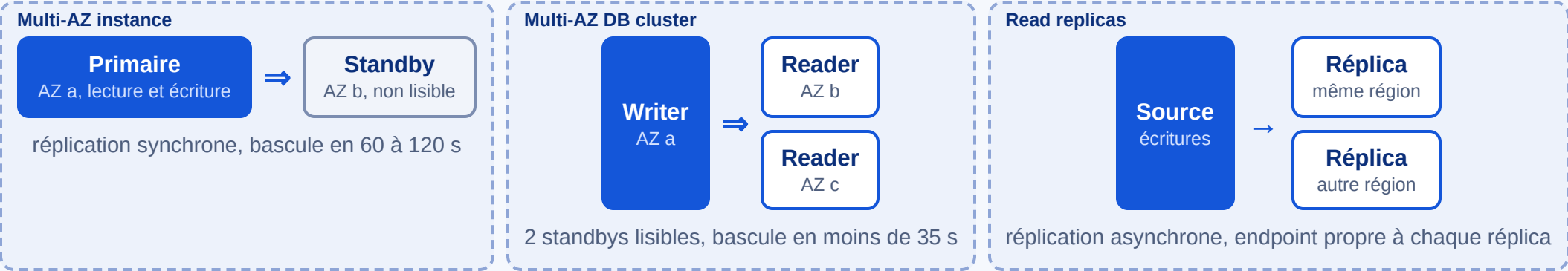
Parameter group et option group

Parameter group : réglages du moteur (`max_connections`, `log_min_duration_statement`). Paramètres dynamiques appliqués à chaud, statiques après redémarrage. **Option group** : fonctions additionnelles (Oracle, SQL Server).

Fenêtres. La *fenêtre de maintenance* (hebdomadaire) reçoit les correctifs ; la *fenêtre de sauvegarde* (quotidienne) le snapshot automatique. À placer en heures creuses.

Amazon RDS

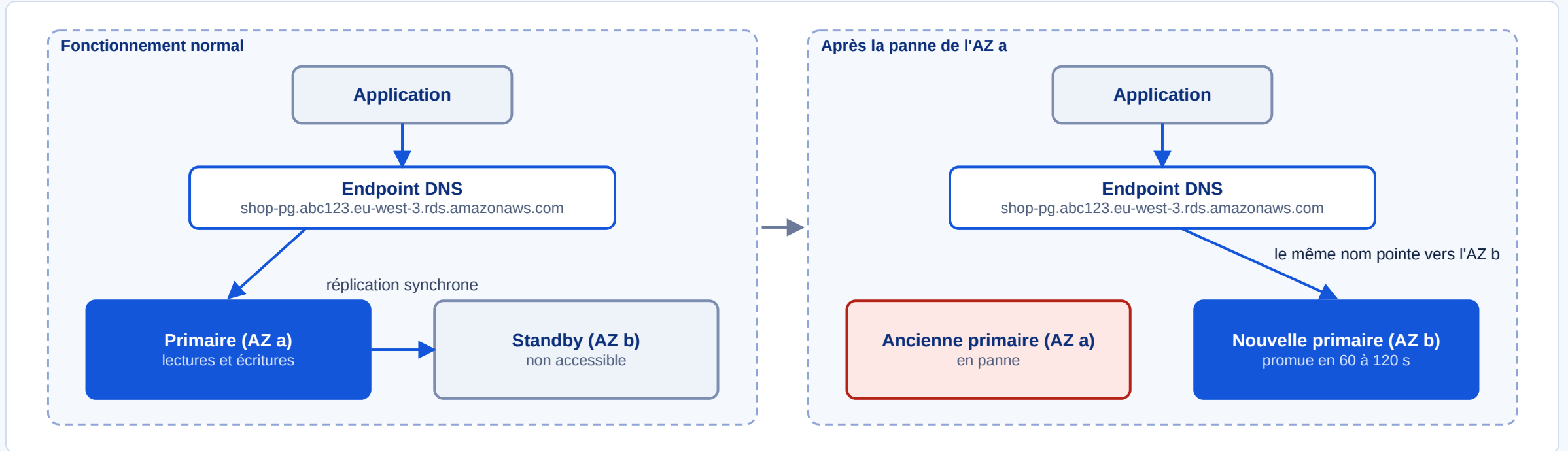
Haute disponibilité et montée en charge en lecture



	Multi-AZ instance	Multi-AZ DB cluster	Read replica
Objectif	Disponibilité	Disponibilité et lectures	Montée en charge des lectures, reprise régionale
Réplication	Synchrone : aucune perte	Semi-synchrone	Asynchrone : retard possible (<code>ReplicaLag</code>)
Bascule	Automatique, même endpoint	Automatique, endpoints writer et reader	Promotion manuelle : devient une base indépendante
Moteurs	Tous	MySQL et PostgreSQL	Jusqu'à 15 réplicas selon le moteur

À retenir. Multi-AZ protège d'une panne ; les réplicas servent à absorber des lectures. Les deux se combinent. Aucun ne remplace les sauvegardes : une suppression par erreur est répliquée instantanément.

Schéma : la bascule Multi-AZ



Avant

Chaque écriture est validée sur la primaire **et** sur le standby avant d'être confirmée à l'application. Le standby ne sert aucune requête.

Pendant

RDS détecte la panne (instance, stockage, réseau ou AZ), promeut le standby et modifie l'enregistrement DNS de l'endpoint. Les connexions ouvertes sont coupées.

Après

L'application se reconnecte au même nom, qui désigne la nouvelle primaire. RDS reconstruit ensuite un standby. Aucune donnée validée n'est perdue.

Côté application : reconnexion automatique et cache DNS court. Une JVM qui garde le DNS indéfiniment reste accrochée à l'ancienne adresse.

Sauvegardes automatiques

- Un snapshot quotidien du volume, plus les journaux de transactions archivés toutes les 5 minutes.
- Rétention de **1 à 35 jours** (0 = désactivé, à proscrire).
- Permettent la restauration à un instant donné (**PITR**), à la seconde près, jusqu'à environ 5 minutes avant maintenant.
- Supprimées avec l'instance, sauf demande de conservation.

Snapshots manuels

- Créés à la demande, conservés jusqu'à suppression explicite.
- Copiables vers une autre région, partageables avec un autre compte.
- À prendre avant une montée de version ou une migration de schéma.

Une restauration crée toujours une nouvelle instance, avec un nouvel endpoint. On ne restaure jamais « par-dessus » l'instance existante : prévoir le changement d'endpoint côté application (ou un renommage).

Objectifs de reprise

Scénario	RPO	RTO
Panne d'AZ, Multi-AZ	0	1 à 2 minutes
Erreur humaine, PITR	5 minutes	Dizaines de minutes à heures, selon la taille
Perte de région, réplica inter-régions	Secondes à minutes	Minutes (promotion)
Perte de région, copie de snapshots	Heures	Heures

RPO : quantité de données qu'on accepte de perdre. RTO : durée d'interruption acceptée. **AWS Backup** centralise les plans de sauvegarde et la copie inter-régions.

Schéma : restauration à un instant donné

Ce que RDS conserve pendant la période de rétention



Restauration à jeudi 14:31



Lecture. RDS part du dernier snapshot antérieur à l'instant demandé, puis rejoue les journaux de transactions jusqu'à cet instant. Plus le snapshot est ancien et l'activité forte, plus la restauration est longue.

Conséquence. L'instance d'origine continue de tourner avec la table supprimée. On récupère les données dans la nouvelle instance (export puis import de la table), ou l'on y bascule l'application.

Réseau

- Instance en sous-réseaux privés, `PubliclyAccessible=false`.
- Security group n'autorisant que le SG de l'application.
- Accès d'administration via VPN, bastion ou Session Manager.

Chiffrement

- **Au repos** : KMS, couvre le volume, les sauvegardes, les snapshots et les réplicas. Se décide **à la création**.
- Chiffrer une base existante : snapshot → copie chiffrée → restauration.
- **En transit** : TLS. Le forcer côté serveur (`rds.force_ssl=1` pour PostgreSQL) et vérifier le certificat côté client.

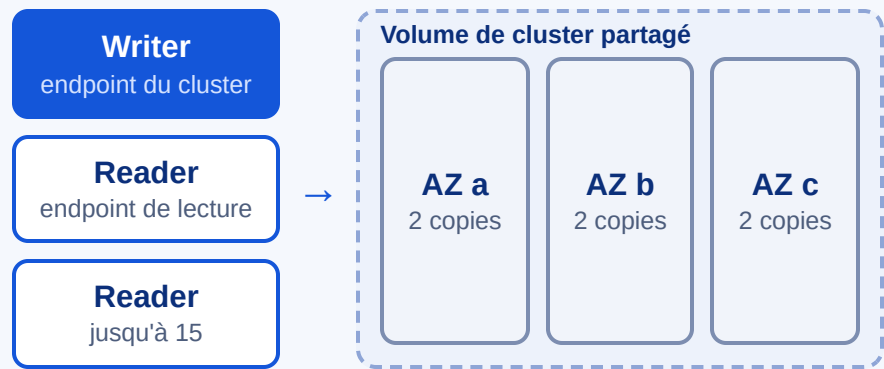
Identités et secrets

- **Mot de passe maître géré** : RDS le crée dans Secrets Manager et le fait tourner automatiquement (`--manage-master-user-password`).
- **Authentification IAM** : l'application se connecte avec un jeton valable 15 minutes, sans mot de passe stocké.
- Comptes SQL applicatifs distincts du compte maître, avec des droits limités au schéma utile.

Audit et protection

- Export des journaux du moteur vers CloudWatch Logs ; extension `pgaudit` pour PostgreSQL.
- CloudTrail trace les appels d'API RDS (création, modification, suppression).
- **Deletion protection** : empêche la suppression accidentelle de l'instance.

Aurora sépare le calcul du stockage. Les instances partagent un **volume de cluster** distribué : 6 copies des données réparties sur 3 AZ, réparation automatique, croissance automatique.



- Les réplicas lisent le même stockage : retard de quelques millisecondes, bascule en général en moins de 30 secondes.
- Écriture validée dès que 4 copies sur 6 l'ont reçue : la perte d'une AZ entière ne bloque pas les écritures.

Fonction	Intérêt
Serverless v2	Capacité en ACU (environ 2 Go de mémoire chacune), ajustée en continu ; peut descendre à 0 et se mettre en pause.
Global Database	Régions secondaires en lecture, retard inférieur à la seconde, bascule régionale.
Clone	Copie instantanée d'un cluster (copy-on-write) pour tester.
Backtrack (MySQL)	Revenir en arrière dans le temps sans restauration.
Aurora DSQL	SQL distribué serverless, multi-région actif-actif, compatible PostgreSQL.

RDS classique ou Aurora ? RDS : coût prévisible, petites bases, moteurs Oracle ou SQL Server. Aurora : forte disponibilité, beaucoup de lectures, charge variable (Serverless v2), reprise multi-région.

RDS Proxy, mise à l'échelle et changements sans coupure

RDS Proxy

- Mutualise un pool de connexions entre de nombreux clients.
- Indispensable devant Lambda : des centaines d'environnements, peu de connexions réelles.
- Bascule Multi-AZ plus rapide et transparente pour l'application.
- Identifiants lus dans Secrets Manager, authentification IAM possible.

Mise à l'échelle

- **Verticale** : changer la classe d'instance. Coupure brève, réduite par la bascule en Multi-AZ.
- **Lectures** : réplicas, à condition que l'application sépare lectures et écritures.
- **Stockage** : extension à chaud, autoscaling.
- Écritures au-delà d'une instance : partitionnement applicatif, Aurora Limitless ou DynamoDB.

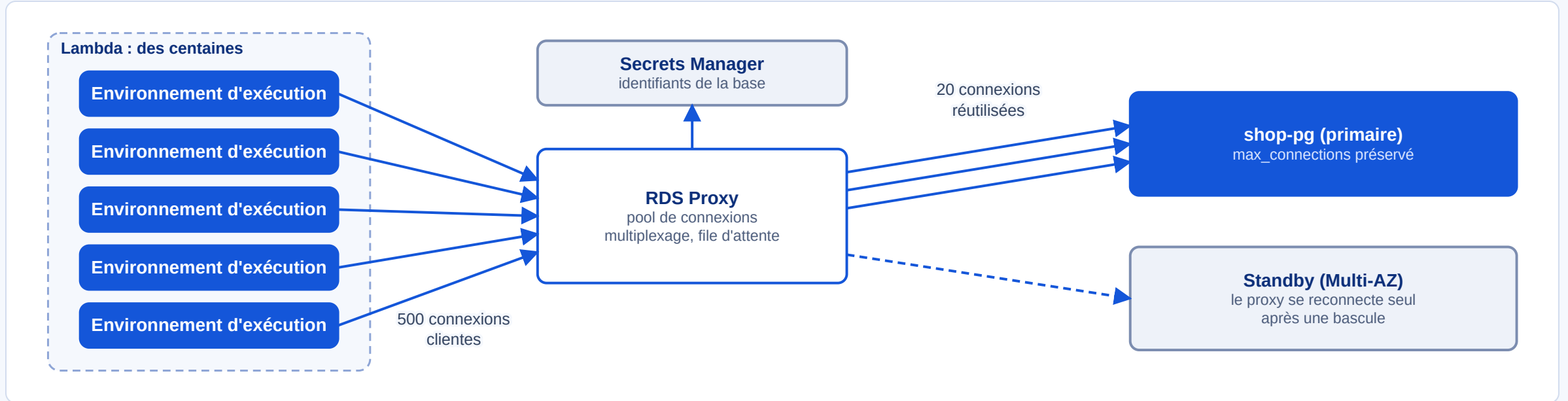
Déploiements blue/green

- RDS crée une copie (*green*) synchronisée par réplication logique.
- On y applique la montée de version majeure ou le changement de paramètres, on teste.
- **Switchover** : bascule en général en moins d'une minute, endpoints conservés.
- L'ancien environnement reste disponible pour vérification.

Montées de version. Mineures : automatiques si l'option est active, pendant la fenêtre de maintenance. Majeures : toujours manuelles, à tester (snapshot ou blue/green).

Observer la base. CloudWatch Database Insights (qui prend la suite de Performance Insights) montre la charge par requête SQL, par attente et par utilisateur ; Enhanced Monitoring donne les métriques de l'OS.

Schéma : RDS Proxy devant Lambda



Le problème

Chaque environnement Lambda ouvre sa propre connexion. Un pic de trafic crée des centaines de connexions : PostgreSQL atteint `max_connections` et refuse les suivantes.

Ce que fait le proxy

Il garde un petit nombre de connexions ouvertes vers la base et les prête le temps d'une transaction. Les clients en trop attendent dans une file au lieu d'être rejetés.

En cas de bascule

Le proxy garde les connexions des clients et se rebranche sur la nouvelle primaire : l'interruption vue par l'application est plus courte qu'avec le seul DNS.

Console : créer une base PostgreSQL

aws

Aurora and RDS > Databases > Create database

Creation method

☒ Standard create

Engine type

☒ PostgreSQL

Templates

☒ Dev/Test

Availability

☒ Single-AZ DB instance deployment

DB instance identifier

shop-pg

Master username

shopadmin

Credentials management

☒ Managed in AWS Secrets Manager

DB instance class

Burstable : db.t4g.micro ▼

Storage

gp3, 20 GiB, autoscaling activé ▼

Public access

☒ Yes (lab uniquement)

VPC security group

☒ Create new : shop-db-sg

Initial database name

shopdb

Backup retention period

7 days ▼

Create database

Onglet de l'instance	Ce qu'on y trouve
Connectivity & security	Endpoint, port, VPC, security groups, certificat.
Monitoring	CPU, connexions, mémoire, IOPS, Database Insights.
Logs & events	Journaux du moteur, événements (bascule, sauvegarde).
Configuration	Classe, stockage, parameter group, secret du mot de passe.
Maintenance & backups	Fenêtres, snapshots, dernier instant restaurable.

Menu Actions	Effet
Take snapshot	Snapshot manuel.
Restore to point in time	Nouvelle instance à l'instant choisi.
Create read replica	Réplica de lecture.
Reboot (with failover)	Tester la bascule Multi-AZ.

« Easy create » applique des valeurs par défaut ; « Standard create » expose tous les réglages.

\$ Prérequis : variables de l'étape 0 (VPC_ID, SUBNET_IDS)

1. Groupe de sous-réseaux : au moins 2 AZ

```
aws rds create-db-subnet-group --db-subnet-group-name shop-db-subnets \  
  --db-subnet-group-description "Sous-reseaux UtopiosShop" \  
  --subnet-ids $(echo $SUBNET_IDS | tr ',' ' ')
```

2. Security group : PostgreSQL ouvert à votre seule adresse IP

```
export DB_SG=$(aws ec2 create-security-group --group-name shop-db-sg \  
  --description "PostgreSQL shop-pg" --vpc-id $VPC_ID --query GroupId --output text)  
MY_IP=$(curl -s https://checkip.amazonaws.com)  
aws ec2 authorize-security-group-ingress --group-id $DB_SG --protocol tcp --port 5432 --cidr $MY_IP/32
```

3. Version par défaut du moteur

```
PG_VERSION=$(aws rds describe-db-engine-versions --engine postgres --default-only \  
  --query 'DBEngineVersions[0].EngineVersion' --output text)
```

4. Instance : mot de passe géré par Secrets Manager, chiffrée, sauvegardes sur 7 jours

```
aws rds create-db-instance --db-instance-identifiant shop-pg \  
  --engine postgres --engine-version $PG_VERSION --db-instance-class db.t4g.micro \  
  --allocated-storage 20 --storage-type gp3 --storage-encrypted \  
  --master-username shopadmin --manage-master-user-password --db-name shopdb \  
  --db-subnet-group-name shop-db-subnets --vpc-security-group-ids $DB_SG \  
  --backup-retention-period 7 --no-multi-az --publicly-accessible
```

5. Attendre l'état « available » (5 à 10 minutes)

```
aws rds wait db-instance-available --db-instance-identifiant shop-pg  
aws rds describe-db-instances --db-instance-identifiant shop-pg \  
  --query 'DBInstances[0].{etat:DBInstanceStatus,endpoint:Endpoint.Address,az:AvailabilityZone}'
```

`--publicly-accessible` ne sert qu'à tester depuis le poste. En production : `--no-publicly-accessible`, `--multi-az`, `--deletion-protection`.

\$ Prérequis : instance shop-pg disponible, client psql et jq installés

```
# Endpoint de l'instance
export DB_HOST=$(aws rds describe-db-instances --db-instance-identifier shop-pg \
  --query 'DBInstances[0].Endpoint.Address' --output text)

# Mot de passe maître : lu dans Secrets Manager
SECRET_ARN=$(aws rds describe-db-instances --db-instance-identifier shop-pg \
  --query 'DBInstances[0].MasterUserSecret.SecretArn' --output text)
export PGPASSWORD=$(aws secretsmanager get-secret-value --secret-id $SECRET_ARN \
  --query SecretString --output text | jq -r .password)

# Connexion chiffrée et jeu d'essai
psql "host=$DB_HOST port=5432 dbname=shopdb user=shopadmin sslmode=require" <<'SQL'
CREATE TABLE products (
  id      serial PRIMARY KEY,
  name    text NOT NULL,
  price   numeric(8,2) NOT NULL CHECK (price >= 0)
);
INSERT INTO products (name, price)
VALUES ('Clavier mecanique', 89.90), ('Souris sans fil', 34.50);
SELECT * FROM products ORDER BY price DESC;
SELECT ssl, version AS tls FROM pg_stat_ssl WHERE pid = pg_backend_pid();
SQL
```

Ce que montre l'exemple

- L'application n'a besoin que de l'**endpoint**, du port et d'identifiants.
- Le mot de passe n'apparaît nulle part : il est lu dans Secrets Manager au moment de la connexion.
- `sslmode=require` chiffre la connexion ; la dernière requête le confirme (`ssl = t`).

Connexion qui expire ? Vérifier dans l'ordre : instance `available`, security group ouvert à votre IP actuelle (elle change si vous changez de réseau : relancer alors l'étape 2), accès public actif.

Installer psql. macOS : `brew install libpq` puis `brew link --force libpq`. Debian, Ubuntu, WSL2 : `sudo apt-get install -y postgresql-client`.

```
$ Prérequis : instance shop-pg disponible, variable DB_SG définie (CLI 1/4)

# Snapshot manuel, par exemple avant une migration de schéma
aws rds create-db-snapshot --db-instance-identifiant shop-pg \
  --db-snapshot-identifiant shop-pg-avant-migration
aws rds wait db-snapshot-available \
  --db-snapshot-identifiant shop-pg-avant-migration

# Dernier instant restaurable
aws rds describe-db-instances --db-instance-identifiant shop-pg \
  --query 'DBInstances[0].LatestRestorableTime'

# Restauration PITR : crée une NOUVELLE instance, shop-pg-pitr
aws rds restore-db-instance-to-point-in-time \
  --source-db-instance-identifiant shop-pg \
  --target-db-instance-identifiant shop-pg-pitr \
  --use-latest-restorable-time \
  --db-subnet-group-name shop-db-subnets \
  --vpc-security-group-ids $DB_SG --db-instance-class db.t4g.micro

# Réplica de lecture
aws rds create-db-instance-read-replica \
  --db-instance-identifiant shop-pg-ro \
  --source-db-instance-identifiant shop-pg

# Les trois instances, chacune avec son endpoint
aws rds describe-db-instances --output table --query \
  'DBInstances[].[DBInstanceIdentifier,DBInstanceStatus,Endpoint.Address]'
```

Ce qu'il faut observer

- `shop-pg-pitr` est une instance indépendante : même contenu qu'à l'instant choisi, autre endpoint. Pour restaurer à une heure précise, remplacer `--use-latest-restorable-time` par `--restore-time` suivi d'une date ISO 8601 en UTC.
- `shop-pg-ro` refuse les écritures : `cannot execute INSERT in a read-only transaction`.
- Le retard du réplica se lit dans la métrique `ReplicaLag`.

Après une restauration. Soit l'application pointe vers le nouvel endpoint, soit on renomme : l'ancienne instance en `shop-pg-old`, la nouvelle en `shop-pg` (`modify-db-instance --new-db-instance-identifiant`).

Trois instances tournent à la fin de cette diapositive : penser au nettoyage.

\$ Prérequis : instance shop-pg disponible

```
# Passer en Multi-AZ (sans coupure), puis provoquer une bascule
aws rds modify-db-instance --db-instance-identifiant shop-pg \
  --multi-az --apply-immediately
sleep 60; aws rds wait db-instance-available --db-instance-identifiant shop-pg
aws rds reboot-db-instance --db-instance-identifiant shop-pg --force-failover
aws rds wait db-instance-available --db-instance-identifiant shop-pg
aws rds describe-db-instances --db-instance-identifiant shop-pg \
  --query 'DBInstances[0].[AvailabilityZone,SecondaryAvailabilityZone]'
aws rds describe-events --source-identifiant shop-pg --source-type db-instance \
  --duration 60 --query 'Events[].[Date,Message]' --output table

# Parameter group : journaliser les requêtes de plus de 500 ms
PG_FAMILY=$(aws rds describe-db-engine-versions --engine postgres --default-only \
  --query 'DBEngineVersions[0].DBParameterGroupFamily' --output text)
aws rds create-db-parameter-group --db-parameter-group-name shop-pg-params \
  --db-parameter-group-family $PG_FAMILY --description "Parametres UtopiosShop"
aws rds modify-db-parameter-group --db-parameter-group-name shop-pg-params \
  --parameters \
    "ParameterName=log_min_duration_statement,ParameterValue=500,ApplyMethod=immediate"

# Associer le groupe et exporter les journaux vers CloudWatch Logs
aws rds modify-db-instance --db-instance-identifiant shop-pg \
  --db-parameter-group-name shop-pg-params --apply-immediately \
  --cloudwatch-logs-export-configuration '{"EnableLogTypes":["postgreql"]}'

# Le changement de parameter group ne prend effet qu'après un redémarrage
sleep 60; aws rds wait db-instance-available --db-instance-identifiant shop-pg
aws rds reboot-db-instance --db-instance-identifiant shop-pg
```

Ce qu'il faut observer

- Après la bascule, les deux zones sont **inversées** et l'endpoint n'a pas changé.
- Les événements retracent la bascule : *Multi-AZ instance failover started*, puis *completed*.
- Une session `psql` ouverte est coupée : l'application doit savoir se reconnecter.

Paramètres

- **Dynamique** : appliqué à chaud. **Statique** : au prochain redémarrage (`pending-reboot`).
- Le groupe par défaut n'est pas modifiable : toujours créer le sien.
- La famille (`postgres17`, `postgres18` ...) doit correspondre à la version majeure du moteur.

Bonnes pratiques

- Production : Multi-AZ, sous-réseaux privés, chiffrement, protection contre la suppression.
- Rétention des sauvegardes adaptée au besoin métier, et **restauration testée** régulièrement.
- Mot de passe maître géré par Secrets Manager ; comptes applicatifs à droits limités.
- Pool de connexions côté application, RDS Proxy devant Lambda.
- Alarmes sur `CPUUtilization`, `FreeStorageSpace`, `DatabaseConnections`, `FreeableMemory`, `ReplicaLag`.
- Autoscaling du stockage : une base à court d'espace s'arrête.
- Montées de version majeures préparées en blue/green.

Pièges fréquents

- Classe `db.t` en production : les crédits CPU s'épuisent sous charge soutenue.
- Lire sur un réplica juste après une écriture : la donnée n'est peut-être pas encore répliquée.
- `max_connections` saturé par des connexions non libérées.

\$ Nettoyage du module RDS (après le module Monitoring)

```
# Instances secondaires créées dans les exemples
for DB in shop-pg-ro shop-pg-pitr; do
    aws rds delete-db-instance --db-instance-identifiant $DB \
        --skip-final-snapshot
done
for DB in shop-pg-ro shop-pg-pitr; do
    aws rds wait db-instance-deleted \
        --db-instance-identifiant $DB
done

# Instance principale, sans snapshot final
aws rds delete-db-instance --db-instance-identifiant shop-pg \
    --skip-final-snapshot --delete-automated-backups
aws rds wait db-instance-deleted \
    --db-instance-identifiant shop-pg

# Snapshot manuel, parameter group, subnet group, SG
aws rds delete-db-snapshot \
    --db-snapshot-identifiant shop-pg-avant-migration
aws rds delete-db-parameter-group \
    --db-parameter-group-name shop-pg-params
aws rds delete-db-subnet-group \
    --db-subnet-group-name shop-db-subnets
aws ec2 delete-security-group --group-id $DB_SG
unset PGPASSWORD
```

Module 5

Amazon DynamoDB

Base NoSQL clé-valeur et documents, serverless, à latence constante quelle que soit l'échelle.

- Modèle de données : tables, items, clés
- Partitionnement et choix de la clé
- Modes de capacité et calcul des unités
- Index secondaires, Query et Scan
- Streams, TTL, sauvegardes, tables globales
- Modélisation, exemple console puis CLI

DynamoDB en bref

DynamoDB stocke des **items** identifiés par une clé primaire et les retrouve en quelques millisecondes, que la table contienne mille ou mille milliards d'items. Pas de serveur, pas de version, pas de fenêtre de maintenance.

- **Serverless** : capacité à la demande ou provisionnée, données répliquées sur 3 AZ, chiffrées par défaut.
- **Accès par API HTTPS** (pas de connexion persistante) : idéal avec Lambda.
- **Schéma souple** : seuls les attributs de clé sont imposés ; chaque item peut avoir ses propres attributs.
- **Contrepartie** : pas de jointure, pas d'agrégat, requêtes limitées aux clés et aux index. Le modèle se conçoit à partir des requêtes.

Bien adapté

- Paniers, sessions, profils, commandes.
- Catalogues à très fort trafic, classements, IoT.
- Charges imprévisibles ou très variables.
- Architectures serverless et événementielles.

Mal adapté

- Requêtes ad hoc, reporting, analytique.
- Modèle très relationnel aux accès changeants.
- Items volumineux (limite : 400 Ko par item).

Monde relationnel	DynamoDB	Différence à retenir
Table	Table	Pas de schéma au-delà de la clé primaire.
Ligne, colonne	Item, attribut	Deux items d'une même table peuvent avoir des attributs différents.
Clé primaire	Clé de partition, plus clé de tri facultative	Elle détermine où la donnée est stockée et comment on peut la lire.
Index	GSI, LSI	Une copie de la table organisée autrement, pas un simple accélérateur.
Jointure	Aucune	On regroupe à l'écriture ce qui doit se lire ensemble.

Clé primaire : deux formes

- **Clé de partition seule** (HASH) : chaque item a une valeur unique. Accès par clé exacte.
- **Clé composite** : clé de partition + **clé de tri** (RANGE). Plusieurs items partagent la même partition et sont triés par la clé de tri : on peut alors lire une page.

Types d'attributs

- Scalaires : **S** chaîne, **N** nombre, **B** binaire, **BOOL**, **NULL**.
- Documents : **M** map, **L** liste (imbrication jusqu'à 32 niveaux).
- Ensembles : **SS**, **NS**, **BS**.
- Les clés sont forcément de type S, N ou B.

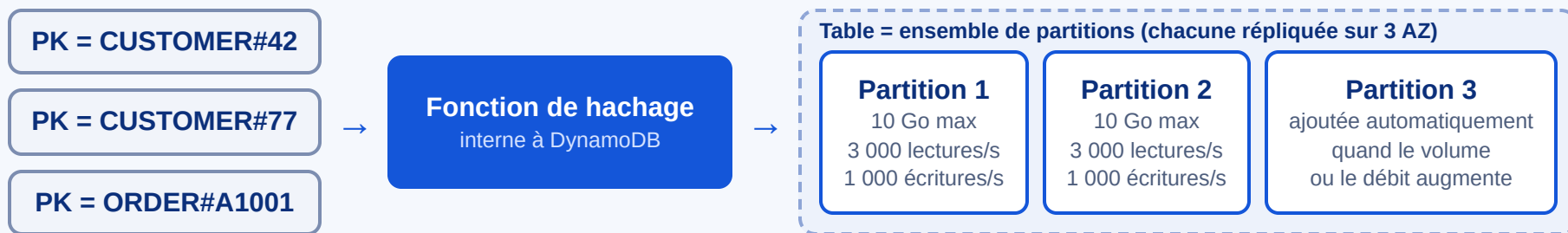
PK (partition)	SK (tri)	Autres attributs
CUSTOMER#42	PROFILE	name=Léa Martin, email=lea@example.org
CUSTOMER#42	ORDER#2026-09-18#A0977	status=DELIVERED, total=54.00
CUSTOMER#42	ORDER#2026-10-01#A1001	status=PAID, total=129.90
CUSTOMER#42	ORDER#2026-10-03#A1007	status=PAID, total=34.50, coupon=AUTOMNE
CUSTOMER#77	PROFILE	name=Karim Benali

`{ }` Un item au format de l'API (types explicites)

```
{ "PK":      { "S": "CUSTOMER#42" },
  "SK":      { "S": "ORDER#2026-10-01#A1001" },
  "status":  { "S": "PAID" },
  "total":   { "N": "129.90" },
  "lines":   { "L": [ { "M": { "sku": { "S": "KB-01" }, "qty": { "N": "1" } } } ] }
```

Les items d'une même partition forment une **collection d'items**, triée par SK. Ici, toutes les commandes d'octobre du client 42 se lisent en une requête : `PK = CUSTOMER#42 AND begins_with(SK, ORDER#2026-10)`.

Partitionnement : pourquoi la clé de partition est décisive



Mécanisme

La valeur de la clé de partition est hachée ; le résultat désigne la partition physique. Tous les items de même PK sont stockés ensemble, triés par SK. DynamoDB découpe les partitions seul, sans interruption.

Partition chaude

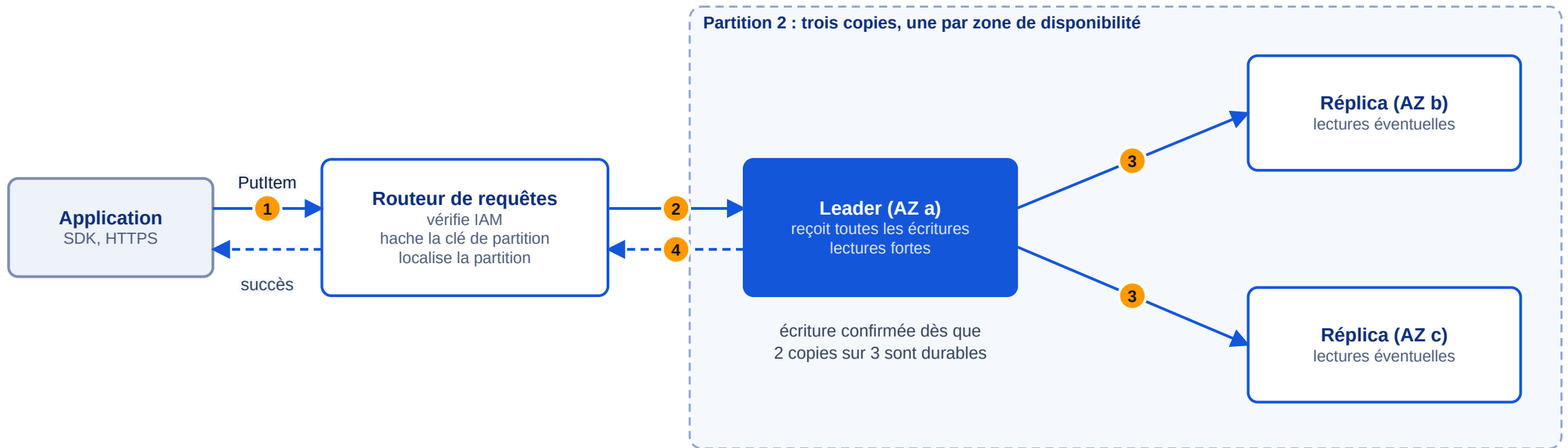
Si une valeur de PK concentre le trafic (un client géant, une date du jour, un statut), sa partition atteint ses limites et les requêtes sont limitées (*throttling*), même si la table est globalement peu chargée.

Bonne clé de partition

Forte cardinalité et accès bien répartis : identifiant de client, de commande, d'appareil. À éviter seuls : statut, pays, date, booléen. Au besoin, suffixer la clé (*write sharding*).

L'**adaptive capacity** réaffecte automatiquement le débit vers les partitions les plus sollicitées et peut isoler un item très demandé : elle atténue le problème, sans corriger une clé mal choisie.

Schéma : le chemin d'une écriture



1. L'application envoie `PutItem` en HTTPS, signé avec ses identifiants IAM.
2. Le routeur vérifie les droits, hache la clé de partition et transmet au leader de la bonne partition.
3. Le leader écrit et réplique vers les deux autres copies.
4. Dès que deux copies sur trois ont enregistré l'écriture, le succès est renvoyé.

Conséquence pour les lectures. Une lecture à cohérence forte est servie par le leader : elle voit toujours la dernière écriture. Une lecture à cohérence éventuelle peut tomber sur un réplica en retard de quelques millisecondes, pour moitié prix.

	À la demande	Provisionné
Principe	Paieement à la requête, aucune capacité à prévoir.	On réserve des RCU et WCU par seconde.
Montée en charge	Instantanée (jusqu'au double du pic précédent, puis progressive).	Auto scaling entre un minimum et un maximum, avec un temps de réaction.
Idéal pour	Trafic nouveau, irrégulier, par pics.	Trafic stable et prévisible, coût optimisé.
Dépassement	Rare (limites par partition).	Throttling au-delà de la capacité réservée.

On peut changer de mode sur une table existante. Le mode à la demande est le choix par défaut raisonnable ; on passe en provisionné quand la charge est connue et régulière.

Classes de table. Standard, ou Standard-IA : stockage moins cher et requêtes plus chères, pour des données rarement lues.

Unités de capacité

- **1 RCU** = 1 lecture fortement cohérente par seconde d'un item jusqu'à **4 Ko** (ou 2 lectures à cohérence éventuelle).
- **1 WCU** = 1 écriture par seconde d'un item jusqu'à **1 Ko**.
- Arrondi à l'unité supérieure, par item. Les transactions coûtent le double.

Opération	Calcul	Coût
Lire un item de 6 Ko, cohérence forte	6 / 4 arrondi à 2	2 RCU
Le même, cohérence éventuelle	2 / 2	1 RCU
Écrire un item de 2,5 Ko	2,5 arrondi à 3	3 WCU
Query renvoyant 40 items de 500 octets	20 Ko au total / 4	5 RCU (2,5 en éventuel)
100 écritures/s d'items de 1,5 Ko	100 × 2	200 WCU

Un **Scan** ou un **FilterExpression** paie pour tout ce qui est **lu**, pas pour ce qui est renvoyé.

DynamoDB

Index secondaires : LSI et GSI

Un index est une copie de la table, organisée selon une autre clé et maintenue automatiquement. Il permet d'autres requêtes que celles de la clé primaire.

	Local Secondary Index (LSI)	Global Secondary Index (GSI)
Clé	Même clé de partition que la table, autre clé de tri .	Clé de partition et clé de tri quelconques .
Création	Uniquement à la création de la table.	À tout moment, suppression possible.
Nombre	5 par table.	20 par table (quota ajustable).
Cohérence	Forte ou éventuelle.	Éventuelle uniquement (propagation asynchrone).
Capacité	Partagée avec la table.	Propre à l'index en mode provisionné.
Contrainte	Collection d'items limitée à 10 Go par clé de partition.	Un GSI sous-dimensionné freine les écritures de la table.
Usage type	Trier les commandes d'un client par montant plutôt que par date.	Retrouver les commandes par statut, par e-mail, par produit.

Projection

`KEYS_ONLY` , `INCLUDE` (liste d'attributs) ou `ALL` .
Projeter moins réduit le stockage et le coût d'écriture de l'index.

Index creux (sparse)

Seuls les items possédant l'attribut de clé de l'index y figurent. Un attribut `pendingSince` posé sur les seules commandes en attente donne un index minuscule et très efficace.

Coût

Chaque écriture dans la table déclenche une écriture dans chaque index concerné. Créer des index pour des requêtes réelles, pas « au cas où ».

Opération	Fonctionnement
GetItem	Un item par sa clé primaire complète. L'accès le plus rapide et le moins cher.
Query	Les items d' une partition (égalité sur PK), avec condition optionnelle sur SK : <code>=</code> , <code><</code> , <code>></code> , <code>BETWEEN</code> , <code>begins_with</code> . Résultat trié par SK, inversable.
Scan	Parcourt toute la table ou tout l'index. Coût et durée proportionnels à la taille. À réserver aux exports et aux petites tables.
BatchGetItem	Jusqu'à 100 items par clés, sur plusieurs tables.
TransactGetItems	Lecture cohérente de plusieurs items (jusqu'à 100).

Pagination. Query et Scan renvoient au plus 1 Mo par appel ; la réponse contient `LastEvaluatedKey` tant qu'il reste des données. La CLI et les SDK proposent des paginateurs.

Expressions

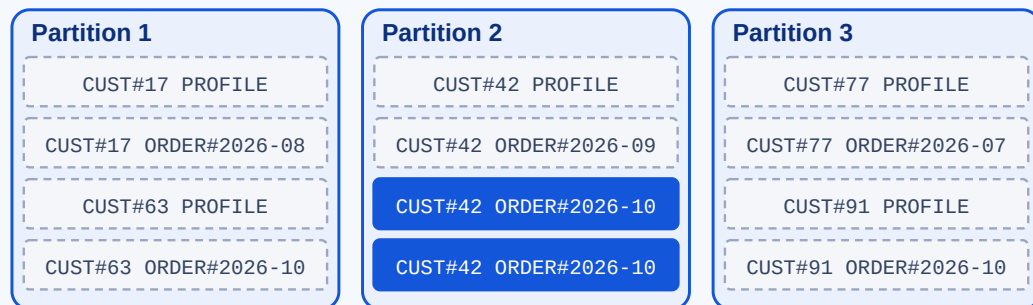
- **KeyConditionExpression** : sélectionne par les clés, détermine ce qui est lu (et facturé).
- **FilterExpression** : appliquée **après** la lecture ; réduit la réponse, pas le coût.
- **ProjectionExpression** : ne renvoie que certains attributs.
- **ConditionExpression** : rend une écriture conditionnelle (verrouillage optimiste, unicité).
- `#nom` et `:valeur` : alias pour les mots réservés (`status` , `name`) et les valeurs.

Cohérence des lectures

- **Éventuelle** (par défaut) : peut renvoyer une donnée datant de quelques millisecondes ; deux fois moins chère.
- **Forte** (`ConsistentRead`) : reflète toutes les écritures validées. Indisponible sur les GSI.

Schéma : Query et Scan, ce qui est réellement lu

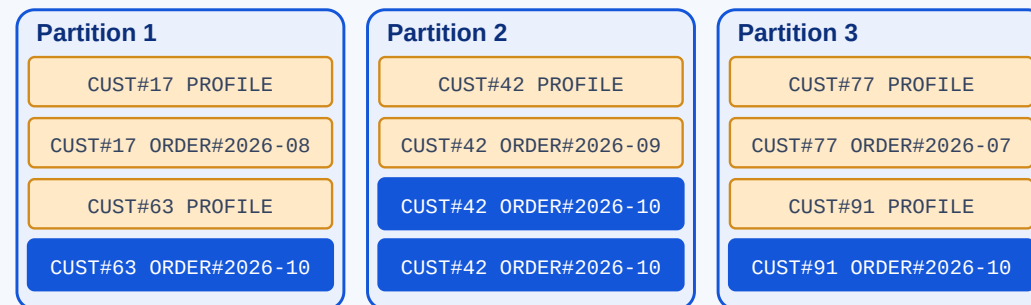
Query : PK = CUSTOMER#42 et SK commence par ORDER#2026-10



2 items lus dans une seule partition

2 items renvoyés, 2 items facturés
durée indépendante de la taille de la table

Scan avec filtre : SK commence par ORDER#2026-10



12 items lus dans toutes les partitions

4 items renvoyés après filtrage, 12 items facturés
durée proportionnelle à la taille de la table

Query. La clé de partition désigne une seule partition ; la condition sur la clé de tri sélectionne une plage contiguë. DynamoDB ne touche que ces items.

Scan. Toutes les partitions sont parcourues. Le filtre s'applique après la lecture : il réduit la réponse, ni le coût ni la durée. Sur une table de 50 Go, le même Scan lit 50 Go.

TTL

Un attribut contenant une date d'expiration (epoch en secondes). DynamoDB supprime les items expirés en arrière-plan, sans coût d'écriture, en général sous quelques jours. Sessions, paniers abandonnés, journaux.

DynamoDB Streams

Journal ordonné des modifications (ancienne et nouvelle image), conservé 24 heures. Branché sur Lambda : réagir à chaque commande, alimenter une recherche, répliquer. Alternative : Kinesis Data Streams.

Sauvegardes

PITR : restauration à la seconde près sur une fenêtre jusqu'à 35 jours. **Sauvegardes à la demande** : conservées jusqu'à suppression. La restauration crée une nouvelle table.

Transactions

`TransactWriteItems` : jusqu'à 100 écritures tout-ou-rien, sur plusieurs tables, avec conditions. Débit un stock et créer la commande en une opération ACID. Coût doublé.

Tables globales

Réplication multi-région, chaque région accepte lectures et écritures. Cohérence éventuelle entre régions (dernière écriture gagnante), ou cohérence forte multi-région en option.

DAX

Cache en mémoire compatible avec l'API DynamoDB : lectures en microsecondes pour les charges très intensives en lecture. Cluster à dimensionner, dans le VPC.

Export et import S3

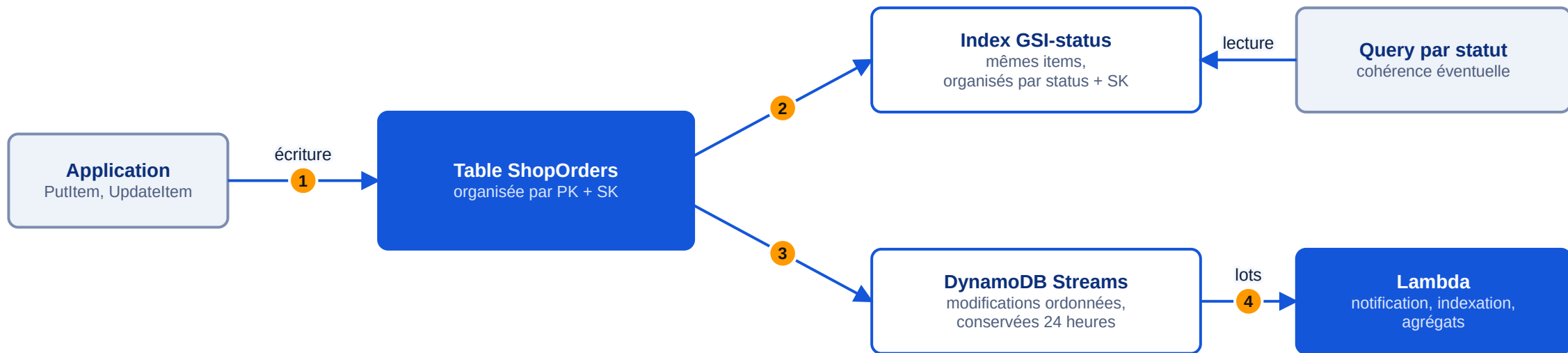
Export d'une table vers S3 sans consommer de capacité (analytique avec Athena). Import en masse depuis S3 vers une nouvelle table.

PartiQL

Syntaxe proche du SQL pour lire et écrire. Pratique, mais les règles restent celles de DynamoDB : un `SELECT` sans clé de partition déclenche un Scan.

Sécurité

Chiffrement au repos systématique (clé AWS ou KMS). Contrôle par IAM, jusqu'au niveau de l'item ou de l'attribut. Accès privé par VPC endpoint. Protection contre la suppression.



1. L'écriture est validée dans la table, et confirmée à l'application.
2. Elle est recopiée de façon asynchrone dans chaque index global concerné : une Query sur l'index peut avoir quelques millisecondes de retard.

3. Si Streams est actif, la modification (ancienne et nouvelle image) est ajoutée au flux, dans l'ordre pour une même clé.

4. Lambda lit le flux par lots et déclenche les traitements : notification, indexation dans un moteur de recherche, mise à jour d'agrégats.

Ni l'index ni le flux ne ralentissent l'écriture, puisqu'ils sont alimentés après coup. En contrepartie, ce que lit l'index n'est jamais garanti à jour à l'instant près.

Démarche

1. Lister les **modes d'accès** de l'application, lectures et écritures.
2. Choisir PK et SK pour que chaque accès fréquent soit un GetItem ou une Query.
3. Ajouter un GSI par accès qui ne tient pas dans la clé primaire.
4. Dénormaliser : dupliquer ce qui doit se lire ensemble.

Single-table design

Plusieurs types d'entités dans une même table, avec des clés génériques (**PK** , **SK**) et des préfixes. Une seule requête ramène un client et ses commandes : c'est la « jointure » précalculée.

Ce modèle est efficace mais rigide : un nouvel accès non prévu peut exiger un nouvel index ou une migration. Plusieurs tables restent un choix valable quand les entités sont indépendantes.

Mode d'accès UtopiosShop	Opération
Profil d'un client	GetItem <code>PK=CUSTOMER#42, SK=PROFILE</code>
Commandes d'un client, récentes d'abord	Query <code>PK=CUSTOMER#42, begins_with(SK, ORDER#)</code> , ordre inversé
Commandes d'un client sur un mois	Query <code>begins_with(SK, ORDER#2026-10)</code>
Client et commandes en un appel	Query <code>PK=CUSTOMER#42</code>
Lignes d'une commande	Query <code>PK=ORDER#A1001, begins_with(SK, ITEM#)</code>
Commandes à expédier (statut PAID)	Query sur le GSI <code>GSI-status : status=PAID</code> , triées par date
Chiffre d'affaires par mois	Hors DynamoDB : export S3 + Athena, ou agrégat tenu à jour par Streams + Lambda

Format de la clé de tri. `ORDER#date ISO#identifiant` : le tri lexicographique devient un tri chronologique, et `begins_with` filtre par année, mois ou jour.

aws

DynamoDB > Tables > Create table

Table name

ShopOrders

Partition key

PK (String)

Sort key (optional)

SK (String)

Table settings

Customize settings

Capacity mode

On-demand

Secondary indexes

Create global index : GSI-status

Encryption at rest

Owned by Amazon DynamoDB

Deletion protection

Turn on

Create table

aws

Explore items > ShopOrders

Mode

Query

Table or index

ShopOrders ▼

PK (Partition key)

CUSTOMER#42

SK (Sort key)

Begins with : ORDER#2026-10 ▼

Run

Onglet de la table	Ce qu'on y fait
Overview	Clés, mode de capacité, nombre d'items (estimé), taille.
Indexes	Créer ou supprimer un GSI, suivre son remplissage.
Monitor	Capacité consommée, latence, requêtes limitées ; Contributor Insights.
Global tables	Ajouter une région de réplication.
Backups	Activer le PITR, créer une sauvegarde, restaurer.
Exports and streams	Export S3, DynamoDB Streams, Kinesis.
Additional settings	TTL, classe de table, capacité, chiffrement.

Explore items permet de créer et modifier des items dans un éditeur (formulaire ou JSON) et d'exécuter Query, Scan ou PartiQL. Le bandeau de résultat indique les unités de capacité consommées : utile pour comparer une Query et un Scan.

\$ Prérequis : variables de l'étape 0

1. Table à clé composite, mode à la demande. Seuls les attributs de clé sont déclarés.

```
aws dynamodb create-table --table-name ShopOrders \  
  --attribute-definitions AttributeName=PK,AttributeType=S AttributeName=SK,AttributeType=S \  
  --key-schema AttributeName=PK,KeyType=HASH AttributeName=SK,KeyType=RANGE \  
  --billing-mode PAY_PER_REQUEST  
aws dynamodb wait table-exists --table-name ShopOrders
```

2. Écrire : la condition refuse d'écraser un item existant

```
aws dynamodb put-item --table-name ShopOrders --condition-expression "attribute_not_exists(PK)" --item \  
  '{"PK":{"S":"CUSTOMER#42"},"SK":{"S":"PROFILE"},"name":{"S":"Lea Martin"},"email":{"S":"lea@example.org"}}'  
aws dynamodb put-item --table-name ShopOrders --condition-expression "attribute_not_exists(PK)" --item \  
  '{"PK":{"S":"CUSTOMER#42"},"SK":{"S":"ORDER#2026-10-01#A1001"},"status":{"S":"PAID"},"total":{"N":"129.90"}}'  
aws dynamodb put-item --table-name ShopOrders --condition-expression "attribute_not_exists(PK)" --item \  
  '{"PK":{"S":"CUSTOMER#42"},"SK":{"S":"ORDER#2026-10-03#A1007"},"status":{"S":"PAID"},"total":{"N":"34.50"}}'
```

3. Lire un item par sa clé complète, en cohérence forte

```
aws dynamodb get-item --table-name ShopOrders --consistent-read \  
  --key '{"PK":{"S":"CUSTOMER#42"},"SK":{"S":"PROFILE"}}'
```

4. Query : commandes d'octobre 2026 du client 42, les plus récentes d'abord

```
aws dynamodb query --table-name ShopOrders --no-scan-index-forward \  
  --key-condition-expression "PK = :pk AND begins_with(SK, :sk)" \  
  --expression-attribute-values '{":pk":{"S":"CUSTOMER#42"},":sk":{"S":"ORDER#2026-10"}}' \  
  --return-consumed-capacity TOTAL
```

5. Scan filtré : même résultat possible, mais TOUTE la table est lue (comparer ScannedCount et Count)

```
aws dynamodb scan --table-name ShopOrders --filter-expression "begins_with(SK, :sk)" \  
  --expression-attribute-values '{":sk":{"S":"ORDER#"}}' --return-consumed-capacity TOTAL \  
  --query '{renvoies:Count,lus:ScannedCount,capacite:ConsumedCapacity.CapacityUnits}'
```

\$ Suite : la table ShopOrders existe et contient les items précédents

6. Mise à jour conditionnelle : PAID -> SHIPPED uniquement si le statut est encore PAID

(status est un mot réservé : alias #s obligatoire)

```
aws dynamodb update-item --table-name ShopOrders \
  --key '{"PK":{"S":"CUSTOMER#42"},"SK":{"S":"ORDER#2026-10-01#A1001"}}' \
  --update-expression "SET #s = :new, shippedAt = :ts" --condition-expression "#s = :old" \
  --expression-attribute-names '{"#s":"status"}' \
  --expression-attribute-values '{":new":{"S":"SHIPPED"},":old":{"S":"PAID"},":ts":{"S":"2026-10-04T09:30:00Z"}}' \
  --return-values ALL_NEW
```

Relancer la commande : ConditionalCheckFailedException, la commande n'est plus PAID

7. Ajouter un index global : commandes par statut, triées par SK (donc par date)

```
aws dynamodb update-table --table-name ShopOrders \
  --attribute-definitions AttributeName=status,AttributeType=S AttributeName=SK,AttributeType=S \
  --global-secondary-index-updates '[{"Create":{"IndexName":"GSI-status",
    "KeySchema":[{"AttributeName":"status","KeyType":"HASH"}, {"AttributeName":"SK","KeyType":"RANGE"}],
    "Projection":{"ProjectionType":"ALL"}}}]'
```

8. Attendre que l'index soit ACTIVE (remplissage en arrière-plan), puis l'interroger

```
until [ "$(aws dynamodb describe-table --table-name ShopOrders \
  --query 'Table.GlobalSecondaryIndexes[0].IndexStatus' --output text)" = "ACTIVE" ]; do sleep 10; done
aws dynamodb query --table-name ShopOrders --index-name GSI-status \
  --key-condition-expression "#s = :s" --expression-attribute-names '{"#s":"status"}' \
  --expression-attribute-values '{":s":{"S":"PAID"}}' --projection-expression "PK, SK, total"
```

9. PartiQL : même lecture que l'étape 4, en syntaxe SQL

```
aws dynamodb execute-statement --statement \
  "SELECT SK, total FROM ShopOrders WHERE PK = 'CUSTOMER#42' AND begins_with(SK, 'ORDER#')"
```

Le profil du client n'a pas d'attribut `status` : il n'apparaît pas dans `GSI-status`. C'est le principe de l'index creux.

\$ Fonctions de la table

```
# TTL : les items portant expiresAt (epoch, secondes)
# sont supprimés après cette date
aws dynamodb update-time-to-live --table-name ShopOrders \
  --time-to-live-specification \
    "Enabled=true,AttributeName=expiresAt"

# Panier qui expire dans 1 heure
aws dynamodb put-item --table-name ShopOrders --item \
  '{"PK":{"S":"CUSTOMER#42"},"SK":{"S":"CART"},
    "expiresAt":{"N":"'$( (date +%s) + 3600 )'"}'

# Restauration à un instant donné (PITR)
aws dynamodb update-continuous-backups \
  --table-name ShopOrders \
  --point-in-time-recovery-specification \
    PointInTimeRecoveryEnabled=true

# Sauvegarde à la demande
aws dynamodb create-backup --table-name ShopOrders \
  --backup-name shop-orders-avant-migration

# Streams : journal des modifications pour Lambda
aws dynamodb update-table --table-name ShopOrders \
  --stream-specification \
    StreamEnabled=true,StreamViewType=NEW_AND_OLD_IMAGES
```

\$ Écriture par lot (25 items au maximum par appel)

```
cat > batch.json <<'EOF'
{"ShopOrders":[
  {"PutRequest":{"Item":{"PK":{"S":"ORDER#A1001"},
    "SK":{"S":"ITEM#1"},"sku":{"S":"KB-01"},"qty":{"N":"1"}}}},
  {"PutRequest":{"Item":{"PK":{"S":"ORDER#A1001"},
    "SK":{"S":"ITEM#2"},"sku":{"S":"MS-07"},"qty":{"N":"2"}}}}
]}
EOF
aws dynamodb batch-write-item --request-items file://batch.json
aws dynamodb query --table-name ShopOrders \
  --key-condition-expression "PK = :pk" \
  --expression-attribute-values '{":pk":{"S":"ORDER#A1001"}}'
```

\$ Nettoyage du module DynamoDB

```
BACKUP_ARN=$(aws dynamodb list-backups \
  --table-name ShopOrders \
  --query 'BackupSummaries[0].BackupArn' --output text)
aws dynamodb delete-backup --backup-arn $BACKUP_ARN
aws dynamodb delete-table --table-name ShopOrders
aws dynamodb wait table-not-exists --table-name ShopOrders
```

batch-write-item peut renvoyer des `UnprocessedItems` : il faut les renvoyer, avec une attente croissante entre les essais.

DynamoDB

RDS ou DynamoDB, et bonnes pratiques

Critère	RDS / Aurora	DynamoDB
Modèle	Relationnel, schéma strict	Clé-valeur, documents
Requêtes	SQL libre, jointures, agrégats	Par clé et index, définies à l'avance
Montée en charge	Verticale, réplicas de lecture	Horizontale, automatique
Exploitation	Instance, versions, maintenance	Aucune
Connexions	Persistantes, pool à gérer	API HTTPS sans état
Coût à vide	Instance facturée à l'heure	Stockage seul (mode à la demande)
Transactions	ACID complètes	ACID, limitées à 100 items

Les deux cohabitent. Dans UtopiosShop : catalogue, stocks et facturation dans PostgreSQL (relations, reporting) ; paniers, sessions et commandes à fort débit dans DynamoDB.

Bonnes pratiques DynamoDB

- Concevoir à partir des modes d'accès ; clé de partition à forte cardinalité.
- Query plutôt que Scan ; pas de `FilterExpression` pour compenser une mauvaise clé.
- Items petits ; les gros contenus vont dans S3, l'item garde la référence.
- Écritures conditionnelles pour l'idempotence et la concurrence.
- PITR et protection contre la suppression en production.
- Reprises avec attente exponentielle (les SDK le font).
- Alarmes sur les requêtes limitées et les erreurs système ; Contributor Insights pour repérer les clés chaudes.

Piège. Reproduire un schéma relationnel (une table par entité, des « jointures » dans le code) cumule les défauts des deux mondes.

Module 6

Monitoring et observabilité

Savoir ce qui se passe, être prévenu quand cela se dégrade, comprendre pourquoi.

- Panorama des outils et des questions auxquelles ils répondent
- CloudWatch : métriques, alarmes, logs, tableaux de bord
- Métriques clés d'ECS, EKS, Lambda, RDS, DynamoDB
- Traces : X-Ray et Application Signals
- Container Insights, Lambda Insights, Database Insights
- Audit et réaction : CloudTrail, EventBridge

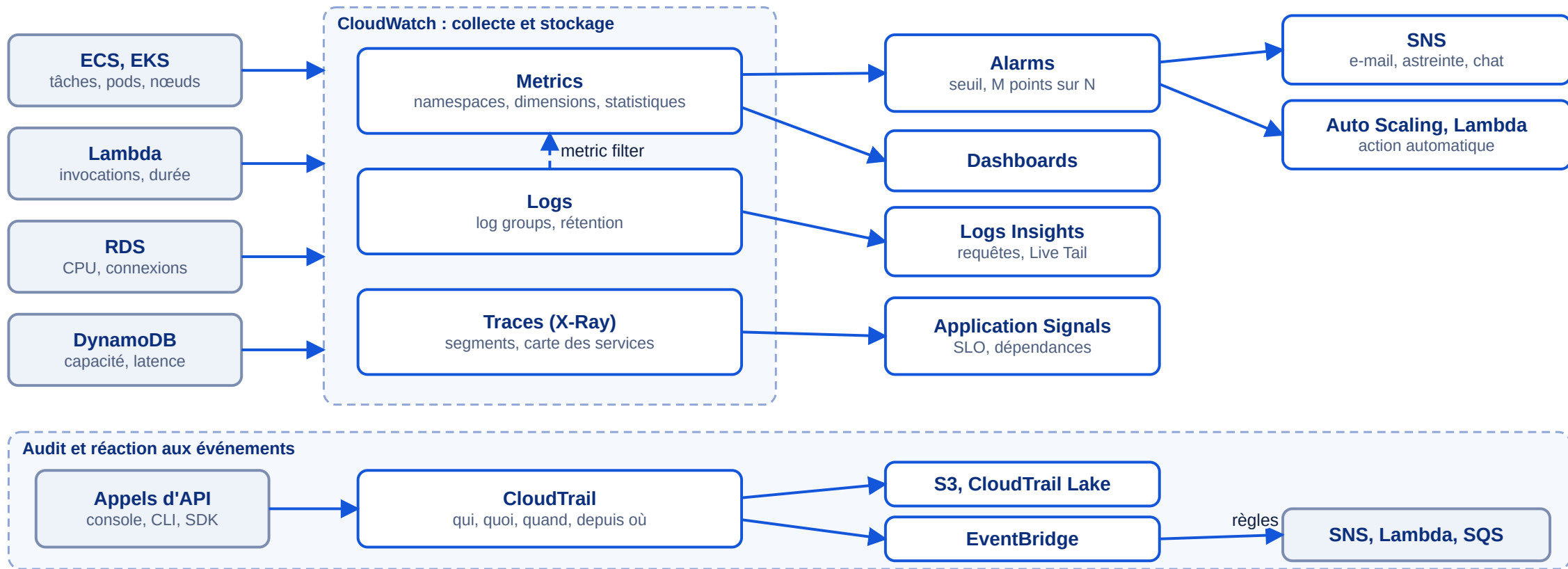
Panorama : quel outil pour quelle question

Outil	Type de signal	Question à laquelle il répond
CloudWatch Metrics	Métriques	Combien ? À quelle vitesse ? Le CPU, la latence, le taux d'erreur évoluent-ils ?
CloudWatch Alarms	Alerte	Un seuil est-il franchi ? Qui prévenir, quelle action lancer ?
CloudWatch Logs, Logs Insights	Journaux	Que s'est-il passé exactement, dans quel ordre, avec quel message d'erreur ?
X-Ray, Application Signals	Traces	Où part le temps d'une requête qui traverse plusieurs services ? Quel appel échoue ?
Container Insights	Métriques et logs	Quel cluster, service, tâche, pod ou conteneur consomme ou redémarre ?
Lambda Insights	Métriques système	Mémoire, CPU, réseau et démarrages à froid d'une fonction.
Database Insights	Charge de la base	Quelle requête SQL, quelle attente, quel utilisateur charge la base ?
CloudTrail	Audit	Qui a fait quel appel d'API, quand, depuis où ?
EventBridge	Événements	Comment réagir automatiquement à un changement d'état ?
Managed Prometheus, Managed Grafana	Métriques, visualisation	Comment garder l'outillage Prometheus et Grafana sans l'exploiter soi-même ?

Trois piliers. Métriques (tendances, alertes), logs (détail), traces (parcours d'une requête). Le *monitoring* dit qu'il y a un problème ; l'*observabilité* permet d'en trouver la cause sans l'avoir prévue.

Ne pas confondre **CloudWatch** (comportement des ressources) et **CloudTrail** (actions effectuées sur le compte).

Schéma : le circuit des signaux



Haut du schéma. Les services émettent métriques, logs et traces vers CloudWatch. Les alarmes lisent les métriques et déclenchent une notification ou une action ; un metric filter transforme des lignes de log en métrique.

Bas du schéma. CloudTrail enregistre chaque appel d'API, l'archive dans S3 et le publie sur EventBridge, où des règles déclenchent des cibles.

Anatomie d'une métrique

- **Namespace** : le conteneur (`AWS/Lambda` , `AWS/RDS` , `Shop/App`).
- **Nom** : `Errors` , `CPUUtilization` .
- **Dimensions** : paires nom-valeur qui identifient la ressource (`FunctionName=shop-hello`). Chaque combinaison est une métrique distincte.
- **Points de données** : horodatage, valeur, unité.

Statistique et période

- On ne lit pas les points bruts, mais une **statistique** sur une **période** : `Average` , `Sum` , `Minimum` , `Maximum` , `SampleCount` , centiles (`p95` , `p99`).
- Bon réflexe : `Sum` pour des compteurs (erreurs), centiles pour des latences, `Average` ou `Maximum` pour des taux d'utilisation.

Granularité des points

Durée de conservation

Moins de 60 s (haute résolution)

3 heures

1 minute

15 jours

5 minutes

63 jours

1 heure

455 jours (15 mois)

Métriques personnalisées

- API `PutMetricData` (CLI, SDK).
- **Embedded Metric Format** : un log JSON structuré que CloudWatch transforme en métrique. Le bon choix pour Lambda et les conteneurs.
- Agent CloudWatch, OpenTelemetry.
- **Metric math** : calculer un taux d'erreur à partir de deux métriques.

Coût. Chaque métrique personnalisée est facturée. Une dimension à forte cardinalité (identifiant de client) crée des milliers de métriques.

Métriques clés par service

Service	Namespace	Métriques à suivre	Signal d'alerte typique
ECS	AWS/ECS, ECS/ContainerInsights	CPUUtilization, MemoryUtilization, RunningTaskCount, PendingTaskCount, DeploymentCount	Mémoire au-dessus de 85 %, nombre de tâches actives inférieur au nombre voulu.
ALB	AWS/ApplicationELB	TargetResponseTime, HTTPCode_Target_5XX_Count, HealthyHostCount, RequestCount	Erreurs 5xx en hausse, cibles saines à 0, latence p95 au-dessus de l'objectif.
EKS	ContainerInsights	node_cpu_utilization, node_memory_utilization, pod_number_of_container_restarts, cluster_failed_node_count	Redémarrages de pods répétés, nœud en échec, nœuds saturés.
Lambda	AWS/Lambda	Invocations, Errors, Duration, Throttles, ConcurrentExecutions, IteratorAge, AsyncEventAge	Taux d'erreur, throttles supérieurs à 0, durée p95 proche du timeout, retard de traitement d'un flux.
RDS	AWS/RDS	CPUUtilization, FreeableMemory, FreeStorageSpace, DatabaseConnections, ReadLatency, WriteLatency, ReplicaLag, DBLoad	Espace libre inférieur à 15 %, connexions proches du maximum, retard de réplication.
DynamoDB	AWS/DynamoDB	ConsumedReadCapacityUnits, ConsumedWriteCapacityUnits, ReadThrottleEvents, WriteThrottleEvents, SuccessfulRequestLatency, SystemErrors	Requêtes limitées, erreurs système, latence anormale.
SQS	AWS/SQS	ApproximateNumberOfMessagesVisible, ApproximateAgeOfOldestMessage	La file s'allonge, le plus vieux message vieillit : les consommateurs ne suivent pas.

Méthode RED pour un service : *Rate* (débit), *Errors* (erreurs), *Duration* (latence). **Méthode USE** pour une ressource : *Utilization*, *Saturation*, *Errors*.

Alerter sur les symptômes perçus par l'utilisateur (erreurs, latence) plutôt que sur les causes (CPU à 80 %), qui servent au diagnostic.

Anatomie d'une alarme

- **Métrique, statistique** et **période** (60 s).
- **Seuil** et opérateur de comparaison.
- **Datapoints to alarm** : « M points sur N » (2 sur 3) pour ignorer un pic isolé.
- **Données manquantes** : `missing`, `notBreaching`, `breaching`, `ignore`. Pour un compteur d'erreurs, l'absence de données signifie « aucune erreur » : `notBreaching`.
- **Actions** à chaque changement d'état.



Type	Usage
Seuil statique	Valeur fixe : erreurs $\geq$ 1, espace libre $<$ 5 Go.
Détection d'anomalie	Bande apprise sur l'historique : utile quand le trafic a une saisonnalité.
Metric math	Alarme sur une expression : taux d'erreur = erreurs / invocations.
Alarme composite	Combine des alarmes (ET, OU) pour réduire le bruit : alerter si erreurs et latence.

Actions possibles

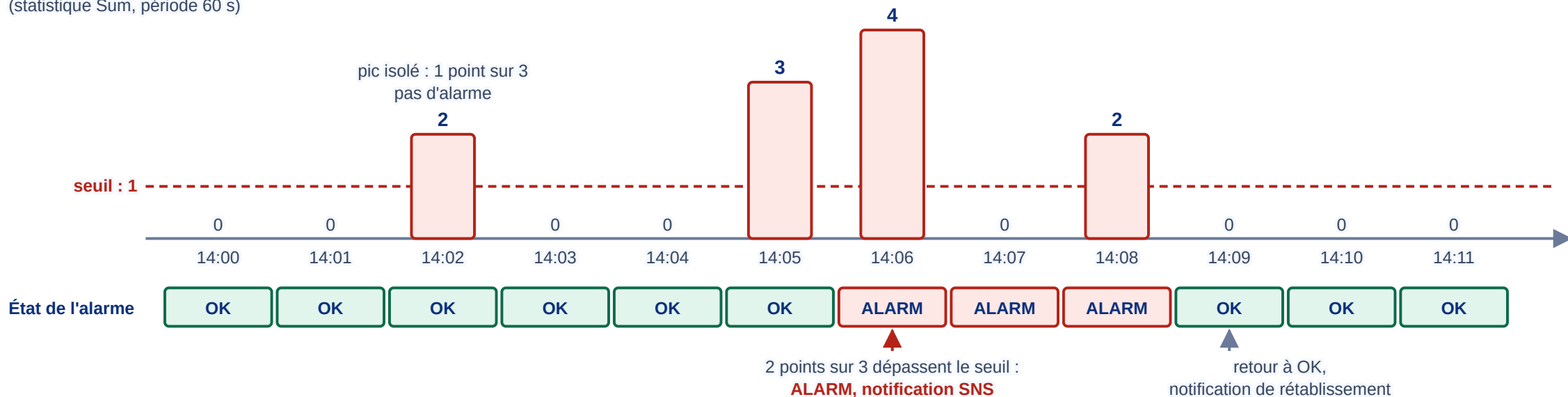
- **SNS** : e-mail, SMS, webhook, outil d'astreinte, messagerie d'équipe.
- **Auto Scaling** : ajouter ou retirer de la capacité.
- **Lambda** : remédiation automatique.
- **Systems Manager** : ouvrir un incident ou un OpsItem.

Une alarme utile est **actionnable** : quelqu'un sait quoi faire en la recevant. Sinon, c'est un graphique de tableau de bord.

Schéma : comment une alarme est évaluée

Erreurs par minute

(statistique Sum, période 60 s)



Réglage de l'exemple

Métrique `Errors`, statistique `Sum`, période 60 s, seuil ≥ 1 , **2 points sur 3**.

Évaluation

À chaque minute, CloudWatch regarde les 3 derniers points. Si au moins 2 dépassent le seuil, l'état passe à ALARM ; sinon il revient à OK.

Effet

Le pic isolé de 14:02 ne réveille personne. L'incident de 14:05 à 14:08 est signalé dès le deuxième point, soit un délai de 2 minutes.

Organisation

- **Log group** : un par application ou ressource (`/aws/lambda/shop-hello` , `/ecs/shop-web`). Porte la rétention, le chiffrement, les filtres.
- **Log stream** : une source au sein du groupe (un environnement Lambda, une tâche ECS).
- **Log event** : une ligne horodatée.

Qui écrit où

Lambda	<code>/aws/lambda/<fonction></code> , automatique
ECS	Groupe défini par le pilote <code>awslogs</code>
EKS, control plane	<code>/aws/eks/<cluster>/cluster</code> , à activer
EKS, pods	<code>/aws/containerinsights/<cluster>/application</code>
RDS	<code>/aws/rds/instance/<id>/postgresql</code> , à activer

Exploiter les logs

- **Logs Insights** : langage de requête pour filtrer, extraire, agréger sur plusieurs groupes.
- **Live Tail** : flux en direct, filtrable.
- **Metric filter** : compte les lignes qui correspondent à un motif et en fait une métrique, donc une alarme possible.
- **Subscription filter** : diffuse en temps réel vers Lambda, Firehose, OpenSearch.
- **Export S3** : archivage long terme à bas coût.

Rétention. Par défaut, les logs ne sont **jamais supprimés**. Fixer une durée (de 1 jour à 10 ans) sur chaque groupe : c'est le premier poste de coût CloudWatch avec l'ingestion.

Logs structurés. Une ligne JSON par événement : Logs Insights découvre les champs automatiquement, et l'on filtre sur `level` , `orderId` , `request_id` . Ne jamais journaliser de secret ni de donnée personnelle.

query Lambda : durée, p95 et mémoire par tranche de 5 minutes

```
filter @type = "REPORT"
| stats count() as invocations,
      avg(@duration) as moyenne_ms,
      pct(@duration, 95) as p95_ms,
      max(@maxMemoryUsed / 1000 / 1000) as memoire_max_mo
by bin(5m)
```

query Lambda : démarrages à froid

```
filter @type = "REPORT" and ispresent(@initDuration)
| stats count() as demarrages_a_froid,
      avg(@initDuration) as init_moyen_ms
by bin(15m)
```

query Logs JSON de shop-hello : champs découverts automatiquement

```
fields @timestamp, request_id, name, stage
| filter stage = "prod"
| sort @timestamp desc
| limit 20
```

query Toutes applications : dernières erreurs

```
fields @timestamp, @logStream, @message
| filter @message like /^(?i)(error|exception|timed out)/
| sort @timestamp desc
| limit 50
```

query ECS, accès nginx : réponses 5xx par minute

```
parse @message '" * * "' as code, taille
| filter code like /^5/
| stats count() as erreurs_5xx by bin(1m)
```

query RDS PostgreSQL : requêtes les plus lentes

```
filter @message like /duration:/
| parse @message "duration: * ms" as duree_ms
| sort duree_ms desc
| limit 10
```

Champs automatiques : `@timestamp` , `@message` , `@logStream` , `@log` .
Commandes principales : `fields` , `filter` , `parse` , `stats ... by` , `sort` ,
`limit` . La facturation dépend du volume analysé : réduire la plage de temps.

Le principe du traçage distribué

Une requête reçoit un **identifiant de trace**, propagé d'un service à l'autre dans un en-tête. Chaque service enregistre un **segment** (début, fin, erreur) et des **sous-segments** pour ses appels sortants (DynamoDB, RDS, HTTP).



La trace montre immédiatement que le temps est passé dans l'appel SQL, pas dans le code de la fonction.

Échantillonnage

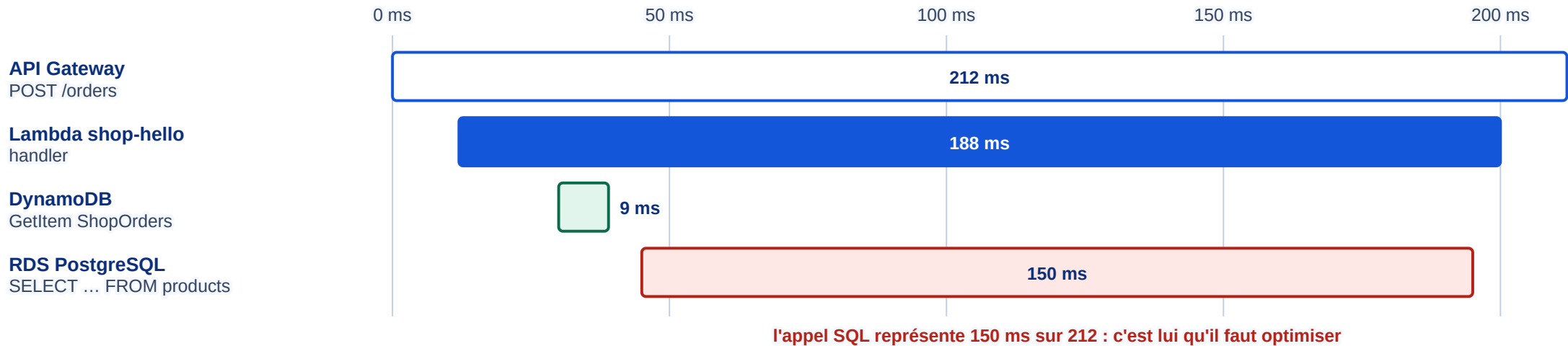
Par défaut : la première requête de chaque seconde, puis 5 % des suivantes. Des règles ajustent le taux par route ou par service.

Service	Activer le traçage
Lambda	Option <i>Active tracing</i> (<code>--tracing-config Mode=Active</code>) et droit <code>AWSXRayDaemonWriteAccess</code> sur le rôle.
ECS	Conteneur annexe : collecteur OpenTelemetry (ADOT) ou agent CloudWatch.
EKS	Add-on <code>amazon-cloudwatch-observability</code> ou <code>adot</code> .
API Gateway, ALB	Option de traçage sur le stage ; l'ALB propage l'en-tête de trace.

Application Signals

- Instrumentation automatique (OpenTelemetry) sans modifier le code, pour Java, Python, Node.js, .NET.
- Tableau de bord par service : volume, latence, erreurs, dépendances.
- SLO** : objectif de niveau de service (99,9 % des requêtes en moins de 300 ms) et alarmes sur le budget d'erreur.

OpenTelemetry est le standard d'instrumentation à privilégier : AWS Distro for OpenTelemetry (ADOT) envoie vers X-Ray, CloudWatch ou Prometheus.



Lecture

Chaque barre est un segment : son début, sa durée, le service concerné. Les barres imbriquées sont les appels sortants du segment au-dessus.

Diagnostic

La fonction dure 188 ms, dont 150 ms d'attente de la base. Augmenter la mémoire de Lambda n'y changerait rien : il faut un index SQL ou un cache.

Ce que les métriques ne disent pas

La métrique `Duration` aurait montré une fonction lente, sans dire pourquoi. La trace relie la lenteur à sa cause.

Container Insights

- Métriques du cluster jusqu'au conteneur, tableaux de bord préconstruits.
- ECS : réglage `containerInsights=enhanced`.
- EKS : add-on `amazon-cloudwatch-observability`.

Lambda Insights

- Extension ajoutée à la fonction sous forme de layer.
- Mémoire réellement utilisée, CPU, réseau, démarrages à froid, arrêts anormaux.
- Vue multi-fonctions pour repérer les plus coûteuses.

Database Insights

- Charge de la base (sessions actives) découpée par requête SQL, attente, hôte, utilisateur.
- Mode standard inclus, mode avancé payant (historique long, plans d'exécution, verrous).
- Pour RDS et Aurora.

Contributor Insights (DynamoDB)

- Clés les plus lues, les plus écrites, les plus limitées.
- L'outil pour identifier une partition chaude.

Managed Service for Prometheus

- Stockage compatible Prometheus, requêtes PromQL, sans serveur à gérer.
- Collecte par scraper managé ou *remote write* depuis EKS et ECS.
- Choix naturel quand l'équipe utilise déjà Prometheus.

Managed Grafana

- Grafana managé, authentification par IAM Identity Center.
- Sources : CloudWatch, Prometheus, X-Ray, Athena, OpenSearch, sur plusieurs comptes et régions.

CloudWatch ou Prometheus pour EKS ? CloudWatch : intégré, rien à exploiter. Prometheus et Grafana : standard Kubernetes, portabilité. Beaucoup d'équipes combinent les deux.

CloudTrail : le journal d'audit

- Enregistre les appels d'API : identité, heure, adresse IP source, paramètres, résultat.
- **Management events** : opérations de gestion (créer, modifier, supprimer). Historique de 90 jours consultable gratuitement (*Event history*).
- **Data events** : accès aux données (objets S3, invocations Lambda, items DynamoDB). À activer, payants.
- **Trail** : livraison continue vers S3 (et CloudWatch Logs) pour conserver au-delà de 90 jours. **CloudTrail Lake** : requêtes SQL sur les événements.

Exemples. Qui a supprimé la base ? Qui a ouvert le port 22 ? Quelle clé d'accès a été utilisée depuis une IP inconnue ?

EventBridge : réagir aux événements

- Les services AWS publient leurs changements d'état sur un **bus d'événements**.
- Une **règle** filtre par motif JSON et envoie vers des **cibles** : SNS, Lambda, SQS, Step Functions.
- **Scheduler** : déclenchements planifiés (cron, date unique).

`{ }` Motif : une tâche ECS de shop-cluster s'est arrêtée

```
{ "source": ["aws.ecs"],
  "detail-type": ["ECS Task State Change"],
  "detail": {
    "lastStatus": ["STOPPED"],
    "clusterArn": [ { "suffix": ":cluster/shop-cluster" } ]
  } }
```

Autres événements utiles : bascule RDS, échec de déploiement ECS, changement d'état d'une alarme CloudWatch, événement AWS Health. **AWS Config** complète l'ensemble en historisant la configuration des ressources et en évaluant des règles de conformité.

aws

CloudWatch > Alarms > All alarms > Create alarm

Step 1 : Specify metric and conditions

Select metric

Lambda > By Function Name > shop-hello > Errors

Statistic / Period

Sum / 1 minute ▼

Whenever Errors is

☒ Greater/Equal than 1

Datapoints to alarm

2 out of 3

Missing data treatment

Treat missing data as good (not breaching) ▼

Step 2 : Configure actions

Send a notification to

Create new topic : shop-alerts

Step 3 : Add name and description

Alarm name

shop-hello-errors

Create alarm

L'abonnement e-mail au topic SNS doit être **confirmé** en cliquant sur le lien reçu, sinon aucune notification n'arrive.

aws

CloudWatch > Logs > Logs Insights

Query language

Logs Insights QL ▼

Select log groups

/aws/lambda/shop-hello ▼

Time range

1h ▼

Query

filter @type = "REPORT" | stats avg(@duration) by bin(5m)

Run query

Menu CloudWatch	Usage
Dashboards	Tableaux de bord personnalisés ou automatiques par service.
Metrics > All metrics	Explorer par namespace, tracer, appliquer du metric math.
Logs > Log groups	Rétention, filtres de métriques, abonnements, Live Tail.
Application Signals	Services, SLO, carte des traces.
Insights	Container Insights, Lambda Insights, Database Insights.

\$ Prérequis : variables de l'étape 0 ; la fonction shop-hello et la base shop-pg existent encore (sinon les alarmes restent en INSUFFICIENT_DATA)

```
# 1. Topic SNS et abonnement e-mail (confirmer en cliquant sur le lien reçu)
export TOPIC_ARN=$(aws sns create-topic --name shop-alerts --query TopicArn --output text)
printf "Adresse e-mail qui recevra les alertes : "; read ALERT_EMAIL
aws sns subscribe --topic-arn $TOPIC_ARN --protocol email --notification-endpoint "$ALERT_EMAIL"

# 2. Alarme Lambda : au moins 1 erreur par minute, sur 2 des 3 dernières minutes
aws cloudwatch put-metric-alarm --alarm-name shop-hello-errors \
  --namespace AWS/Lambda --metric-name Errors --dimensions Name=FunctionName,Value=shop-hello \
  --statistic Sum --period 60 --evaluation-periods 3 --datapoints-to-alarm 2 \
  --threshold 1 --comparison-operator GreaterThanOrEqualToThreshold \
  --treat-missing-data notBreaching --alarm-actions $TOPIC_ARN --ok-actions $TOPIC_ARN

# 3. Alarme RDS : CPU moyen au-dessus de 80 % pendant 10 minutes
aws cloudwatch put-metric-alarm --alarm-name shop-pg-cpu \
  --namespace AWS/RDS --metric-name CPUUtilization --dimensions Name=DBInstanceIdentifier,Value=shop-pg \
  --statistic Average --period 300 --evaluation-periods 2 \
  --threshold 80 --comparison-operator GreaterThanThreshold --alarm-actions $TOPIC_ARN

# 4. Alarme composite : ne prévenir que si les deux alarmes sont actives en même temps
aws cloudwatch put-composite-alarm --alarm-name shop-degradation \
  --alarm-rule 'ALARM("shop-hello-errors") AND ALARM("shop-pg-cpu")' --alarm-actions $TOPIC_ARN

# 5. Tester la chaîne de notification sans provoquer de panne, puis lister les états
aws cloudwatch set-alarm-state --alarm-name shop-hello-errors \
  --state-value ALARM --state-reason "Test de la notification"
aws cloudwatch describe-alarms --query 'MetricAlarms[][AlarmName,StateValue]' --output table
aws cloudwatch describe-alarm-history --alarm-name shop-hello-errors --max-records 5 \
  --query 'AlarmHistoryItems[][Timestamp,HistorySummary]' --output table
```

\$ Lire et publier des métriques

```
# Métriques disponibles pour la fonction
aws cloudwatch list-metrics --namespace AWS/Lambda \
  --dimensions Name=FunctionName,Value=shop-hello \
  --query 'Metrics[].MetricName'

# Durée moyenne et maximale, par minute, sur 1 heure
# (la CLI accepte les dates en secondes epoch)
aws cloudwatch get-metric-statistics \
  --namespace AWS/Lambda --metric-name Duration \
  --dimensions Name=FunctionName,Value=shop-hello \
  --start-time $(( $(date +%s) - 3600 )) \
  --end-time $(date +%s) \
  --period 60 --statistics Average Maximum \
  --output table --query \
  'sort_by(Datapoints,&Timestamp)[].[Timestamp,Average,Maximum]'

# Métrique métier personnalisée
aws cloudwatch put-metric-data --namespace Shop/App \
  --metric-name OrdersCreated --value 3 --unit Count \
  --dimensions Env=dev

# La retrouver (délai d'apparition : 1 à 2 minutes)
aws cloudwatch list-metrics --namespace Shop/App
```

\$ Tableau de bord décrit en JSON

```
ALARMS=arn:aws:cloudwatch:$AWS_REGION:$AWS_ACCOUNT_ID:alarm
cat > dashboard.json <<EOF
{"widgets":[
  {"type":"metric","x":0,"y":0,"width":12,"height":6,
    "properties":{"title":"Lambda shop-hello",
      "region":"${AWS_REGION}","stat":"Sum","period":60,
      "metrics":[
        ["AWS/Lambda","Invocations","FunctionName","shop-hello"],
        [".","Errors",".","."],
        [".","Throttles",".","."]]}},
  {"type":"metric","x":12,"y":0,"width":12,"height":6,
    "properties":{"title":"RDS shop-pg",
      "region":"${AWS_REGION}","stat":"Average","period":300,
      "metrics":[
        ["AWS/RDS","CPUUtilization","DBInstanceIdentifier","shop-pg"],
        [".","DatabaseConnections",".","."]]}},
  {"type":"alarm","x":0,"y":6,"width":24,"height":3,
    "properties":{"title":"Alarmes UtopiosShop","alarms":[
      "${ALARMS}:shop-hello-errors","${ALARMS}:shop-pg-cpu"]}
  }}
EOF
aws cloudwatch put-dashboard \
  --dashboard-name shop-overview \
  --dashboard-body file://dashboard.json
aws cloudwatch list-dashboards \
  --query 'DashboardEntries[].DashboardName'
```

```
$ Prérequis : la fonction shop-hello a été invoquée au moins une fois (son groupe de logs existe)

# 1. Groupes de logs et rétention actuelle (null = conservation illimitée)
aws logs describe-log-groups --query 'logGroups[][logGroupName,retentionInDays,storedBytes]' --output table

# 2. Fixer la rétention à 30 jours
aws logs put-retention-policy --log-group-name /aws/lambda/shop-hello --retention-in-days 30

# 3. Suivre les logs en direct, ou chercher un motif sur la dernière heure
aws logs tail /aws/lambda/shop-hello --follow --format short
aws logs tail /aws/lambda/shop-hello --since 1h --filter-pattern "ERROR"

# 4. Filtre de métrique : chaque ligne contenant ERROR incrémente Shop/App AppErrors
aws logs put-metric-filter --log-group-name /aws/lambda/shop-hello \
  --filter-name shop-hello-errors --filter-pattern "ERROR" \
  --metric-transformations metricName=AppErrors,metricNamespace=Shop/App,metricValue=1,defaultValue=0

# 5. Logs Insights : lancer une requête, attendre, lire le résultat
QUERY_ID=$(aws logs start-query --log-group-name /aws/lambda/shop-hello \
  --start-time $(( $(date +%s) - 3600 )) --end-time $(date +%s) \
  --query-string 'filter @type = "REPORT" | stats count() as invocations, avg(@duration) as moyenne_ms' \
  --query queryId --output text)
sleep 5
aws logs get-query-results --query-id $QUERY_ID --query '{etat:status,resultat:results}'

# 6. Journaux PostgreSQL de RDS (activés au module RDS par l'export vers CloudWatch Logs)
aws logs tail /aws/rds/instance/shop-pg/postgresql --since 30m
```

`aws logs tail` n'existe qu'en AWS CLI v2. Sur un filtre de métrique JSON, le motif s'écrit `{ $.level = "ERROR" }` : il ne compte que les lignes dont le champ `level` vaut `ERROR`.

\$ CloudTrail : qui a fait quoi

```
# Qui a créé des instances RDS (90 derniers jours) ?
aws cloudtrail lookup-events --max-results 5 \
  --lookup-attributes \
  AttributeKey=EventName,AttributeValue=CreatedDBInstance \
  --query 'Events[].[EventTime,Username,EventName]' \
  --output table

# Mes dix dernières actions sur le compte
ME=$(aws sts get-caller-identity --query Arn --output text)
aws cloudtrail lookup-events --max-results 10 \
  --lookup-attributes \
  AttributeKey=Username,AttributeValue=${ME##*/} \
  --query 'Events[].[EventTime,EventSource,EventName]' \
  --output table

# Détail complet d'un événement (IP source, paramètres)
aws cloudtrail lookup-events --max-results 1 \
  --lookup-attributes \
  AttributeKey=EventName,AttributeValue=CreatedDBInstance \
  --query 'Events[0].CloudTrailEvent' --output text | jq .
```

Critères de recherche : `EventName` , `Username` , `ResourceName` , `EventSource` , `ReadOnly` . Un événement apparaît quelques minutes après l'appel.

\$ EventBridge : notifier chaque arrêt de tâche ECS

```
# Autoriser EventBridge et CloudWatch à publier sur le topic
cat > topic-policy.json <<EOF
{"Version":"2012-10-17","Statement":[{"Effect":"Allow",
  "Principal":{"Service":["events.amazonaws.com",
    "cloudwatch.amazonaws.com"]},
  "Action":"sns:Publish","Resource":"${TOPIC_ARN}"]}
EOF
aws sns set-topic-attributes --topic-arn $TOPIC_ARN \
  --attribute-name Policy \
  --attribute-value file://topic-policy.json

# Règle et cible
aws events put-rule --name shop-ecs-task-stopped \
  --event-pattern '{"source":["aws.ecs"],
  "detail-type":["ECS Task State Change"],
  "detail":{"lastStatus":["STOPPED"]}}'
aws events put-targets --rule shop-ecs-task-stopped \
  --targets "Id=sns,Arn=$TOPIC_ARN"

# Test : arrêter une tâche du service, un e-mail arrive
aws ecs stop-task --cluster shop-cluster --task \
  $(aws ecs list-tasks --cluster shop-cluster \
    --query 'taskArns[0]' --output text) \
  --query 'task.lastStatus'
```

Prérequis : topic `shop-alerts` créé (CLI 1/5). Le test suppose que le service `shop-web` tourne encore ; ECS relance aussitôt une tâche de remplacement.

\$ X-Ray : tracer la fonction shop-hello

```
POLICY=arn:aws:iam::aws:policy/AWSXRayDaemonWriteAccess
aws iam attach-role-policy --role-name shop-lambda-role \
  --policy-arn $POLICY
aws lambda update-function-configuration \
  --function-name shop-hello --tracing-config Mode=Active
aws lambda wait function-updated-v2 \
  --function-name shop-hello

# Générer un peu de trafic
for i in 1 2 3 4 5; do
  aws lambda invoke --function-name shop-hello \
    --cli-binary-format raw-in-base64-out \
    --payload '{}' /dev/null --query StatusCode
done

# Résumé des traces des 10 dernières minutes
sleep 30
aws xray get-trace-summaries \
  --start-time $(( $(date +%s) - 600 )) \
  --end-time $(date +%s) \
  --query 'TraceSummaries[][Id,Duration,HasError]' \
  --output table
```

Dans la console : CloudWatch, Application Signals, **Trace Map**. Chaque nœud de la carte montre la latence et le taux d'erreur ; un clic ouvre les traces détaillées.

\$ Container Insights sur ECS et EKS

```
# ECS : activer sur un cluster existant
aws ecs update-cluster-settings --cluster shop-cluster \
  --settings name=containerInsights,value=enhanced

# EKS : le rôle des nœuds doit pouvoir écrire dans CloudWatch
NODE_ROLE=$(aws eks describe-nodegroup \
  --cluster-name shop-eks --nodegroup-name ng-main \
  --query nodegroup.nodeRole --output text)
aws iam attach-role-policy --role-name ${NODE_ROLE##*/} \
  --policy-arn \
  arn:aws:iam::aws:policy/CloudWatchAgentServerPolicy

# EKS : installer l'add-on d'observabilité
aws eks create-addon --cluster-name shop-eks \
  --addon-name amazon-cloudwatch-observability
aws eks wait addon-active --cluster-name shop-eks \
  --addon-name amazon-cloudwatch-observability
kubectl get pods -n amazon-cloudwatch
```

Ce bloc suppose que `shop-cluster` et `shop-eks` existent encore. Avant de supprimer le cluster EKS avec `eksctl`, détacher `CloudWatchAgentServerPolicy` du rôle des nœuds, sinon la suppression du rôle échoue.

Résultat : CloudWatch, Insights, **Container Insights**. Les métriques apparaissent après quelques minutes, par cluster, service, tâche, namespace et pod.

Bonnes pratiques

- Partir des objectifs de service (disponibilité, latence) et en déduire alarmes et tableaux de bord.
- Alarmes peu nombreuses, actionnables, reliées à une procédure ; alarmes composites contre le bruit.
- Rétention définie sur tous les groupes de logs ; logs structurés en JSON.
- Identifiant de corrélation propagé partout (trace ID, request ID).
- Un trail CloudTrail multi-région vers un bucket protégé, dans un compte dédié.
- Alarmes, tableaux de bord et règles décrits en code, comme le reste.
- Tester l'alerte : une alarme jamais déclenchée n'est pas une alarme validée.

Maîtriser les coûts

- Postes principaux : ingestion des logs, métriques personnalisées, Container Insights renforcé, requêtes Logs Insights.
- Réduire le niveau de log en production, échantillonner les traces, limiter les dimensions.

\$ Nettoyage du module Monitoring

```
# Alarmes : la composite d'abord, puis les autres
aws cloudwatch delete-alarms --alarm-names shop-degradation
aws cloudwatch delete-alarms \
    --alarm-names shop-hello-errors shop-pg-cpu

# Tableau de bord
aws cloudwatch delete-dashboards \
    --dashboard-names shop-overview

# Filtre de métrique
aws logs delete-metric-filter \
    --log-group-name /aws/lambda/shop-hello \
    --filter-name shop-hello-errors

# Règle EventBridge : retirer la cible, puis la règle
aws events remove-targets \
    --rule shop-ecs-task-stopped --ids sns
aws events delete-rule --name shop-ecs-task-stopped

# Topic SNS (supprime aussi les abonnements)
aws sns delete-topic --topic-arn $TOPIC_ARN

# Politique X-Ray ajoutée au rôle Lambda
aws iam detach-role-policy --role-name shop-lambda-role \
    --policy-arn arn:aws:iam::aws:policy/AWSXRayDaemonWriteAccess
```

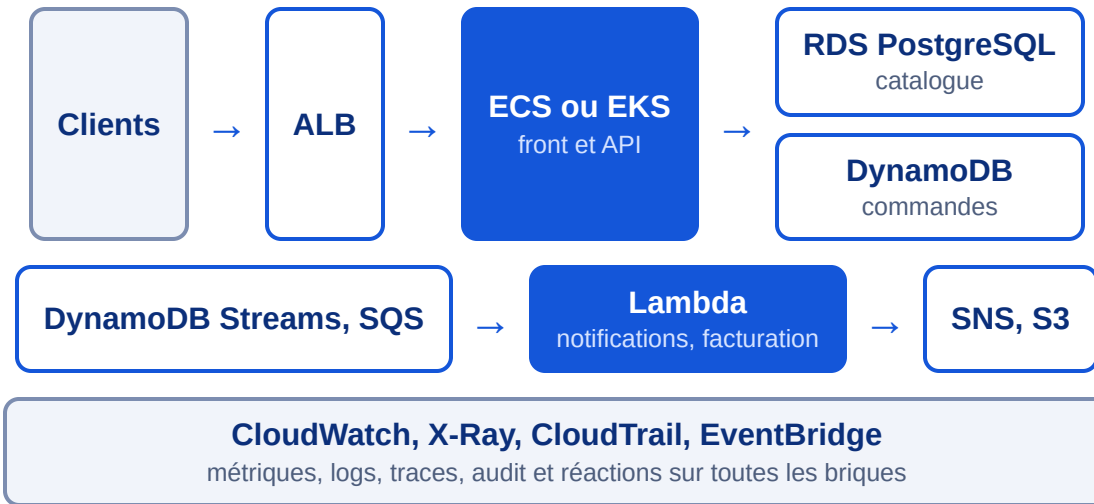
Synthèse

Synthèse : aide au choix

Besoin de calcul	Choix
Code court déclenché par des événements, trafic irrégulier	Lambda
API ou worker en conteneur, équipe AWS, peu d'exploitation	ECS sur Fargate
Conteneurs avec GPU ou coût optimisé à grande échelle	ECS sur EC2 ou Managed Instances
Standard Kubernetes, écosystème CNCF, hybride ou multi-cloud	EKS (Auto Mode pour alléger)
Traitement long, par lots, de plus de 15 minutes	ECS run-task, Batch, Step Functions

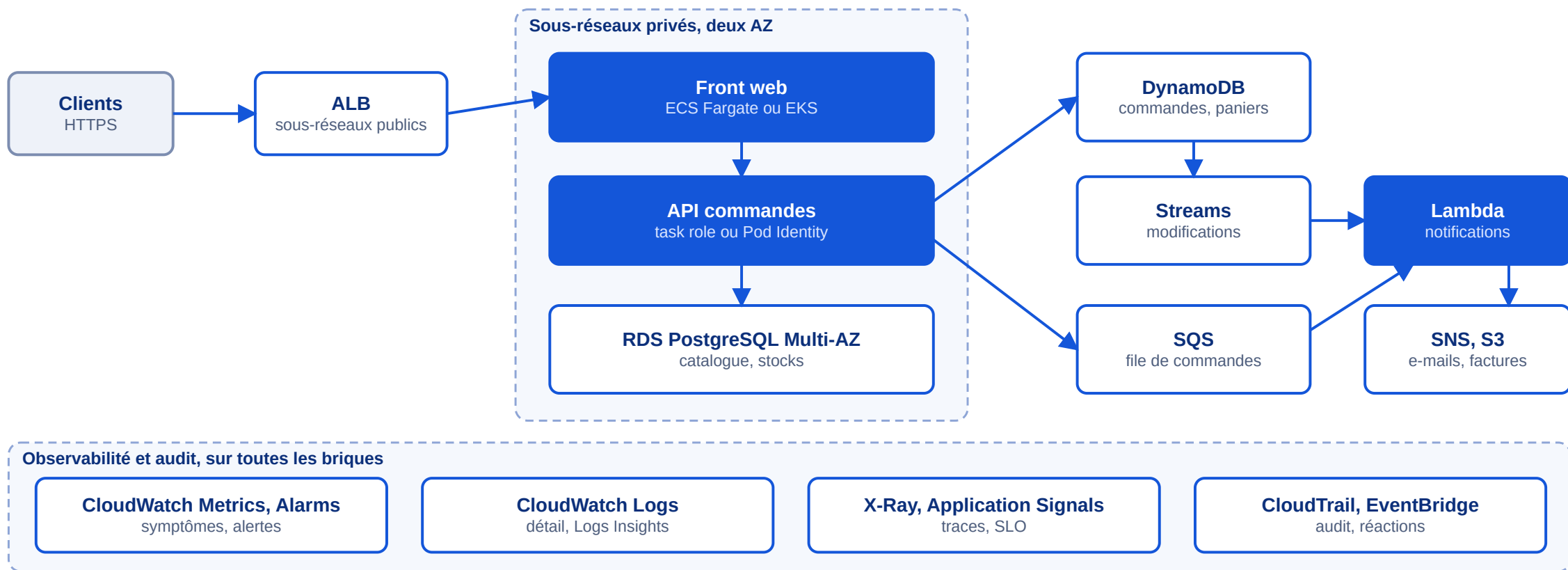
Besoin de données	Choix
Relations, jointures, reporting, transactions complexes	RDS ou Aurora
Charge SQL très variable ou intermittente	Aurora Serverless v2
Accès par clé, très fort débit, latence constante, serverless	DynamoDB
Lambda vers une base relationnelle	RDS Proxy en frontal

UtopiosShop, architecture d'ensemble



Fil conducteur. Dans chaque service : un rôle IAM au moindre privilège, un déploiement réparti sur plusieurs AZ, des logs avec une rétention, des alarmes sur les symptômes, et l'infrastructure décrite en code.

Schéma : architecture complète de UtopiosShop



Flux synchrone. Client, ALB, front, API, puis PostgreSQL pour le catalogue et DynamoDB pour la commande.

Flux asynchrone. La commande écrite dans DynamoDB ou déposée dans SQS déclenche Lambda, qui notifie et produit la facture.

Transverse. Un rôle IAM par brique, plusieurs AZ partout, et les mêmes outils d'observabilité sur l'ensemble.

\$ Prérequis : variables de l'étape 0 ; nettoyages des six modules déjà exécutés

```
# Rôle partagé créé au module ECS
aws iam detach-role-policy --role-name ecsTaskExecutionRole \
  --policy-arn arn:aws:iam::aws:policy/service-role/AmazonECSTaskExecutionRolePolicy
aws iam delete-role --role-name ecsTaskExecutionRole

# Contrôle : chaque commande doit renvoyer une liste vide (hors ressources étrangères au lab)
aws ecs list-clusters --query clusterArns
aws eks list-clusters --query clusters
aws lambda list-functions --query 'Functions[].FunctionName'
aws rds describe-db-instances --query 'DBInstances[].DBInstanceIdentifier'
aws rds describe-db-snapshots --snapshot-type manual --query 'DBSnapshots[].DBSnapshotIdentifier'
aws dynamodb list-tables --query TableNames
aws dynamodb list-backups --query 'BackupSummaries[].BackupName'
aws elbv2 describe-load-balancers --query 'LoadBalancers[].LoadBalancerName'
aws elb describe-load-balancers --query 'LoadBalancerDescriptions[].LoadBalancerName'
aws ec2 describe-nat-gateways --filter Name=state,Values=available --query 'NatGateways[].NatGatewayId'
aws ecr describe-repositories --query 'repositories[].repositoryName'
aws sqs list-queues --query QueueUrls
aws sns list-topics --query 'Topics[].TopicArn'
aws cloudwatch describe-alarms --query 'MetricAlarms[].AlarmName'
aws logs describe-log-groups --query 'logGroups[].logGroupName'
aws cloudformation list-stacks --stack-status-filter CREATE_COMPLETE UPDATE_COMPLETE DELETE_FAILED \
  --query 'StackSummaries[].StackName'
```

Groupes de logs restants. Container Insights, EKS et RDS en créent automatiquement. Les supprimer avec `aws logs delete-log-group --log-group-name` suivi du nom affiché.

Vérification par les coûts. Dans Cost Explorer, regrouper par service sur les derniers jours : un service encore facturé signale un oubli.

1. ECS : différence entre execution role et task role ?

2. Que fait le circuit breaker de déploiement ECS ?

3. EKS : que gère AWS, que gérez-vous ?

4. Pourquoi Pod Identity ou IRSA plutôt que le rôle du nœud ?

5. Lambda : qu'est-ce qu'un démarrage à froid, comment le limiter ?

6. 200 requêtes par seconde de 500 ms : quelle concurrence ?

7. RDS : Multi-AZ ou read replica ?

8. Que produit une restauration PITR ?

9. DynamoDB : pourquoi éviter Scan et FilterExpression ?

10. Coût en RCU d'une lecture fortement cohérente d'un item de 9 Ko ?

11. GSI ou LSI pour retrouver les commandes par statut ?

12. CloudWatch ou CloudTrail pour savoir qui a supprimé une base ?

Cliquer sur une question pour afficher la réponse.

Sujet	Documentation officielle
ECS	docs.aws.amazon.com/ecs
EKS	docs.aws.amazon.com/eks
Lambda	docs.aws.amazon.com/lambda
RDS et Aurora	docs.aws.amazon.com/rds
DynamoDB	docs.aws.amazon.com/dynamodb
CloudWatch	docs.aws.amazon.com/cloudwatch
CloudTrail	docs.aws.amazon.com/cloudtrail
AWS CLI v2	docs.aws.amazon.com/cli

Aide intégrée à la CLI. `aws ecs help`, `aws ecs create-service help` : syntaxe, options et exemples, toujours à jour avec la version installée.

Guides de référence

- **AWS Well-Architected Framework** : six piliers (excellence opérationnelle, sécurité, fiabilité, performance, coûts, durabilité).
- **EKS Best Practices Guide** : sécurité, réseau, mises à niveau, coûts.
- **ECS Best Practices Guide** : déploiement, réseau, auto scaling.
- **Powertools for AWS Lambda** : logs structurés, traces, métriques et idempotence prêts à l'emploi.
- **AWS Observability Best Practices** : recettes CloudWatch, OpenTelemetry, Prometheus.

Outils utiles

- `eksctl`, `kubect1`, `k9s` pour EKS.
- AWS SAM et CDK pour Lambda ; Terraform pour l'ensemble.
- NoSQL Workbench pour modéliser DynamoDB.
- AWS Pricing Calculator pour chiffrer une architecture.

Les services évoluent vite : quotas, tarifs et libellés de console sont à vérifier dans la documentation au moment de la formation.